QUICK REVISION EDITION

The Ultimate Q&A Interview Kit Playwright TypeScript 2026

Every question, distilled to the key points.
For fast, last-minute revision.

Sreenidhi Rajakrishnan · QA Automation Architect
sreenidhirajakrishnan.com

TypeScript Fundamentals for SDETs

Q 1 LEVEL · Junior

Why is TypeScript the preferred language for Playwright automation?

- Frame it as outcomes: autocomplete -> speed, types -> maintainability.
- Autocomplete from Playwright's type defs cuts real development time.
- tsc --noEmit as the first CI step catches type errors in seconds.
- Contract changes surface at compile time, not as silent runtime failures.
- Trap to avoid: stopping at 'it is type-safe', too junior.
- Follow-up to pre-empt: where you catch errors, name tsc and no-floating-promises.

```
// JavaScript: wrong types accepted silently
async function login(page, email, password) {
  await page.locator('#email').fill(email);
}
login(page, 12345, true); // no error; bug ships

// TypeScript: caught at compile time
import { Page } from '@playwright/test';
async function login(page: Page, email: string, password: string): Promise<void> {
  await page.getByLabel('Email').fill(email);
}
login(page, 12345, true); // error: number not assignable to string
```

Q 2 LEVEL · Junior to Mid

What is the difference between interfaces and types, and when do you use each?

- Rule: interfaces for extends/implements; types for unions and derivations.
- Define a response shape once; derive create/update with Omit and Partial.
- One interface change flags every affected test at compile time.
- Prevents silent drift when an API contract changes.
- The common miss: getting lost in declaration-merging trivia instead of showing impact.
- Asked next: a new required field, demonstrate the one-edit compiler fan-out.

```
interface BasePage {
  navigate(url: string): Promise<void>;
  waitForLoad(): Promise<void>;
}
interface LoginPage extends BasePage {
  login(credentials: UserCredentials): Promise<void>;
}

type Environment = 'dev' | 'staging' | 'production';

interface UserResponse {
  id: number; name: string; email: string; createdAt: string;
}
type CreateUserRequest = Omit<UserResponse, 'id' | 'createdAt'>;
type UpdateUserRequest = Partial<CreateUserRequest>;
```

Q 3 LEVEL · Junior

How do async/await work, and why does a missing await break a Playwright test?

Every Playwright action returns a Promise; await sequences them. Missing await runs the next line against an unsettled DOM, flakiness. Prevent, do not chase: no-floating-promises catches it at lint time. Promise.all: start waitForResponse BEFORE the click that triggers it. You can only capture a response you were already listening for. Follow-up: capturing a triggered API call, the Promise.all pattern answers it.

```
// Broken: nothing is awaited
test('broken', async ({ page }) => {
  page.goto('/login');          // fires, not awaited
  page.fill('#email', 'admin'); // race condition
});

// Correct: capture the API call the click triggers
const [response] = await Promise.all([
  page.waitForResponse('**/api/login'), // listen first
  page.getByRole('button', { name: 'Sign in' }).click(),
]);
expect(response.status()).toBe(200);
```

Q 4 LEVEL · Mid

What are TypeScript generics and how do you use them in a Playwright framework?

- Open on the typed API client, not an abstract definition.
- Name the alternatives avoided: function-per-type, or casting to any.
- One typed client serves every endpoint with a typed response.
- Response-shape changes light up every typed call site at compile time.
- Second example for the follow-up: a generic factory with Partial overrides.
- Avoid: reciting ‘a generic is a type parameter’ with no application.

```
import { APIRequestContext, expect } from '@playwright/test';

async function apiGet<TResponse>(
  request: APIRequestContext,
  endpoint: string,
): Promise<TResponse> {
  const response = await request.get(endpoint);
  expect(response.ok()).toBeTruthy();
  return (await response.json()) as TResponse;
}

interface User { id: number; name: string; email: string; }
const user = await apiGet<User>(request, '/users/1');
console.log(user.name); // typed: TS knows .name exists

function createTestData<T>(defaults: T, overrides?: Partial<T>): T {
  return { ...defaults, ...overrides };
}
```

Q 5 LEVEL · Junior to Mid

What are TypeScript enums and utility types, and how do they help in automation?

- Enums replace magic strings: no more 'admin' vs 'Admin' typos.
- Partial / Omit / Pick / Record derive shapes instead of duplicating.
- Record keyed by an enum enforces exhaustiveness on test data.
- Add a role to the enum and the compiler flags the missing map entry.
- The easy mistake: treating enums as cosmetic rather than as bug-class removal.
- Expect next: keeping test data in sync as the app grows, the exhaustive Record.

```
enum UserRole { Admin = 'admin', User = 'user', Viewer = 'viewer' }

interface User {
  id: number; name: string; email: string; role: UserRole;
}
type CreateUser = Omit<User, 'id'>;
type UserSummary = Pick<User, 'id' | 'name' | 'email'>;

// Record keyed by the enum: must cover every role
const credentials: Record<UserRole, { email: string; password: string }> = {
  [UserRole.Admin]:  { email: 'admin@test.com',  password: 'Admin@123' },
  [UserRole.User]:   { email: 'user@test.com',   password: 'User@123' },
  [UserRole.Viewer]: { email: 'viewer@test.com', password: 'Viewer@123' },
};
```

Q 6 LEVEL · Mid

What is TypeScript strict mode and why is it essential for automation projects?

- strict: true bundles strictNullChecks, noImplicitAny and more.
- strictNullChecks forces explicit null/undefined handling at compile time.
- Env vars are typed as possibly-undefined: must be validated.
- Anchor on a real incident, not a flag list.
- Trap: naming flags with no consequence attached.
- They usually ask next: ‘isn’t it annoying?’, cryptic crash becomes actionable compile error.

```
// Without strictNullChecks: crashes if undefined
const user = findUser(id); // User | undefined
console.log(user.name);    // runtime crash

// With strictNullChecks: guard required
const found = findUser(id);
if (!found) throw new Error(`User ${id} not found`);
console.log(found.name);   // safe: narrowed to User

// Fixtures: validate env vars explicitly
export const test = base.extend<{ adminEmail: string }>({
  adminEmail: async ({}, use) => {
    const email = process.env.ADMIN_EMAIL; // string | undefined
    if (!email) throw new Error('ADMIN_EMAIL is required');
    await use(email);
  },
});
```

Q 7 LEVEL · Junior to Mid

How do you configure path aliases and project structure for a large Playwright framework?

Aliases map a prefix to a folder: imports become location-independent. Refactor-safe: moving files does not break imports. Self-documenting: a prefix always means one kind of file. Centralise interfaces/enums in a types directory. Senior detail: resolve the alias at run time too, not just in the editor. Where this goes wrong: describing folder layout without the refactor-safety reason.

```
// tsconfig.json
{
  "compilerOptions": {
    "strict": true,
    "baseUrl": ".",
    "paths": {
      "@pages/*":    ["src/pages/*"],
      "@fixtures/*": ["src/fixtures/*"],
      "@data/*":     ["src/test-data/*"],
      "@types/*":    ["src/types/*"]
    }
  }
}

// fragile: import { LoginPage } from '../../../src/pages/auth/LoginPage';
import { LoginPage } from '@pages/auth/LoginPage'; // stable
```

Q 8 LEVEL · Mid

How do you configure ESLint for a Playwright TypeScript project?

- eslint-plugin-playwright + typescript-eslint = framework + type-aware rules.
- no-floating-promises flags missing await before CI runs.
- prefer-web-first-assertions: isVisible is one-shot, toBeVisible retries.
- no-focused-tests stops a stray test.only reaching main.
- The next question: stop ignored warnings by failing the build on any warning.
- 2026 default is flat config; trap is reciting a config with no rationale.

```
// eslint.config.ts (flat config, 2026 default)
import playwright from 'eslint-plugin-playwright';
import tseslint from 'typescript-eslint';

export default tseslint.config(
  ...tseslint.configs.recommendedTypeChecked,
  {
    files: ['**/*.ts'],
    plugins: { playwright },
    rules: {
      '@typescript-eslint/no-floating-promises': 'error',
      'playwright/no-wait-for-timeout': 'warn',
      'playwright/no-focused-tests': 'error',
      'playwright/prefer-web-first-assertions': 'error',
    },
  },
);
```

Q 9 LEVEL · Mid

How do you structure TypeScript types for a large framework?

- Central types directory, split by domain, with a barrel export.
- Shared generic shapes (API response, pagination) live there too.
- One edit + compiler shows every test that must adapt.
- Without it, breaking changes surface only as scattered runtime failures.
- What loses the point: framing it as tidiness instead of change-safety.
- Asked next: a breaking API change across tests, the central types answer.

```
// src/types/user.types.ts
export enum UserRole { Admin = 'admin', User = 'user', Viewer = 'viewer' }
export interface UserCredentials { email: string; password: string; }
export interface UserProfile extends UserCredentials {
  id: number; firstName: string; lastName: string; role: UserRole;
}

// src/types/api.types.ts
export interface APIResponse<T> { data: T; status: number; message: string; }
export interface PaginatedResponse<T> extends APIResponse<T[]> {
  total: number; page: number; pageSize: number;
}

// src/types/index.ts: barrel export
export * from './user.types';
export * from './api.types';
```

Q 10 LEVEL · Mid

How do you handle TypeScript compilation in CI pipelines?

- tsc --noEmit type-checks without output and fails in seconds.
- Order: typecheck -> lint -> install browsers -> run tests.
- Quantify: seconds via tsc vs minutes-per-shard if it slips through.
- Shift-left for type safety: catch errors at the cheapest stage.
- Follow-up: package scripts the workflow calls keep local and CI identical.
- The common miss: 'I run tsc in CI' with no ordering or cost rationale.

```
// package.json
{
  "scripts": {
    "typecheck": "tsc --noEmit",
    "lint": "eslint . --max-warnings 0",
    "test": "playwright test",
    "ci": "npm run typecheck && npm run lint && npm run test"
  }
}

# .github/workflows: cheapest checks gate the expensive ones
- run: npm run typecheck   # fails fast, no browser needed
- run: npm run lint
- run: npx playwright install --with-deps chromium
- run: npm run test -- --shard=${{ matrix.shard }}/${{ matrix.total }}
```

Q 11 LEVEL · Mid to Senior

What is the satisfies operator, and when do you use it instead of `as` or a type annotation?

satisfies validates structure WITHOUT widening literal types. Type annotation widens; `as` silences; satisfies preserves narrow types. Use on config objects, test data, tool-argument shapes, keeps literal info. as is appropriate ONLY for external untyped data after a type guard. Expect next: as for narrowed JSON.parse output is fine; never as default. Avoid: using `as` because it compiles, it silences real errors.

```
// Three-way comparison
type ProjectConfig = { name: string; viewport: { width: number; height: number } };

// 1. Type annotation: widens, loses literal
const a: ProjectConfig = { name: 'chromium', viewport: { width: 1280, height: 720 } };
// a.name is `string`, not `'chromium'`: can't use in literal switch

// 2. `as`, dangerous, silences errors
const b = { name: 'chromium', viewprt: { width: 1280, height: 720 } } as ProjectConfig;
// typo `viewprt` compiles: production bug

// 3. `satisfies`: validates structure, keeps narrow types
const c = { name: 'chromium', viewport: { width: 1280, height: 720 } } satisfies ProjectConfig;
// c.name is the literal 'chromium': can drive exhaustive switches
// typo `viewprt` would fail compilation: real safety
```

Q 12 LEVEL · Mid to Senior

What are discriminated unions, and how do you use them to model test scenarios cleanly?

- Union of object types sharing a literal discriminator field.
- Compiler narrows the union when you check the discriminator.
- Each variant has DIFFERENT fields available: not all optional.
- Compiler enforces a state machine: impossible states become impossible.
- They usually ask next: in the 'failed' branch, error/stack exist with no ambiguity.
- The easy mistake: flat shape with optional fields, every access is undefined-prone.

```
// Discriminated union for test results
type TestResult =
  | { status: 'passed';  duration: number }
  | { status: 'failed';  error: string; stack: string }
  | { status: 'skipped'; reason: string };

function report(result: TestResult): string {
  switch (result.status) {
    case 'passed':
      return `✓ ${result.duration}ms`;          // duration exists here
    case 'failed':
      return `✗ ${result.error}\n${result.stack}`; // error + stack exist here
    case 'skipped':
      return `- ${result.reason}`;               // only reason exists here
  }
  // No need for `result.error?`: the compiler narrows automatically
}
```

Q 13 LEVEL · Mid

What are type guards and how do you use them to safely narrow `unknown` to a specific type?

- Type predicate `obj is SomeType`: compiler narrows on true return.
- Pattern: typeof check -> null check -> cast to Record -> validate each field.
- Narrows unknown safely; the only correct way to handle external data.
- NEVER use `as` on external data without a type guard first.
- The next question: use everywhere external data enters, APIs, JSON, postMessage.
- Trap: `as` silences the compiler but not the runtime, errors crash later.

```
interface ApiUser { id: number; email: string; isAdmin: boolean; }

function isApiUser(obj: unknown): obj is ApiUser {
  if (typeof obj !== 'object' || obj === null) return false;
  const u = obj as Record<string, unknown>;
  return typeof u.id === 'number'
      && typeof u.email === 'string'
      && typeof u.isAdmin === 'boolean';
}

// At the boundary: API response
const raw: unknown = await response.json();
if (!isApiUser(raw)) throw new Error(`Malformed user response: ${JSON.stringify(raw)}`);
// raw is now narrowed to ApiUser: every field safely typed
console.log(raw.email.toLowerCase()); // safe
```

Q 14 LEVEL · Senior

What are branded (nominal) types and how do they prevent ID-mixup bugs?

- TypeScript is structural: UserId and OrderId as `string` are interchangeable.
- Branded type: `type UserId = string & { __brand: 'UserId' }`.
- Phantom intersection: distinct types at compile time.
- ZERO runtime cost: brand exists only at the type level.
- Asked next: at runtime it's still a string; the safety is compile-time-only.
- Where this goes wrong: leaving domain IDs as raw `string`, ID-mixup bugs ship undetected.

```
// Brand each domain ID: zero runtime cost
type UserId    = string & { readonly __brand: 'UserId' };
type OrderId   = string & { readonly __brand: 'OrderId' };
type TenantId  = string & { readonly __brand: 'TenantId' };

// Constructor functions create the branded type from raw strings
const UserId   = (s: string) => s as UserId;
const OrderId  = (s: string) => s as OrderId;

// Domain API takes the right type
async function deleteOrder(id: OrderId) { /* ... */ }

const user  = UserId('u_123');
const order = OrderId('o_789');

await deleteOrder(order); // OK
await deleteOrder(user);  // ✗ Error: UserId is not OrderId
                          //   compiler catches the bug at the call site
```

Q 15 LEVEL · Senior

What are mapped types, and how do you use key remapping to derive types from existing ones?

- Mapped type: { [K in keyof T]: F<T[K]> }, transforms keys/values.
- Pick, Omit, Partial, Required, Readonly are all built on this.
- Key remapping (TS 4.1+): `as` inside brackets renames keys.
- Combine with template literals: derive `getName` from `name`, etc.
- Follow-up: derive POM getters, builder types, mock types from one source.
- What loses the point: manually-duplicated interfaces drift; derived types update automatically.

```
// Derive a getter interface from a data interface via key remapping
interface User { id: string; name: string; email: string; isAdmin: boolean; }

type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => Promise<T[K]>
};

type UserGetters = Getters<User>;
// equivalent to:
//   interface UserGetters {
//     getId():       Promise<string>;
//     getName():     Promise<string>;
//     getEmail():    Promise<string>;
//     getIsAdmin():  Promise<boolean>;
//   }

// Add a User field -> UserGetters updates automatically, single source of truth

// Make every field optional (Partial)
type PartialUser = { [K in keyof User]?: User[K] };
...
```

Q 16 LEVEL · Senior

What are template literal types, and how can they make selectors and routes type-safe?

- Backtick syntax mirrors runtime template literals, same intuition.
- Combine with Uppercase/Lowercase/Capitalize for rules on strings.
- Encode protocol in the type system: not just in CONTRIBUTING.md.
- Use cases: **typed test-ID selectors**, **route patterns**, **event-handler names**.
- Expect next: typed selectors catch typos before the test runs.
- The common miss: enforcing string conventions with comments and hoping reviewers catch.

```
// Typed test-ID selectors
type TestIdSelector = `[data-test="${string}"]`;
function byTestId(s: TestIdSelector) { /* ... */ }

byTestId('[data-test="submit"]');  // OK
byTestId('#submit');                // ✗ not a TestIdSelector

// Typed route patterns (TS 4.5+ supports template literal pattern matching well)
type UserOrderRoute = `/users/${number}/orders`;
const valid:   UserOrderRoute = `/users/${42}/orders`;   // OK
// const wrong: UserOrderRoute = `/users/abc/orders`;    // ✗ 'abc' is not number

// Derived event-handler names
type EventName = 'click' | 'submit' | 'change';
type Handlers = { [E in EventName as `on${Capitalize<E>}`]: () => void };
// -> { onClick: () => void; onSubmit: () => void; onChange: () => void }
```

Q 17 LEVEL · Senior

What is the `never` type, and how does it enforce exhaustive switch statements at compile time?

never = the type of values that can never occur, narrows-to-empty. In default branch of a switch, assign discriminator to a `never` variable. All cases handled -> narrows to never -> compiles; missing case -> fails build. Adding a new union variant later forces handling, compile-time enforced. They usually ask next: never is the only type that captures 'nothing should be here'. Avoid: **runtime default that throws**, **finds bug in prod**, **not on dev's machine**.

```
type TestResult =
  | { status: 'passed'  }
  | { status: 'failed'  }
  | { status: 'skipped' };

function assertExhaustive(value: never): never {
  throw new Error(`Unhandled variant: ${JSON.stringify(value)}`);
}

function report(r: TestResult): string {
  switch (r.status) {
    case 'passed':  return '✓';
    case 'failed':  return '✗';
    case 'skipped': return '-';
    default:        return assertExhaustive(r);  // compiles only if all cases handled
  }
}

// Adding `| { status: 'blocked' }` to TestResult, build FAILS until report() handles it
```

Q 18 LEVEL · Senior

What is declaration merging, and how does it let you extend Playwright's `test` and `expect` cleanly?

Declaration merging: multiple declarations with the same name combine. test.extend uses inference: no manual merging needed for fixtures. Use module augmentation for custom matchers, TestInfo fields, config. Pattern: `declare module '@playwright/test'` + extend the interface. Put all augmentations in ONE file (types/playwright.d.ts) for discoverability. The next question: custom matcher needs BOTH expect.extend AND Matchers augmentation. Matchers<R, T> returns R, not Promise<R>; and never shadow a built-in matcher name.

```
// types/playwright.d.ts: ONE place for all augmentations
export {};

declare global {
  namespace PlaywrightTest {
    // 1. Custom matcher. Note the return type is R, not Promise<R>
    interface Matchers<R, T> {
      toHaveRowCount(expected: number, options?: { timeout?: number }): R;
    }
  }
}
// 2. Custom TestInfo field
declare module '@playwright/test' {
  interface TestInfo { jiraTicket?: string; }
}

// fixtures/expect.ts: register the implementation, export the wrapped expect
import { expect as baseExpect, Locator } from '@playwright/test';

export const expect = baseExpect.extend({
...
```

QUICK QUIZ

1. Which command type-checks the whole project without producing output files?
   A) tsc --noEmit
   B) tsc --build
   C) tsc --watch
   D) tsc --strict
2. Which ESLint rule catches a missing await, the most common Playwright bug?
   A) @typescript-eslint/no-explicit-any
   B) @typescript-eslint/no-floating-promises
   C) playwright/no-focused-tests
   D) playwright/no-wait-for-timeout
3. You define UserResponse once and want a create-request type without id and createdAt. Which utility type?
   A) Partial<UserResponse>
   B) Pick<UserResponse, 'id'>
   C) Readonly<UserResponse>
   D) Omit<UserResponse, 'id' | 'createdAt'>
4. What does a Record keyed by an enum give you that a plain object does not?
   A) Compile-time exhaustiveness over every enum member
   B) Faster property access
   C) Automatic JSON serialisation
   D) Runtime validation of values
5. Why must waitForResponse be started before the click that triggers the request?
   A) Because clicks are synchronous
   B) Because the network is slower than the UI
   C) Because Playwright can only capture a response it was already listening for
   D) Because await reorders the calls automatically

Answers: 1:A 2:B 3:D 4:A 5:C

Playwright Architecture and Setup

Q 19 LEVEL · Mid to Senior

Explain Playwright's internal architecture in depth.

Persistent bidirectional WebSocket; CDP for Chromium/Edge. Classic Selenium is stateless HTTP: a new request per command. Persistent channel cuts latency and enables real-time event subscriptions. Interception/trace/auto-wait exist because you can listen continuously. Currency: Selenium 4 has CDP, and 4.18+ has WebDriver BiDi interception over a WebSocket. So the honest gap is uniformity and maturity, not 'Selenium cannot intercept'. What loses the point: listing features without the architectural cause.

```
// One persistent connection: subscribe to events in real time
page.on('console', msg => console.log('Browser log:', msg.text()));
page.on('requestfailed', req => console.log('Failed:', req.url()));
page.on('dialog', dialog => dialog.accept());

// Listen for traffic, then act: only possible on an open channel
page.on('response', res => {
  if (res.url().includes('/api/') && !res.ok()) {
    console.log('API error', res.status(), res.url());
  }
});
```

Q 20 LEVEL · Mid

What is the difference between Browser, BrowserContext, and Page?

Browser: the process, expensive, shared across a worker's tests. Context: isolated session (cookies/storage/permissions), fresh per test. Page: a tab inside a context where interactions happen. Two contexts in one test drive a real two-actor workflow. Pre-empt this: synchronise the actors with Promise.all. The common miss: listing definitions without the multi-context use case.

```
test('admin approves a user request', async ({ browser }) => {
  const adminCtx = await browser.newContext({ storageState: 'auth/admin.json' });
  const userCtx  = await browser.newContext({ storageState: 'auth/user.json' });
  const adminPage = await adminCtx.newPage();
  const userPage  = await userCtx.newPage();

  await userPage.goto('/requests/new');
  await userPage.getByRole('button', { name: 'Submit Request' }).click();

  await adminPage.goto('/admin/pending-requests');
  await adminPage.getByRole('button', { name: 'Approve' }).click();

  await expect(userPage.getByTestId('request-status')).toHaveText('Approved');
  await Promise.all([adminCtx.close(), userCtx.close()]);
});
```

Q 21 LEVEL · Mid

Walk through a production-grade playwright.config.ts.

Defend three decisions, don't narrate the whole file. forbidOnly tied to CI fails the build on a stray test.only. trace: 'on-first-retry', full trace when it matters, no storage waste. Deliberate actionTimeout surfaces regressions instead of hiding them. Auth belongs in a setup project with dependencies, not in globalSetup. The next question: why not trace:**on + big timeout?**, **storage cost**, **hidden regressions**. Avoid: reciting every field with no rationale.

```
import { defineConfig, devices } from '@playwright/test';
import dotenv from 'dotenv';
dotenv.config({ path: `.env.${process.env.TEST_ENV || 'staging'}` });

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 4 : undefined,
  reporter: [['html', { open: 'never' }], ['junit', { outputFile: 'results/junit.xml' }], ['github']],
  use: {
    baseURL: process.env.BASE_URL,
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    actionTimeout: 15_000,
    testIdAttribute: 'data-testid',
  },
  projects: [
    // auth once, inside the runner, so a broken login gets a trace
...
```

Q 22 LEVEL · Mid

How does globalSetup work and what should you put in it?

Runs once before any worker starts, and OUTSIDE the runner: no fixtures, no trace. Keep it for browser-less env work: **health check**, **DB seed**, **mock server**, **env vars**. Health-check the environment first; abort fast if it is down. Auth moved OUT: login belongs in a setup project with dependencies: ['setup']. Same storageState saving, but a broken login now gets a trace and a report line. The easy mistake: reciting the old login-in-globalSetup answer, it reads a version behind.

```
// global.setup.ts, browser-less environment work only
import { FullConfig } from '@playwright/test';

export default async function globalSetup(config: FullConfig) {
  const baseURL = config.projects[0].use.baseURL!;
  const health = await fetch(`${baseURL}/health`);
  if (!health.ok) throw new Error(`Environment ${baseURL} is not healthy.`);
}

// auth.setup.ts, a REAL test, so a broken login gets a trace
import { test as setup } from '@playwright/test';

for (const role of ['admin', 'user', 'viewer'] as const) {
  setup(`authenticate as ${role}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(process.env[`${role.toUpperCase()}_EMAIL`]!);
    await page.getByLabel('Password').fill(process.env[`${role.toUpperCase()}_PASSWORD`]!);
    await page.getByRole('button', { name: 'Sign in' }).click();
    await page.waitForURL('**/dashboard');
    await page.context().storageState({ path: `.auth/${role}.json` });
...
```

Q 23 LEVEL · Junior to Mid

How do you manage multiple environments in Playwright?

dotenv + TEST_ENV selects the environment file. Commit URLs and structure; never commit credentials. Passwords are CI secrets injected at runtime. Guard BASE_URL early with a clear error to avoid confusing mass failures. Asked next: secrets in CI, injection; restrict prod to smoke on one browser. Trap: implying credentials live in the committed .env files.

```
// playwright.config.ts
import dotenv from 'dotenv';
dotenv.config({ path: `.env.${process.env.TEST_ENV || 'staging'}` });

// package.json
{
  "scripts": {
    "test:staging":    "TEST_ENV=staging playwright test",
    "test:production": "TEST_ENV=production playwright test --project=chromium --grep @smoke",
    "test:smoke":      "TEST_ENV=staging playwright test --grep @smoke"
  }
}
```

Q 24 LEVEL · Junior

How does Playwright manage browser downloads and versions?

Playwright pins and downloads browsers per release, no driver mgmt. Upgrading Playwright pulls compatible browsers automatically. npx playwright install; add --with-deps for Linux CI system libs. Eliminates the Selenium driver-mismatch class of CI flakiness. Follow-up: speed up CI, cache binaries keyed on the lockfile. Where this goes wrong: stating the fact with no why-it-matters story.

```
# Install all browsers
npx playwright install

# CI on Linux: include OS-level dependencies
npx playwright install --with-deps chromium

# GitHub Actions: cache keyed on the lockfile
- uses: actions/cache@v4
  with:
    path: ~/.cache/ms-playwright
    key: playwright-${{ hashFiles('package-lock.json') }}
```

Q 25 LEVEL · Junior

What is the difference between @playwright/test and the playwright library?

@playwright/test: **fixtures**, **expect**, **reporters**, **parallelism**, **retries**. playwright: the bare automation engine, no test runner. Use the test package for any framework: scaffolding for free. Use the library for scripts: **screenshots**, **PDF**, **scraping**, **serverless**. Expect next: a concrete bare-library use case, e.g. a screenshot pipeline. What loses the point: defining both and stopping, with no judgement shown.

Q 26 LEVEL · Senior

What exactly does BrowserContext isolate, and why is it the unit of test parallelism?

Context isolates browser state: cookies/storage/cache/perms/locale/routes. Does NOT isolate OS state: **filesystem**, **env vars**, **/tmp**. Fresh context per test by default (test fixture builds + disposes). Multiple contexts share one browser binary: cheap to spin up. They usually ask next: namespace disk paths by worker ID, OS isn't isolated. The common miss: assuming downloads to /tmp are isolated, they collide in parallel.

```
// Default flow: fresh context per test, automatic
test('isolated test', async ({ page }) => {
  // `page` came from a fresh context created by the test fixture
  // Cookies, localStorage, cache, permissions: all empty
});

// Explicit context for multi-user scenarios
test('two users', async ({ browser }, testInfo) => {
  const ctxA = await browser.newContext({ storageState: 'auth/user-a.json' });
  const ctxB = await browser.newContext({ storageState: 'auth/user-b.json' });
  // Two contexts inside one browser: cheap, fully isolated cookies/storage

  // BUT: filesystem is NOT isolated. Namespace by worker:
  const downloadPath = `tmp/worker-${testInfo.parallelIndex}/report.pdf`;
  await Promise.all([ctxA.close(), ctxB.close()]);
});
```

Q 27 LEVEL · Mid to Senior

Explain the Page/Frame hierarchy and how navigation events flow through it.

Page contains a tree of Frames: mainFrame + iframes (transitively). Each Frame has its own URL, navigation state, execution context. Frame events fire per-frame: **framenavigated**, **frameattached**, **framedetached**. Page events (load, domcontentloaded) fire ONLY for main frame. The next question: page.on('framenavigated') with filter for iframe waits. Avoid: expecting page.on('load') to fire for iframe loads, it doesn't.

```
// Page-level event: main frame only
page.on('load', () => console.log('main frame loaded'));

// Frame-level events: fire for every frame
page.on('framenavigated', frame => {
  if (frame === page.mainFrame()) return;       // skip main
  console.log('iframe navigated:', frame.url());
});
page.on('frameattached', f => console.log('attached:', f.url()));
page.on('framedetached', f => console.log('detached:', f.url()));

// Inspect the tree
console.log('frame count:', page.frames().length);
console.log('main URL:', page.mainFrame().url());

// Implicit iframe wait: frameLocator auto-waits on the frame + element
await page.frameLocator('iframe[title="Payment"]')
  .getByLabel('Card number')
  .fill('4111 1111 1111 1111');
```

Q 28 LEVEL · Mid to Senior

Walk through the request/response lifecycle in Playwright and the four events it surfaces.

1. Four events: request -> (route) -> response -> requestfinished/requestfailed.
2. request: initiated; response: headers arrived; finished: body complete; failed: any failure.
3. page.route() sits between request and response, fulfill/continue/abort.
4. waitForRequest/waitForResponse: convenient wrappers for specific URL patterns.
5. Asked next: response for status/headers; requestfinished for body; failed for retry.
6. The easy mistake: reading body on 'response' event, body may not have arrived yet.

```
// Listen to the four events in order
page.on('request',         req => console.log('->', req.method(), req.url()));
page.on('response',        res => console.log('<-', res.status(), res.url()));
page.on('requestfinished', req => console.log('✓', req.url(), 'body received'));
page.on('requestfailed',   req => console.log('✗', req.url(), req.failure()?.errorText));

// page.route sits between request and response
await page.route('**/api/users/**', async route => {
  if (route.request().method() === 'GET') {
    await route.fulfill({ status: 200, body: JSON.stringify(mockUsers) });
  } else {
    await route.continue();                                  // forward unchanged
  }
});

// Wait for specific request/response in a test action
const [response] = await Promise.all([
  page.waitForResponse(r => r.url().includes('/api/orders') && r.status() === 200),
  page.getByRole('button', { name: 'Place Order' }).click(),
]);
...
```

Q 29 LEVEL · Senior

What is the project model in Playwright, and how do project dependencies enable setup-then-test flows?

Project = named config (browser, viewport, storageState, testMatch). Projects are the unit of parallelism across configurations. dependencies array (1.31+) makes one project run before another. Setup-as-project: auth runs once; gets reporting/retries/trace like a test. Follow-up: globalSetup for non-test work (env health check, data seed). Trap: defaulting to globalSetup when project dependencies give better observability.

```
// playwright.config.ts: setup project as dependency
export default defineConfig({
  projects: [
    { name: 'setup', testMatch: /.*\.setup\.ts/ },
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'], storageState: 'auth/admin.json' },
      dependencies: ['setup'],          // runs after setup
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'], storageState: 'auth/admin.json' },
      dependencies: ['setup'],
    },
  ],
});

// tests/auth.setup.ts: the setup is itself a test file
import { test as setup } from '@playwright/test';
setup('authenticate as admin', async ({ page }) => {
...
```

Q 30 LEVEL · Senior

What is the full test-runner lifecycle, and in what order do beforeAll, fixtures, beforeEach, the test, and teardown run?

1. 10 steps: worker -> worker fixtures -> beforeAll -> test fixtures -> beforeEach -> test -> afterEach -> fixture teardown -> afterAll -> worker teardown.
2. beforeAll runs PER WORKER, not globally: 4 workers -> 4 invocations.
3. Fixtures resolve in dependency-graph topological order, not file order.
4. Fixtures tear down in REVERSE order: dependencies outlive consumers.
5. Expect next: 4 workers + 1 file -> beforeAll fires 4 times.
6. Where this goes wrong: assuming beforeAll runs once across the whole suite.

Q 31 LEVEL · Senior

What does Playwright's trace viewer actually capture, and what's the performance cost of trace: 'on'?

Trace = zip with DOM snapshots, network, logs, timing, screenshots, source. Cost is real but suite-specific: measure it, don't quote a figure. trace: 'on' for local debugging only, too expensive in CI. trace: **'on-first-retry' for CI**, **zero cost on green**, **full trace on flake**. They usually ask next: on-first-retry is the CI default, same observability, lower cost. What loses the point: trace: 'on' in CI, paying on every green run for data nobody opens.

```
// playwright.config.ts: right trace mode per environment
export default defineConfig({
  use: {
    trace: process.env.CI ? 'on-first-retry' : 'on',
    //  CI: zero cost on passing tests, full trace on retries
    //  local: trace every test for debugging
  },
});

// Inspect a trace
//   1. Download trace.zip from CI artifacts
//   2. https://trace.playwright.dev (drag-and-drop the zip)
//   3. Scrub timeline; see DOM + network at every action
//
// Alternative modes
//   'on'                  every test: expensive
//   'off'                 never
//   'on-first-retry'      only when a test retries (CI default)
//   'retain-on-failure'   record every test, delete on pass
```

Q 32 LEVEL · Senior

What is Playwright's reporter API, and how would you build a custom reporter?

Reporter interface: **onBegin**, **onTestBegin/End**, **onStepBegin/End**, **onError**, **onEnd**. Class-based; exported and listed in the reporter config option. Integration layer: **Slack**, **JIRA**, **observability**, **traceability dashboards**. Receives full result objects: **status**, **duration**, **error**, **retries**. The next question: **real reporters**, **Slack-on-failure**, **REQ-traceability**, **flakiness trends**. The common miss: treating built-in HTML reporter as the only option, it lives in isolation.

```
// reporters/slack-on-failure.ts
import type { Reporter, TestCase, TestResult, FullResult } from '@playwright/test/reporter';

export default class SlackReporter implements Reporter {
  private failed: { test: TestCase; result: TestResult }[] = [];

  onTestEnd(test: TestCase, result: TestResult): void {
    if (result.status === 'failed' || result.status === 'timedOut') {
      this.failed.push({ test, result });
    }
  }

  async onEnd(result: FullResult): Promise<void> {
    if (this.failed.length === 0) return;
    const summary = this.failed.slice(0, 3).map(f =>
      `• ${f.test.title}: ${f.result.error?.message?.slice(0, 200)}`
    ).join('\n');
    await postToSlack(`✗ ${this.failed.length} tests failed\n${summary}`);
  }
}
...
```

QUICK QUIZ

1. What kind of connection does Playwright maintain to the browser?
   A) A new stateless HTTP request per command
   B) A persistent bidirectional WebSocket
   C) A polling REST endpoint
   D) A shared-memory channel
2. Which object is the isolated session equivalent to an incognito window?
   A) Browser
   B) Page
   C) BrowserContext
   D) Worker
3. Why prefer trace: 'on-first-retry' over trace: 'on'?
   A) It avoids storing traces for passing tests, saving storage
   B) It produces higher-quality traces
   C) It is required for parallel execution
   D) It disables tracing in CI
4. What is the main purpose of saving storageState in a setup project?
   A) To cache browser binaries
   B) To configure reporters
   C) To enable sharding
   D) To let tests start already authenticated instead of logging in each time
5. Which package includes fixtures, the expect API, reporters and parallelism?
   A) playwright
   B) @playwright/browser
   C) playwright-core
   D) @playwright/test

Answers: 1:B 2:C 3:A 4:D 5:D

Locators and DOM Strategies

Q 33 LEVEL · Junior to Mid

What is the Locator API and how is it fundamentally different from Selenium's findElement?

Locators are lazy: no DOM query until an action runs. Auto-retrying until actionable or timeout; strict on multiple matches. Mental model: a locator is an instruction, not a reference. Selenium findElement queries now and can go stale. Asked next: stale elements cannot occur by construction, a top flake source gone. Avoid: listing method differences without explaining the why.

```
// Selenium (Java): immediate query, can go stale
// WebElement b = driver.findElement(By.id("submit")); b.click();

// Playwright: lazy, auto-retrying
const button = page.locator('#submit'); // no query yet
await button.click();                   // queries + retries

// Strict mode guards against ambiguity
// await page.locator('button').click();
// Error: strict mode violation, resolved to 6 elements
await page.getByRole('button', { name: 'Submit Order' }).click();
```

Q 34 LEVEL · Junior

What is the recommended locator priority, and why?

Order: **getByRole**, **getByLabel**, **getByPlaceholder**, **getByText**, **getByTestId**. Top options target user-perceived attributes, not implementation. getByRole survives CSS/DOM refactors and is a free accessibility audit. Follow-up: getByTestId is a deliberate last-resort contract with the frontend. The order exists to keep locators surviving UI change. The easy mistake: reciting five names with no rationale for the order.

```
await page.getByRole('button', { name: 'Submit' }).click();
await page.getByLabel('Email address').fill('user@test.com');
await page.getByPlaceholder('Search...').fill('laptops');
await page.getByText('Welcome back').waitFor();
await page.getByTestId('checkout-btn').click(); // stable contract
```

Q 35 LEVEL · Mid

How do you chain and filter locators for complex scenarios?

Chaining scopes a locator inside a known parent container. filter({ hasText }) narrows by text; filter({ has }) by a child locator. Reads like a spec; replaces brittle position-based XPath. Expect next: better than XPath because it targets content/roles, not paths. Best pattern for tables and lists; survives structural change. Trap: treating chaining/filtering as syntax trivia.

```
// Scope within a parent container
const loginForm = page.locator('form#login-form');
await loginForm.getByRole('button', { name: 'Sign in' }).click();

// filter({ hasText }): the table-row pattern
const targetRow = page.locator('tbody tr').filter({ hasText: 'ORDER-12345' });
await targetRow.getByRole('button', { name: 'View Details' }).click();

// filter({ has }): rows that contain an Active badge
const activeRows = page.locator('tbody tr').filter({
  has: page.getByText('Active'),
});
await expect(activeRows).toHaveCount(3);
```

Q 36 LEVEL · Mid

How do you handle iframes with Playwright?

frameLocator scopes all subsequent locators to that iframe; nest by chaining. Selenium needs switchTo frame/defaultContent: stateful and fragile. An error cannot strand you in the wrong frame in Playwright. They usually ask next: real use case is PCI payment iframes (Stripe, PayPal, Adyen). No frame bookkeeping: reliability that matters around money flows. Where this goes wrong: naming frameLocator without the statefulness contrast.

```
// Payment iframe: scope once, interact normally
const payment = page.frameLocator('#stripe-payment-iframe');
await payment.getByLabel('Card Number').fill('4111111111111111');
await payment.getByLabel('Expiry date').fill('12/28');
await payment.getByLabel('Security code').fill('123');
await payment.getByRole('button', { name: 'Pay Now' }).click();

// Nested iframes
const inner = page.frameLocator('#outer').frameLocator('#inner');
await inner.getByRole('button', { name: 'Submit' }).click();
```

Q 37 LEVEL · Mid

How does Playwright handle shadow DOM elements?

Playwright pierces shadow DOM automatically with normal locators. No special syntax for everyday interactions; Selenium needs JS traversal. War story: a Lit Web Components app where Selenium needed helpers everywhere. The next question: evaluate + shadowRoot only for raw-markup assertions. Major reason to pick Playwright for component-based UIs. What loses the point: overcomplicating an answer that is genuinely simple.

```
// Shadow DOM is pierced transparently
await page.locator('my-custom-component input[type="text"]').fill('value');
await expect(page.locator('custom-button #inner-btn')).toBeVisible();

// Only when you need the raw shadow content
const html = await page.locator('my-component').evaluate(
  el => el.shadowRoot?.innerHTML ?? '',
);
```

Q 38 LEVEL · Junior to Mid

How do you handle dynamic elements with changing attributes or IDs?

Rule: never anchor on a value that changes between loads. Prefer semantic locators (role, text) immune to id churn. If you must use attributes, partial-match the stable portion + hasText. Asked next: no stable attribute? Filter by recognisable content. Locators with embedded numbers are the first audit refactor. The common miss: papering over a bad locator with waits or retries.

```
// Breaks every run: do not do this
// await page.locator('#user-card-1679234857233').click();

// Partial attribute match on the stable stem
await page.locator('[id^="user-card-"]').first().click();

// Semantic: immune to id changes
await page.getByRole('listitem').filter({ hasText: 'Alice Johnson' }).click();

// Dynamic table row: filter by content
const row = page.locator('tbody tr').filter({ hasText: 'john@test.com' });
await row.getByRole('button', { name: 'Edit' }).click();
```

Q 39 LEVEL · Mid

How do you handle popups, browser dialogs, and new windows?

Unifying rule: listen first, then trigger. Register dialog handlers before the triggering action. New windows/downloads: Promise.all with the popup/download event. Follow-up: **why once not on**, **once detaches**, **avoiding cross-test pollution**. Wrong ordering (listen after trigger) is the classic flakiness cause. Avoid: attaching the listener after the click.

```
// Dialog: register BEFORE the trigger; once() avoids pollution
page.once('dialog', async dialog => {
  expect(dialog.type()).toBe('confirm');
  await dialog.accept();
});
await page.getByRole('button', { name: 'Delete Account' }).click();

// New tab: listen first
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('link', { name: 'Open Report' }).click(),
]);
await expect(popup).toHaveURL(/report/);

// Download: same shape
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export CSV' }).click(),
]);
expect(await download.path()).toBeTruthy();
```

Q 40 LEVEL · Mid

What are the key options for getByRole, and how do they let you select precisely without ambiguity?

- Options narrow getByRole via accessibility semantics, not CSS.
- name (string or RegExp), level (1-6 for headings), checked/pressed/expanded.
- exact: true forces literal match; default is substring.
- Combining options builds precise stable selectors (e.g. unpressed Save button).
- Expect next: exact: true when substring would match unintended elements.
- The easy mistake: filter chains when an option would have done the same job.

```
// Common patterns
await page.getByRole('heading', { level: 1, name: 'Dashboard' }).click();
await page.getByRole('checkbox', { name: 'Remember me', checked: false }).check();
await page.getByRole('button',   { name: 'Submit',     pressed: false }).click();
await page.getByRole('button',   { name: /save/i }).click();

// `exact` prevents 'Save' from also matching 'Save Draft'
await page.getByRole('button', { name: 'Save', exact: true }).click();

// Combined: the unpressed Save toggle
await page.getByRole('button', { name: /save/i, pressed: false }).click();
```

Q 41 LEVEL · Mid

What are the four filter() options, and when do you use each?

- filter narrows in place: **hasText**, **hasNotText**, **has**, **hasNot**.
- hasText: filter on text content (most common).
- has: filter on presence of a child locator (depth signal).
- Compose: rows.filter({ hasText }).filter({ has }) for table-row pickers.
- They usually ask next: hasText for content; has for structural presence.
- Trap: CSS selectors when filter chains compose more stably.

```
const rows = page.getByRole('row');

// Pick the Acme Corp row that contains a delete button
const acmeWithDelete = rows
  .filter({ hasText: 'Acme Corp' })
  .filter({ has: page.getByRole('button', { name: 'Delete' }) });
await acmeWithDelete.getByRole('button', { name: 'Delete' }).click();

// Pick rows without a checked checkbox
const unchecked = rows.filter({ hasNot: page.getByRole('checkbox', { checked: true }) });

// Skip the header row by excluding its text
const dataRows = rows.filter({ hasNotText: 'Customer Name' });
```

Q 42 LEVEL · Mid to Senior

What do locator.and() and locator.or() do, and when are they the right tool?

and() = intersection of two locators (both must match). or() = union of two locators (either matches). Added Playwright 1.34+; replaces Promise.race and if-then-else awaits. or() classic use: await success-toast OR error-alert after form submit. The next question: or() for outcomes-could-go-either-way, one line, no flake. Where this goes wrong: Promise.race or if/else branching where or() composes cleanly.

```
// or(), await on either outcome after form submit
await page.getByRole('button', { name: 'Save' }).click();
const toast = page.locator('.toast.success');
const error = page.locator('.alert.error');
await toast.or(error).waitFor();          // resolves on whichever appears
if (await toast.isVisible()) { /* success path */ }
else                          { /* error path   */ }

// and(): the visible Save button (when invisible duplicates exist)
const visibleSave = page.getByRole('button', { name: 'Save' })
  .and(page.locator(':visible'));
await visibleSave.click();
```

Q 43 LEVEL · Mid to Senior

What is the order-of-evaluation trap with first(), last(), nth(), and filter()?

filter() narrows the set; first/last/nth picks one, order matters. rows.filter(hasText Acme).first() = the first Acme row. rows.first().filter(hasText Acme) = row 1, only if it has Acme. Rule: narrow the set first, then pick, filter().first() is the default. Asked next: highlight() + reading right-to-left helps debug. What loses the point: first().filter() silently picks wrong element; compiles fine.

```
const rows = page.getByRole('row');

// CORRECT: narrow first, then pick
const firstAcme = rows.filter({ hasText: 'Acme' }).first();
await firstAcme.getByRole('button', { name: 'Edit' }).click();

// WRONG: silent bug. Picks row 1; fails if row 1 doesn't have Acme.
const wrong = rows.first().filter({ hasText: 'Acme' });

// Same trap with nth
const thirdAcme = rows.filter({ hasText: 'Acme' }).nth(2);   // ✓ the 3rd Acme row
const wrongNth  = rows.nth(2).filter({ hasText: 'Acme' });   // ✗ row 3, only if Acme
```

Q 44 LEVEL · Mid to Senior

How do you handle nested iframes with frameLocator chains?

One frameLocator per iframe layer: chain them through nested iframes. frameLocator returns FrameLocator; getBy methods work inside the frame. Select iframes by title or data-testid, NOT auto-generated IDs. Inside the frame, getByRole priority applies as normal. Follow-up: don't explicitly wait, auto-waiting works inside the frame. The common miss: selecting iframes by index, breaks when iframe order changes.

```
// Nested iframes: payment iframe inside OAuth iframe inside main page
const paymentInOauth = page
  .frameLocator('iframe[title="Sign in with Acme ID"]')   // outer (stable title)
  .frameLocator('iframe[title="Card payment"]');           // inner (stable title)

await paymentInOauth.getByLabel('Card number').fill('4111 1111 1111 1111');
await paymentInOauth.getByLabel('Expiry').fill('12 / 30');
await paymentInOauth.getByLabel('CVC').fill('123');
// Auto-waiting works inside the frame: no explicit iframe wait needed

// AVOID: brittle, breaks when iframe order changes
// page.frameLocator('iframe').nth(1).frameLocator('iframe').first()
```

Q 45 LEVEL · Senior

How does Playwright handle open vs closed Shadow DOM, and what are the limits?

- Open Shadow DOM: Playwright pierces transparently, most common case.
- getByRole, getByLabel, locator() all see through open shadow roots.
- Closed Shadow DOM: intentionally inaccessible, host.shadowRoot is null.
- Last resort: page.evaluate, fragile, bypasses actionability checks.
- Expect next: page.evaluate(el => el.shadowRoot) returns null if closed.
- Avoid: reaching for evaluate hacks before checking if shadow is open.

```
// Open shadow DOM: Playwright pierces automatically. No special syntax.
await page.getByRole('button', { name: 'Add to Cart' }).click();
// Works even if Add-to-Cart is inside <my-product-card>'s open shadow root

// Check whether a shadow root is open or closed
const isClosed = await page.locator('my-secret-widget').evaluate(
  el => el.shadowRoot === null  // null -> closed, object -> open
);

// Last-resort access into closed shadow: fragile, bypasses auto-waiting
await page.locator('my-secret-widget').evaluate((host) => {
  // Only works if the component leaked a reference, or via reflection tricks.
  // Lose Playwright's safety net. Avoid unless absolutely necessary.
});
```

Q 46 LEVEL · Mid

How do you configure a custom test-ID attribute (data-test, data-qa) instead of the default data-testid?

use: { testIdAttribute: 'data-test' } in playwright.config.ts. test.use({ testIdAttribute: 'data-qa' }) per describe-block override. Match the codebase's convention: data-testid, data-test, data-qa, data-cy, … Standardise on ONE attribute across the framework, documented. They usually ask next: even add the chosen attribute to legacy components for consistency. The easy mistake: default data-testid when codebase uses data-test, forced mismatch.

```
// playwright.config.ts: match your codebase's convention
export default defineConfig({
  use: {
    baseURL: process.env.BASE_URL,
    testIdAttribute: 'data-test',   // -> getByTestId targets [data-test=...]
  },
});

// Now getByTestId resolves to your codebase's attribute
await page.getByTestId('submit-button').click(); // -> [data-test="submit-button"]

// Per-describe override when truly needed
test.describe('Legacy area on data-qa', () => {
  test.use({ testIdAttribute: 'data-qa' });
  test('legacy flow', async ({ page }) => { /* ... */ });
});
```

Q 47 LEVEL · Mid to Senior

What is locator strict mode, and what does it mean when an action throws on multiple matches?

Strict mode: actions/assertions throw on >1 match, not silent pick. Why: ambiguous locators silently click wrong elements; strict catches it. Right fix: narrow with name option / filter / parent scope. first()/nth()/last() ONLY when you genuinely want one of many. The next question: first() appropriate for 'any row exists' smoke checks. Trap: chaining .first() reflexively to silence, the bug ships.

```
// Two Submit buttons on the page: ambiguous locator
await page.getByRole('button', { name: 'Submit' }).click();
// -> Error: strict mode violation. Resolved to 2 elements:
//   1) <button>Submit</button> in the login form
//   2) <button>Submit</button> in the feedback form

// CORRECT: narrow by container
await page.getByRole('form', { name: 'Login' })
  .getByRole('button', { name: 'Submit' })
  .click();

// WRONG: silences the violation, picks an arbitrary form
// await page.getByRole('button', { name: 'Submit' }).first().click();
```

Q 48 LEVEL · Senior

How does Playwright resolve the accessible name for getByLabel and getByRole, and why does it matter?

ARIA name computation: 6-step priority order. Order: aria-labelledby -> aria-label -> label -> placeholder -> title -> text. aria-label OVERRIDES visible text: source of surprising getByRole names. Locator failures often reveal real accessibility bugs in the app. Asked next: missing for-id pairing means visible label ≠ accessible name. Where this goes wrong: assuming visible text is what name option matches, it isn't always.

```
// HTML:
//   <button aria-label="Submit form">Save</button>
//
// Visible text is 'Save' but accessible name is 'Submit form' (aria-label wins)

await page.getByRole('button', { name: 'Submit form' }).click(); // ✓ matches
// await page.getByRole('button', { name: 'Save' }).click();     // ✗ no match

// Inspect what Playwright sees as the accessible name
const name = await page.getByRole('button').first().evaluate(el => {
  return el.getAttribute('aria-label') ?? el.textContent;
});

// Common bug: visible <label> but no for-id pairing, getByLabel fails
//   <span>Email</span>           <- looks like a label, isn't programmatically one
//   <input type="email">         <- has no associated label
// Fix: add <label for="email">Email</label> + id="email" on the input.
```

Q 49 LEVEL · Mid

What's the difference between toBeVisible, toBeAttached, and toBeInViewport?

attached < visible is a real ladder; in viewport is a separate axis, not a rung. toBeAttached: in DOM, may be hidden/off-screen. toBeVisible: in DOM AND non-zero size AND not display:none/visibility:hidden. toBeInViewport: intersects the viewport (Intersection Observer), visibility not required. So visibility:hidden mid-screen PASSES toBeInViewport and FAILS toBeVisible. Follow-up: tooltip pre-hover -> toBeAttached; tooltip on hover -> toBeVisible. What loses the point: toBeVisible reflexively when the question is attached or in-viewport.

```
// toBeAttached, lowest bar: in DOM, may be hidden
await expect(page.locator('.hidden-form')).toBeAttached();

// toBeVisible, user can see it (size + not display:none/visibility:hidden)
await expect(page.locator('.toast.success')).toBeVisible();

// toBeInViewport: within the scrolled-into-view region
await page.locator('section#pricing').scrollIntoViewIfNeeded();
await expect(page.locator('section#pricing')).toBeInViewport();

// Tooltip example: different assertions for different questions
await expect(page.locator('.tooltip')).toBeAttached();  // exists before hover
await page.locator('button.info').hover();
await expect(page.locator('.tooltip')).toBeVisible();   // appears on hover
```

Q 50 LEVEL · Senior

How do you write Playwright tests that double as accessibility checks, and where do dedicated tools (axe-core, Lighthouse a11y) earn their place?

getByRole/getByLabel/getByAltText: every locator is an implicit a11y check. @axe-core/playwright: explicit ruleset, structured violations with severity. Lighthouse a11y via lighthouse-ci: high-level score, useful as budget gate. Tier: baseline (locators) + explicit (axe per-page) + audit (Lighthouse trend). Keyboard nav: test tab order via page.keyboard, focus-trap on modals. Pitfall: a11y as separate quarterly audit, bugs ship, fixed expensively.

```
import AxeBuilder from '@axe-core/playwright';

test.describe('Checkout page accessibility', () => {
  test('axe-core scan: zero serious or critical violations', async ({ page }) => {
    await page.goto('/checkout');
    await expect(page.getByRole('heading', { name: 'Checkout' })).toBeVisible();

    const results = await new AxeBuilder({ page })
      .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
      .analyze();

    const severe = results.violations.filter(
      v => v.impact === 'serious' || v.impact === 'critical');
    expect(severe, JSON.stringify(severe, null, 2)).toEqual([]);
  });

  test('keyboard tab order traverses checkout form correctly', async ({ page }) => {
    await page.goto('/checkout');
    await page.keyboard.press('Tab'); // first focusable
    await expect(page.getByLabel('Card number')).toBeFocused();
...
```

Q 51 LEVEL · Mid

setInputFiles vs the filechooser event: which do you use for file uploads, and when is the filechooser event the only option?

setInputFiles sets files directly on a native input, no OS dialog opens. It is the default and simplest choice for the large majority of uploads. filechooser event is listen-first: waitForEvent then fileChooser.setFiles(). Only option when a custom widget exposes no targetable file input. Expect next: don't always use the event, it's more ceremony and a flake risk. The common miss: presenting the two as interchangeable.

```
// Default: set files straight on a native input, no OS dialog
await page.getByLabel('Upload CV').setInputFiles('files/cv.pdf');
await page.getByLabel('Attachments').setInputFiles([
  'files/a.png', 'files/b.png',
]);
// In-memory buffer also works
await page.getByLabel('Upload CV').setInputFiles({
  name: 'cv.pdf', mimeType: 'application/pdf', buffer: Buffer.from('...'),
});

// Only option: custom widget with no reachable input. Listen first.
const [fileChooser] = await Promise.all([
  page.waitForEvent('filechooser'),
  page.getByRole('button', { name: 'Choose file' }).click(),
]);
await fileChooser.setFiles('files/cv.pdf');
```

Q 52 LEVEL · Mid

Locator vs ElementHandle (page.$): why is the locator's lazy auto-waiting safer than an ElementHandle's eager snapshot?

Locator = lazy re-evaluated instruction with auto-waiting. ElementHandle (page.$) = eager one-time snapshot of a node. On re-render the locator re-finds; the handle goes detached and throws. Detached-node error is Selenium's stale-element creeping back in. Playwright team discourages ElementHandle: default to Locator. They usually ask next: narrow handle use is passing a live node to evaluate; even that is shrinking.

```
// Discouraged: eager snapshot, no auto-wait, goes stale on re-render
// const handle = await page.$('#cart-total');
// ... app re-renders the cart ...
// await handle.click();  // Error: element is not attached to the DOM

// Preferred: lazy, re-found with auto-waiting on every action
const cartTotal = page.locator('#cart-total');
await cartTotal.click();          // re-queries here
await expect(cartTotal).toHaveText('₹4,000');  // re-queries again

// Rare legitimate handle use: passing a live node into evaluate
const handle = await cartTotal.elementHandle();
await page.evaluate(el => el?.scrollIntoView(), handle);
```

Q 53 LEVEL · Junior to Mid

getByText vs getByLabel vs getByPlaceholder vs getByAltText vs getByTitle, what does each target, and how do you choose between them?

getByText: **visible text content**, **links**, **headings**, **static copy**. getByLabel: form control by its label, default for inputs/selects/checkboxes. getByPlaceholder: placeholder attribute, fallback only when no label exists. getByAltText: image alt text; getByTitle: title attribute (tooltips, icon controls). Choose by the most stable user-perceived attribute; getByRole sits above all five. The next question: label always over placeholder, placeholders vanish on input.

```
await page.getByText('Forgot your password?').click();      // visible text
await page.getByLabel('Email address').fill('a@test.com');  // labelled field
await page.getByPlaceholder('Search products').fill('laptop'); // no label -> fallback
await page.getByAltText('Company logo').click();            // image alt
await page.getByTitle('Close dialog').click();              // tooltip / icon-only

// Prefer label over placeholder when both exist
// BAD:  await page.getByPlaceholder('Enter email').fill(...);
// GOOD: await page.getByLabel('Email address').fill(...);
```

Q 54 LEVEL · Mid to Senior

frameLocator vs page.frame(): when do you use the retrying frameLocator and when the older frame handle?

frameLocator returns a lazy, auto-retrying FrameLocator, the recommended default. Chain getByRole/getByLabel off it; it re-resolves frame + element with auto-waiting. page.frame() is an eager lookup: **returns null if absent**, **no retry**, **old API**. Use page.frame() only for the frame object: **URL/name**, **navigation**, **evaluate**, **page**.frames(). Rule: frameLocator to interact with elements; page.frame() only for the frame itself. Avoid: page.frame() for element work, then bolting on manual waits for null timing.

```
// Default: retrying FrameLocator, auto-waits, chains normal locators
const checkout = page.frameLocator('iframe[title="Card payment"]');
await checkout.getByLabel('Card number').fill('4111 1111 1111 1111');
await checkout.getByRole('button', { name: 'Pay' }).click();

// page.frame(): only when you need the frame OBJECT itself
const frame = page.frame({ name: 'payment' });
if (frame) {
  console.log(frame.url());        // read frame-level URL
  await frame.evaluate(() => document.title);
}

// Iterate every frame on the page
for (const f of page.frames()) console.log(f.name(), f.url());
```

QUICK QUIZ

1. Why can a Playwright Locator never throw StaleElementReferenceException?
   A) It caches the element on creation
   B) It runs in a separate thread
   C) It disables DOM mutations
   D) It is a lazy instruction that re-queries with retries on each action
2. Which locator is recommended first because it survives CSS refactors?
   A) getByTestId
   B) getByRole
   C) getByText
   D) locator with absolute XPath
3. Which filter selects table rows that contain a given child locator?
   A) filter({ has })
   B) filter({ hasText })
   C) filter({ visible: true })
   D) filter({ nth: 0 })
4. How does Playwright handle an input inside a custom element's shadow root?
   A) Requires JavaScript shadowRoot traversal
   B) Cannot interact with shadow DOM
   C) Pierces the shadow boundary automatically with normal locators
   D) Needs a special frameLocator call
5. Why use page.once('dialog', ...) rather than page.on for a confirm dialog?
   A) once removes the listener after the first use, avoiding cross-test pollution
   B) once is faster
   C) on does not support dialogs
   D) once retries automatically

Answers: 1:D 2:B 3:A 4:C 5:A

Auto-waiting and Synchronisation

Q 55 LEVEL · Mid

How does Playwright's auto-waiting mechanism work internally?

Actionability checks run before each action and are action-specific. Click: **visible**, **stable**, **enabled**, **pointer-events**; retried until timeout. Pointer-events check defeats the cookie-overlay misclick. Asked next: force: true bypasses all checks, conscious last resort only. A failing check is usually a real signal, not noise to silence. The easy mistake: reaching for force to mask auto-waiting failures.

```
// All checks run automatically before this click
await page.getByRole('button', { name: 'Submit' }).click();
// internally: visible? stable? receives pointer events? enabled?
//   t=0   visible? no  -> retry
//   t=100 visible yes, stable? no (animating) -> retry
//   t=200 every check this action runs passes -> click

// force bypasses EVERY check: use consciously, not as a workaround
await page.locator('#hidden-trigger').click({ force: true });
```

Q 56 LEVEL · Mid

What are the timeout levels in Playwright and how should you configure them?

- Four layers: test, action, navigation, expect (defaults vary).
- A huge global timeout hides performance regressions.
- Keep action timeout tight (~15s) so slow interactions fail loudly.
- Follow-up: genuinely slow test -> test.slow(), not a bigger baseline.
- Slow tests stay explicitly flagged and tracked.
- Trap: recommending a big global timeout to 'reduce flakiness'.

```
export default defineConfig({
  timeout: 60_000,                // test
  expect: { timeout: 10_000 },    // assertions
  use: {
    actionTimeout: 15_000,        // per action, kept tight
    navigationTimeout: 30_000,    // navigation
  },
});

test('report generation', async ({ page }) => {
  test.slow(); // triples THIS test's timeout, not the global baseline
  // ...
});

await expect(page.locator('#async-data')).toBeVisible({ timeout: 20_000 });
```

Q 57 LEVEL · Mid

When do you need explicit waits despite auto-waiting?

Auto-waiting covers actionability, not navigation/network/custom state. waitForURL for page changes; waitFor({ state: 'hidden' }) for spinners. Promise.all + waitForResponse, listener attached before the click. waitForFunction for custom window readiness flags in legacy apps. Expect next: **never a fixed sleep**, **flaky if short**, **slow if long**. Where this goes wrong: answering with page.waitForTimeout.

```
// URL change
await page.getByRole('button', { name: 'Checkout' }).click();
await page.waitForURL('/checkout', { timeout: 10_000 });

// Disappearance
await page.locator('#loading-spinner').waitFor({ state: 'hidden' });

// Network correlation: listen first
const [response] = await Promise.all([
  page.waitForResponse(r => r.url().includes('/api/orders') && r.status() === 201),
  page.getByRole('button', { name: 'Place Order' }).click(),
]);
expect((await response.json()).orderId).toBeTruthy();

// Custom app state
await page.waitForFunction(() => (window as any).__APP_READY__ === true);
```

Q 58 LEVEL · Mid

What is the difference between waitForResponse and page.route() for API interaction?

waitForResponse: **passive**, **read-only**, **observe real traffic**. route: active, write, control what the app receives. waitForResponse for integration; route for isolated frontend tests. They usually ask next: pick by goal, verify real contract vs isolate the UI. Over-mocking with route can stay green while production breaks. What loses the point: implying route is the default for all API testing.

```
// Observe: validate the real API call
const [res] = await Promise.all([
  page.waitForResponse(r => r.url().includes('/api/orders') && r.request().method() === 'POST'),
  page.getByRole('button', { name: 'Place Order' }).click(),
]);
expect((await res.request().postDataJSON()).items).toHaveLength(2);
expect(res.status()).toBe(201);

// Control: force a payment failure the backend cannot reliably produce
await page.route('**/api/payments', route => route.fulfill({
  status: 402, body: JSON.stringify({ error: 'Card declined' }),
}));
await page.getByRole('button', { name: 'Pay Now' }).click();
await expect(page.getByText('Card declined')).toBeVisible();
```

Q 59 LEVEL · Junior to Mid

How does page.waitForLoadState() work, and when do you use each option?

domcontentloaded: HTML parsed (fastest); load: HTML + resources (default). networkidle: ~500ms of no requests, fragile with background traffic. Prefer domcontentloaded + wait for a meaningful element. The next question: networkidle discouraged, polling/analytics/WebSockets never go quiet. Element readiness targets real app state, not a timing heuristic. The common miss: recommending networkidle as the safe default.

```
await page.goto('/simple', { waitUntil: 'domcontentloaded' }); // fastest
await page.goto('/dashboard');                                 // load (default)

// Deterministic page-ready check in a page object
async waitForDashboardLoad(): Promise<void> {
  await this.page.waitForLoadState('domcontentloaded');
  await this.page.getByRole('region', { name: 'Statistics' }).waitFor();
  await this.page.locator('#loading-skeleton').waitFor({ state: 'hidden' });
}
```

Q 60 LEVEL · Junior to Mid

How does Playwright's expect() retry mechanism work?

Web-first assertions poll/retry until pass or expect timeout. isVisible/isEnabled/isChecked check once and return a boolean. Phrasing: isVisible is a question, toBeVisible is an assertion. Asked next: use isVisible only for genuine conditional logic. prefer-web-first-assertions enforces the retrying form by default. Avoid: asserting on an isVisible boolean, the snapshot race.

```
// Retrying web-first assertion
await expect(page.locator('#status')).toHaveText('Order Confirmed');

// Wrong: point-in-time snapshot
// const visible = await page.locator('#result').isVisible();
// expect(visible).toBe(true); // fails if it appears 1ms later

// Right use of isVisible: conditional logic
if (await page.locator('#phone-number').isVisible()) {
  await page.locator('#phone-number').fill('+91-9876543210');
}
```

Q 61 LEVEL · Mid to Senior

How do you use page.clock to test time-dependent features?

clock.install replaces Date/setTimeout/setInterval/rAF with a fake clock. Install BEFORE navigation so it captures load-time timers. fastForward fires due timers instantly: no real waiting. runFor runs pending timers in a window; setFixedTime pins a date. Follow-up: test a session timeout via install-then-fastForward in ms. The easy mistake: **old workarounds**, **real waits**, **hand-overriding Date**, **backend mocks**.

```
test('logged out after 24h session timeout', async ({ page }) => {
  await page.clock.install();          // before navigation
  await page.goto('/dashboard');
  await expect(page.getByTestId('welcome-banner')).toBeVisible();

  await page.clock.fastForward(24 * 60 * 60 * 1000); // jump 24h instantly

  await expect(page.getByTestId('session-expired-modal')).toBeVisible();
  await expect(page).toHaveURL(/\/login/);
});

// Deterministic relative timestamp
await page.clock.setFixedTime(new Date('2026-04-03T10:00:00Z'));
await page.goto('/posts/123'); // created 2026-04-01
await expect(page.locator('.post-timestamp')).toHaveText('2 days ago');
```

Q 62 LEVEL · Mid to Senior

What are the five actionability checks Playwright runs, and which actions run which?

- The five: **visible**, **stable**, **receives events**, **enabled**, **editable**.
- The set is per action, not one fixed list run before everything.
- click, dblclick, tap, check: visible + stable + receives events + enabled.
- fill and clear: visible + enabled + editable ONLY, no stable, no receives events.
- hover: visible + stable + receives events. focus, press, dispatchEvent: no checks.
- Trap: 'attached' is an assertion, and 'in viewport' is not a check, Playwright scrolls.

```
// Each action runs its OWN subset. From the actionability docs:
//
//   click, dblclick, tap        visible + stable + receives events + enabled
//   check, uncheck, setChecked  same as click
//   hover, dragTo               visible + stable + receives events
//   screenshot                  visible + stable
//   fill, clear                 visible + enabled + editable  (no stable!)
//   selectOption                visible + enabled
//   selectText                  visible
//   scrollIntoViewIfNeeded      stable
//   focus, blur, press,
//   pressSequentially,
//   dispatchEvent,
//   setInputFiles               no actionability checks at all
//
// Why it matters: a cookie banner blocks the click, not the fill
await page.getByLabel('Email').fill('a@b.com');   // works under the overlay
await page.getByRole('button', { name: 'Submit' }).click();  // waits for it to clear

// Failure mode: the error names the failing check
...
```

Q 63 LEVEL · Mid

What is the difference between isVisible() and toBeVisible(), and why does the distinction cause subtle test bugs?

Isvisible()	Tobevisible()
isVisible(): one-shot boolean, no waiting, current value only.	toBeVisible(): retrying assertion, polls until visible or timeout.

expect(await locator.isVisible()).toBe(true) silently passes if not yet visible.

Same pattern: **isEnabled/toBeEnabled**, **isChecked/toBeChecked**, **isHidden/toBeHidden**.

Rule: assertions use toBeXxx; boolean branches use isXxx.

Where this goes wrong: assertion-shaped isVisible() that passes the test before the element renders.

```
// WRONG: silently passes if element hasn't rendered yet
await page.getByRole('button', { name: 'Save' }).click();
expect(await page.locator('.toast').isVisible()).toBe(true); // ✗ one-shot before paint

// CORRECT: polls until visible or timeout
await page.getByRole('button', { name: 'Save' }).click();
await expect(page.locator('.toast')).toBeVisible();           // ✓ retrying

// isVisible() is correct for boolean branching
if (await page.getByRole('dialog', { name: 'Cookie banner' }).isVisible()) {
  await page.getByRole('button', { name: 'Accept' }).click();
}
// here we WANT the current value, not a retried one
```

Q 64 LEVEL · Mid to Senior

When and how do you use waitForFunction to poll arbitrary application state?

Polls a function in the page context until truthy or timeout. Right for state NOT in DOM: **Redux stores**, **window flags**, **JS instances**. Default polling: requestAnimationFrame (every paint); 'mutation' for DOM-tied. Explicit ms interval when paint-rate polling is wasteful. Discipline: only when no DOM signal exists, locators are more robust. What loses the point: waitForFunction when a locator would have worked, loses observability.

```
// State not in DOM: wait on a window-level analytics flag
await page.waitForFunction(
  () => (window as any).__analyticsReady === true,
  null,
  { timeout: 10_000 }
);

// Wait for a Redux store value
await page.waitForFunction(
  () => (window as any).store.getState().cart.items.length > 0
);

// Wait for a chart instance to have rendered data
await page.waitForFunction(
  () => (window as any).myChart?.data?.datasets?.[0]?.data?.length > 0,
  null,
  { polling: 100 }   // explicit ms when paint-rate is overkill
);

// PREFER a locator when the state IS in the DOM
...
```

Q 65 LEVEL · Mid

How does page.waitForURL work, and when is it the right tool?

Waits until URL matches: **glob string**, **RegExp**, or **predicate function**. Right for redirect chains: **OAuth**, **password reset**, **post-checkout flows**. Glob example: '**/dashboard?**', path match with any query string. Pair with toHaveURL: waitForURL for waiting, toHaveURL for assertions. Expect next: click already auto-waits for nav; waitForURL is for assertion after. The common miss: string equality after a redirect chain, fragile to query changes.

```
// After OAuth callback, wait for the dashboard URL with any query string
await page.getByRole('button', { name: 'Sign in with SSO' }).click();
// SSO redirect chain happens here
await page.waitForURL('**/dashboard?**');

// Predicate for complex matching
await page.waitForURL(url => {
  return url.pathname === '/checkout/confirmation'
      && url.searchParams.get('orderId')?.startsWith('ord_');
});

// Assert on the URL (cleaner failure message than waitForURL)
await expect(page).toHaveURL(/\/dashboard$/);

// waitForURL for waiting, toHaveURL for assertions, same matchers, different purpose
```

Q 66 LEVEL · Mid to Senior

Why must waitForEvent be inside Promise.all with the action that triggers the event?

Events fire asynchronously: a late listener never sees the event. Promise.all wraps listener + trigger so listener registers first. Pattern applies to: **downloads**, **popups**, **new pages**, **console**, **WebSocket**, **requests**. Principle: register the listener BEFORE doing the thing. They usually ask next: page.on() before action is the alternative for multi-event capture. Avoid: click first, listen second, test hangs to timeout silently.

```
// CORRECT: listener registered before trigger via Promise.all
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export CSV' }).click(),
]);

// Same pattern for popups (target='_blank')
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('link', { name: 'Open report' }).click(),
]);

// Same for waiting on a specific API call during the test action
const [response] = await Promise.all([
  page.waitForResponse(r => r.url().includes('/api/orders') && r.status() === 201),
  page.getByRole('button', { name: 'Place Order' }).click(),
]);

// WRONG: the event may fire between the click and the await
// await page.getByRole('button', { name: 'Export CSV' }).click();
...
```

Q 67 LEVEL · Mid to Senior

What does the force option do, and when (rarely) is it appropriate?

force: true skips all actionability checks, last resort escape hatch. Legitimate uses: **testing disabled state**, **browser-bug workaround**, **perf opt**. Trade-off: force-click on a covered element silently misses or hits overlay. Rule: always write a comment explaining why force was needed. The next question: covered-by-modal clicks are the classic bug force hides. The easy mistake: reaching for force to silence actionability errors that report real bugs.

```
// LEGITIMATE: testing the disabled state itself
// Reason: we want to verify that clicking the disabled button does NOT submit.
const submit = page.getByRole('button', { name: 'Submit' });
await expect(submit).toBeDisabled();
await submit.click({ force: true });  // force needed: clicking a disabled button
await expect(page).not.toHaveURL(/success/);

// LEGITIMATE: known browser bug, link the issue
// https://github.com/microsoft/playwright/issues/12345
// Actionability incorrectly reports failure on this SVG button.
await svgIcon.click({ force: true });

// WRONG: silencing a real failure
// await submitBtn.click({ force: true });  // because the test failed without it
```

Q 68 LEVEL · Mid

How does navigation auto-waiting work when a click triggers a page transition?

A click that navigates already waits for that navigation: no explicit wait needed. Discipline: don't add waitForNavigation after a click, it is duplicate logic. click's options: **button**, **clickCount**, **delay**, **force**, **modifiers**, **noWaitAfter**, **position**, **timeout**, **trial**. No lifecycle option on an action: those belong to page.goto. noWaitAfter is deprecated and will default to true, so assert on the destination. waitForURL for: **asserting the destination**, **multiple navigations**, **SPA route changes**.

```
// A click that navigates already waits: no extra wait needed
await page.getByRole('link', { name: 'Dashboard' }).click();
await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
// the click waited for the navigation to start loading before resolving

// WRONG: duplicate logic, the click already waited
// await page.getByRole('link', { name: 'Reports' }).click();
// await page.waitForNavigation();

// One click, several navigations (login then redirect): name the destination
await page.getByRole('button', { name: 'Sign in' }).click();
await page.waitForURL('**/dashboard');

// SPA route change (no page load): assert on the new URL or new content
await page.getByRole('button', { name: 'Open profile' }).click();
await expect(page).toHaveURL(/\/profile$/);
```

Q 69 LEVEL · Mid

In page.goto, what does waitUntil 'load' vs 'domcontentloaded' vs 'networkidle' each wait for, and why is networkidle discouraged?

domcontentloaded: **DOM parsed**, **before resources**, **fastest**. load (default): DOM + all sub-resources loaded, safe middle ground. networkidle: **~500ms of no requests**, **strongest-sounding**, **weakest in practice**. Why: load/domcontentloaded fire on guaranteed events; networkidle waits on a quiet that may never come. Discouraged because polling/analytics/WebSockets keep the network busy. Trap: networkidle as the 'safe' default, prefer domcontentloaded + a meaningful element.

```
// domcontentloaded: fastest, DOM only
await page.goto('/article', { waitUntil: 'domcontentloaded' });

// load: the default, DOM + images/CSS/fonts
await page.goto('/dashboard');                 // waitUntil: 'load'

// networkidle, discouraged: a polling SPA never goes quiet
// await page.goto('/feed', { waitUntil: 'networkidle' }); // may hang to timeout

// Preferred: fast lifecycle + a meaningful readiness signal
await page.goto('/feed', { waitUntil: 'domcontentloaded' });
await expect(page.getByRole('region', { name: 'Latest posts' })).toBeVisible();
```

Q 70 LEVEL · Junior to Mid

What is the difference between headless and headed run mode, when do you run headed, and why can a test pass in one and fail in the other?

Headless: **no visible window**, **faster**, **the CI default**; headed: real window for local debugging. Run headed with --headed / headless:false, often plus --debug or slowMo to step through. Differences that bite: **viewport/DPR**, **fonts**, **timing/races**, **automation-or-no-display detection**. A pass-headed/fail-headless test was leaning on the environment, not the app. Asked next: debug by rerunning headed with --debug or trace, and pin an explicit viewport. Where this goes wrong: treating a headless-only failure as flakiness to retry away.

```
// Run modes
//   npx playwright test                -> headless (default, CI)
//   npx playwright test --headed       -> visible window
//   npx playwright test --debug        -> headed + inspector, step through

// Pin the environment so headed and headless render identically
export default defineConfig({
  use: {
    viewport: { width: 1280, height: 720 }, // same breakpoint in both modes
    headless: true,                         // default; flip per-run with --headed
    trace: 'on-first-retry',                // capture why a headless run failed
  },
});

// Local debugging of a headless-only failure
//   npx playwright test login.spec.ts --headed --debug
```

Q 71 LEVEL · Mid to Senior

For click(), what does the default action do versus the force, trial, and noWaitAfter options, what does each bypass, and what is the risk of each?

Default click(): its actionability checks -> click -> wait for a triggered navigation. force: skips the checks BEFORE, risk is a silent misclick (hits an overlay, stays green). trial: keeps checks but does NOT click, dry run; risk is assuming it changed state. noWaitAfter: clicks but skips the wait AFTER, risk is a race with the click's side effect. Model: **force=before**, **noWaitAfter=after**, **trial=skips the click itself**. noWaitAfter is deprecated and will default to true: name it, never write it. Follow-up: trial to probe, force only as a commented last resort; downloads use a waiter promise.

```
// Default: checks, click, then wait for the triggered navigation to settle
await page.getByRole('button', { name: 'Save' }).click();

// force, skip the checks BEFORE; risk: silent misclick on an overlay
await page.getByRole('button', { name: 'Save' }).click({ force: true });

// trial: run checks but DON'T click; a dry run probing clickability
await page.getByRole('button', { name: 'Save' }).click({ trial: true });
// nothing changed: it only verified the element would be clickable

// noWaitAfter is DEPRECATED and will default to true. Don't write it.
// The current idiom for a download: set the waiter up, click, then await it
const downloadPromise = page.waitForEvent('download');
await page.getByRole('button', { name: 'Export' }).click();
const download = await downloadPromise;
await download.saveAs('./tmp/report.csv');
```

QUICK QUIZ

1. Which actionability check stops Playwright clicking a button hidden under a cookie overlay?
   A) receives events
   B) stable
   C) enabled
   D) editable
2. Why keep the action timeout tight (around 15s) rather than setting a large global timeout?
   A) Tight timeouts run tests faster
   B) A large global timeout hides performance regressions
   C) Playwright requires a 15s action timeout
   D) It reduces memory usage
3. You need a test to wait for a loading spinner to go away. Which is correct?
   A) spinner.waitFor({ state: 'hidden' })
   B) page.waitForTimeout(2000)
   C) expect(spinner).toBeVisible()
   D) spinner.isVisible()
4. What is the difference between waitForResponse and page.route()?
   A) They are identical
   B) route is read-only; waitForResponse mocks
   C) waitForResponse observes real traffic; route controls what the app receives
   D) Only route works in CI
5. Why install page.clock before calling page.goto()?
   A) goto resets the clock
   B) Otherwise fastForward waits real time
   C) It is required for screenshots
   D) So the fake clock captures timers set during page load

Answers: 1:A 2:B 3:A 4:C 5:D

Page Object Model in Playwright

Q 72 LEVEL · Mid

How do you implement production-grade POM in Playwright TypeScript?

- Locators are readonly properties because they are lazy and auto-retry.
- No re-finding elements per method (the Selenium habit).
- Action methods describe intent and hide the steps; login() returns void.
- No assertions inside page classes: the test decides what to verify.
- Expect next: why properties not methods, lazy locators never go stale.
- What loses the point: describing generic POM that could be any language.

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;
  readonly errorAlert: Locator;

  constructor(readonly page: Page) {
    this.emailInput    = page.getByLabel('Email address');
    this.passwordInput = page.getByLabel('Password');
    this.loginButton   = page.getByRole('button', { name: 'Sign in' });
    this.errorAlert    = page.getByRole('alert');
  }

  async navigate(): Promise<void> { await this.page.goto('/login'); }

  async login(email: string, password: string): Promise<void> {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
...
```

Q 73 LEVEL · Mid

How do you use fixtures for page object dependency injection?

Extend the base test once; register each page object as a fixture. The new keyword never appears in a test. Constructor change -> update one fixture, every test untouched. They usually ask next: better than beforeEach, fixtures compose and lazy-instantiate. Re-export expect so each test has one import line. The common miss: newing up page objects inside each test or beforeEach.

```
// src/fixtures/pages.fixture.ts
import { test as base, expect } from '@playwright/test';
import { LoginPage } from '@pages/LoginPage';
import { DashboardPage } from '@pages/DashboardPage';

type PageFixtures = { loginPage: LoginPage; dashboardPage: DashboardPage };

export const test = base.extend<PageFixtures>({
  loginPage:     ({ page }, use) => use(new LoginPage(page)),
  dashboardPage: ({ page }, use) => use(new DashboardPage(page)),
});
export { expect };

// tests/login.spec.ts: zero new keyword, one import line
import { test, expect } from '@fixtures/pages.fixture';
test('valid login redirects', async ({ loginPage, dashboardPage }) => {
  await loginPage.navigate();
  await loginPage.login('admin@test.com', 'Admin@123');
  await expect(dashboardPage.welcomeHeading).toBeVisible();
});
```

Q 74 LEVEL · Mid

How do you build an abstract BasePage class?

BasePage holds shared navigation and helpers; pages extend it. abstract prevents direct instantiation: enforces extend-only. protected hides shared helpers from test code. Class-field initialisers read cleaner and show current TypeScript. The next question: only cross-cutting behaviour belongs in BasePage. Avoid: a swollen BasePage god-class.

```
import { Page, Locator } from '@playwright/test';

export abstract class BasePage {
  constructor(protected readonly page: Page) {}

  async navigate(path: string): Promise<void> {
    await this.page.goto(path);
    await this.page.waitForLoadState('domcontentloaded');
  }

  protected async selectOption(dropdown: Locator, option: string): Promise<void> {
    await dropdown.click();
    await this.page.getByRole('option', { name: option }).click();
  }
}

export class DashboardPage extends BasePage {
  readonly welcomeHeading = this.page.getByRole('heading', { level: 1 });
}
```

Q 75 LEVEL · Mid to Senior

How do you implement component-level page objects?

Component takes a container locator; scopes all internals to it. Same class serves every instance (e.g. every table): different scope. Collapses duplicated interaction code into one reusable class. Behaviour change -> fix the component once, all instances updated. Asked next: **modals**, **nav bars**, **date pickers**, **any repeated widget**. The easy mistake: duplicating table logic across multiple page classes.

```
import { Locator } from '@playwright/test';

export class DataTable {
  constructor(private readonly container: Locator) {}

  rowByText(text: string): Locator {
    return this.container.locator('tbody tr').filter({ hasText: text });
  }
  async rowCount(): Promise<number> {
    return this.container.locator('tbody tr').count();
  }
  async clickActionInRow(rowText: string, action: string): Promise<void> {
    await this.rowByText(rowText).getByRole('button', { name: action }).click();
  }
}

// Same component, two tables, different containers
class UsersPage extends BasePage {
  readonly usersTable = new DataTable(this.page.locator('table#users'));
  readonly auditTable = new DataTable(this.page.locator('table#audit-log'));
...
```

Q 76 LEVEL · Mid to Senior

How do you handle navigation flows between page objects?

Navigation methods return the next page object, a type-safe chain. Tests read like user stories; no instantiation, no scattered URL asserts. The page object owns waiting and URL knowledge. URL change -> update one method, all journeys stay correct. Follow-up: same-page actions return void/this, not a page. Trap: forcing a page return where there is no real next page.

```
export class LoginPage extends BasePage {
  async loginAs(email: string, password: string): Promise<DashboardPage> {
    await this.page.getByLabel('Email').fill(email);
    await this.page.getByLabel('Password').fill(password);
    await this.page.getByRole('button', { name: 'Sign in' }).click();
    await this.page.waitForURL('/dashboard');
    return new DashboardPage(this.page);
  }
}

// Test reads like a user journey
test('admin can view all users', async ({ loginPage }) => {
  const dashboard = await loginPage.loginAs('admin@test.com', 'Admin@123');
  const usersPage = await dashboard.navigateToUsers();
  await expect(usersPage.usersTable.rowCount()).resolves.toBeGreaterThan(0);
});
```

Q 77 LEVEL · Mid to Senior

Do you store locators as class properties or chain them inline in methods, and what's the trade-off?

Properties: **named vocabulary**, **reuse**, **single definition**, **single construction**. Inline chains: dynamic args work naturally, less boilerplate for one-shots. Pragmatic rule: properties for reused/static, chains for dynamic/single-use. Many POMs do both: not religious about one pattern. Expect next: prefer the right tool per locator, not either-or. Where this goes wrong: storing every locator as a property, dead vocabulary for one-shots.

```
export class UsersPage {
  constructor(private readonly page: Page) {}

  // Static, reused: stored as properties
  readonly heading      = this.page.getByRole('heading', { name: 'Users' });
  readonly searchInput  = this.page.getByLabel('Search users');
  readonly newUserBtn   = this.page.getByRole('button', { name: 'New user' });
  readonly table        = this.page.getByRole('table');

  // Dynamic, argument-dependent: inline chain inside method
  async deleteUser(email: string): Promise<void> {
    await this.table
      .getByRole('row', { name: email })
      .getByRole('button', { name: 'Delete' })
      .click();
    await this.page.getByRole('button', { name: 'Confirm' }).click();
  }
}
```

Q 78 LEVEL · Senior

Should assertions live inside POM methods, and what's the case for both sides?

For: **cleaner tests**, **centralised assertion logic**, **focus on flow**. Against: assertions are intent (test territory); POMs that assert are opaque. Pragmatic: POMs return locators/data; tests do the assertions. Exception: page-state checks (expectLoaded) the POM owns as internal knowledge. They usually ask next: expectLoaded() is the exception, page state, not test intent. What loses the point: religious adherence, the real answer balances both with the exception.

```
export class LoginPage {
  constructor(private readonly page: Page) {}

  readonly emailInput  = this.page.getByLabel('Email');
  readonly errorBanner = this.page.getByRole('alert');

  // EXCEPTION: page-state check the POM owns
  async expectLoaded(): Promise<void> {
    await expect(this.emailInput).toBeVisible();
    await expect(this.page).toHaveURL(/\/login/);
  }

  // PREFERRED: expose locator/data, let the test assert
  // getErrorMessage(): Locator { return this.errorBanner; }   // returning Locator
}

// Test does the intent assertion explicitly
test('invalid credentials shows error', async ({ loginPage }) => {
  await loginPage.expectLoaded();                                  // page-state
  await loginPage.signIn('bad@user.com', 'wrong');
...
```

Q 79 LEVEL · Senior

When do you use inheritance versus composition for sharing headers, footers, and navigation across POMs?

Inheritance: clean for uniform layouts; fails as pages diverge. Composition: explicit deps, isolated testability; more boilerplate. Rule: inheritance for is-a, composition for has-a. Modern apps have less uniform shells: composition is the default lean. The next question: composition by default in 2026, SPAs and route-specific layouts. The common miss: defaulting to inheritance because it feels OO, divergence breaks the hierarchy.

```
// COMPOSITION: default for modern web apps
export class DashboardPage {
  readonly header:  Header;
  readonly sidebar: Sidebar;
  readonly footer:  Footer;

  constructor(public readonly page: Page) {
    this.header  = new Header(page);
    this.sidebar = new Sidebar(page);
    this.footer  = new Footer(page);
  }
}

// Pages that lack the shell simply don't instantiate it, no override gymnastics
export class CheckoutPage {
  constructor(public readonly page: Page) {}
  // no header, no sidebar: by design (focused flow)
  readonly steps = this.page.getByRole('list', { name: 'Checkout steps' });
}

...
```

Q 80 LEVEL · Mid to Senior

When should action methods return a POM (fluent API), and when should they return void?

Fluent return: chain reads as user journey; type-safe navigation enforced. Void return: no implicit transitions; less coupling between POMs. Rule: fluent for linear journey-matching flows; void for divergent flows. Submit-that-might-not-navigate is the classic void case. Asked next: void for actions where next page depends on outcome. Avoid: fluent everywhere, tightly-coupled import graph across POMs.

```
// FLUENT: for linear journey flows
export class LoginPage {
  constructor(private readonly page: Page) {}
  async signIn(email: string, password: string): Promise<DashboardPage> {
    await this.page.getByLabel('Email').fill(email);
    await this.page.getByLabel('Password').fill(password);
    await this.page.getByRole('button', { name: 'Sign in' }).click();
    return new DashboardPage(this.page);
  }
}

// VOID: for divergent or might-not-navigate actions
export class SearchPage {
  constructor(private readonly page: Page) {}
  async submitSearch(query: string): Promise<void> {
    await this.page.getByRole('searchbox').fill(query);
    await this.page.getByRole('button', { name: 'Search' }).click();
    // Could land on results, no-results, or error, test decides which POM next
  }
}
...
```

Q 81 LEVEL · Senior

What is the Screenplay pattern, and when is it a better fit than POM?

Actor + Ability + Task + Question: higher abstraction than POM. Tests read as user behaviour: 'Anna logs in' not 'click submit'. Fits: **BDD-heavy teams**, **multi-persona scenarios**, **cross-screen tasks**. POM still right for engineering-only audiences, indirection isn't valuable. Rule: POM default; Screenplay deliberate for teams whose audience uses it. The easy mistake: recommending Screenplay where it isn't needed, indirection without value.

```
// Screenplay style (via @serenity-js/playwright)
import { actorCalled } from '@serenity-js/core';
import { BrowseTheWeb, LogIn } from './abilities-and-tasks';

test('Anna can see her balance', async ({ page }) => {
  const Anna = actorCalled('Anna').whoCan(BrowseTheWeb.using(page));
  await Anna.attemptsTo(
    LogIn.as('anna@bank.com', 'Secret!1'),
  );
  await Anna.attemptsTo(
    Ensure.that(TheBalance.ofAccount('savings'), equals(1_234.56)),
  );
});

// POM equivalent (more direct, less ceremony)
test('user sees savings balance', async ({ loginPage }) => {
  const dashboard = await loginPage.signIn('anna@bank.com', 'Secret!1');
  await expect(dashboard.balanceOf('savings')).toHaveText('1,234.56');
});
// Same test, different audience optimisations
```

QUICK QUIZ

1. Why are Playwright locators stored as readonly properties rather than re-found in each method?
   A) Properties are faster to type
   B) Methods cannot be async
   C) TypeScript forbids methods returning locators
   D) Locators are lazy and auto-retry, so a once-defined locator stays correct
2. How does fixture-based DI remove the new keyword from test files?
   A) The fixture constructs the page object and injects a ready instance
   B) It bans the keyword via ESLint only
   C) Page objects become global variables
   D) Tests share a single page object
3. What does marking BasePage abstract achieve?
   A) It makes methods faster
   B) It prevents direct instantiation: BasePage can only be extended
   C) It enables parallel execution
   D) It auto-generates locators
4. How does a reusable DataTable component object work?
   A) It hardcodes one table's selectors
   B) It requires a new class per table
   C) It takes a container locator and scopes all internal locators to it
   D) It only works with getByTestId
5. What is the benefit of a navigation method returning the next page object?
   A) It speeds up navigation
   B) It removes the need for waits
   C) It builds a type-safe chain so tests read like the user journey
   D) It disables URL assertions globally

Answers: 1:D 2:A 3:B 4:C 5:C

Network Interception and Mocking

Q 82 LEVEL · Mid

How does page.route() work in depth?

route(pattern, handler) intercepts every matching request. Four options: fulfill (mock), continue (pass/modify), abort (fail), fetch (real+modify). fetch makes the real call and lets you modify the real response. Follow-up: an edge case the backend won't produce, fetch-modify-fulfill. Reproduces shape-dependent bugs (e.g. only past 100 items) with no backend change. Trap: only knowing fulfill and missing fetch.

```
// Mock
await page.route('**/api/users', route => route.fulfill({
  status: 200, contentType: 'application/json',
  body: JSON.stringify([{ id: 1, name: 'Alice' }]),
}));

// Modify headers and pass through
await page.route('**/api/**', route =>
  route.continue({ headers: { ...route.request().headers(), 'X-Test-Run': 'true' } }));

// Fetch real response, modify, fulfill
await page.route('**/api/products', async route => {
  const response = await route.fetch();
  const json = await response.json();
  json.push({ id: 999, name: 'Injected', price: 0 });
  await route.fulfill({ response, json });
});
```

Q 83 LEVEL · Mid

How do you mock API responses for an entire test suite?

Define route handlers in a fixture: applies to every test. Block analytics/tracking on every test so runs do not skew dashboards. No per-test config: tests inherit mocks and blocks automatically. Keep mock payloads in version-controlled JSON files. Expect next: JSON mocks beside tests so schema changes update both together. Where this goes wrong: repeating the same route setup in every test.

```
// src/fixtures/mocks.fixture.ts
import { test as base } from '@playwright/test';
import usersData from '@data/mocks/users.json';

export const test = base.extend({
  page: async ({ page }, use) => {
    // Never pollute production analytics with test traffic
    await page.route('**/analytics/**', route => route.abort());
    await page.route('**/hotjar/**',    route => route.abort());
    // Known data state for every test
    await page.route('**/api/users', route => route.fulfill({
      status: 200, contentType: 'application/json',
      body: JSON.stringify(usersData),
    }));
    await use(page);
  },
});
```

Q 84 LEVEL · Mid

How do you test error states and edge cases using route()?

Error states are hard to trigger reliably but critical to test. 500 needs a broken backend; 401 needs token manipulation; empty needs no data. route produces all three in ms with no backend dependency. Assert the UI response: **error banner**, **redirect**, **empty-state message**. They usually ask next: happy path rarely fails, error handling silently breaks. What loses the point: testing only success responses.

```
test('shows error banner on 500', async ({ page }) => {
  await page.route('**/api/orders', route => route.fulfill({ status: 500 }));
  await page.goto('/orders');
  await expect(page.getByRole('alert')).toContainText('Something went wrong');
});

test('redirects to login on session expiry', async ({ page }) => {
  await page.route('**/api/**', route => route.fulfill({ status: 401 }));
  await page.goto('/dashboard');
  await expect(page).toHaveURL('/login');
});

test('shows empty state when no data', async ({ page }) => {
  await page.route('**/api/products', route => route.fulfill({ status: 200, body: '[]' }));
  await page.goto('/products');
  await expect(page.getByText('No products found')).toBeVisible();
});
```

Q 85 LEVEL · Mid

How do you validate request payloads using route()?

The route handler exposes method, headers and body of the request. Capture the body with postDataJSON; fulfill success, then assert the payload. Bridges UI testing and API contract testing without a real backend. The next question: unlike an API test, it exercises the UI's serialisation layer. Catches refactors that silently change a sent field name. The common miss: only asserting on rendered output, never on what the UI sends.

```
test('order creation sends correct payload', async ({ page }) => {
  let captured: any;
  await page.route('**/api/orders', async route => {
    captured = route.request().postDataJSON();
    await route.fulfill({ status: 201, body: JSON.stringify({ orderId: 'ORD-001' }) });
  });

  await page.goto('/checkout');
  await page.getByRole('button', { name: 'Place Order' }).click();

  expect(captured.items).toHaveLength(2);
  expect(captured.paymentMethod).toBe('credit_card');
  expect(captured.shippingAddress).toBeDefined();
});
```

Q 86 LEVEL · Mid to Senior

How do you use HAR files for network mocking?

HAR records all network traffic; routeFromHAR records once and replays offline. Scope replay by URL; choose fall-through or fail for unmatched requests. Best for third-party integrations you cannot control. Asked next: keep it fresh by re-recording with update; the diff shows changes. The committed HAR documents the contract the UI depends on. Avoid: hand-writing dozens of route handlers for one complex flow.

```
// Record once against the real backend
test('record HAR', async ({ page }) => {
  await page.routeFromHAR('fixtures/checkout.har', { update: true });
  await page.goto('/checkout');
  // complete the flow: all requests are recorded
});

// Replay offline in tests
test('checkout flow from HAR', async ({ page }) => {
  await page.routeFromHAR('fixtures/checkout.har', {
    url: '**/api/**',
    notFound: 'fallthrough',
  });
  await page.goto('/checkout');
  await page.getByRole('button', { name: 'Place Order' }).click();
  await expect(page.getByTestId('order-confirmation')).toBeVisible();
});
```

Q 87 LEVEL · Mid to Senior

In what order do multiple route() handlers run, and how do you compose them?

LIFO order: most recently registered handler runs first. fulfill, abort and continue all END the chain; only fallback defers. continue sends the request straight to the network, other handlers are skipped. Layered pattern: suite default + per-test specific + fallback chain. fallback() lets broader handlers handle non-matching cases. Follow-up: no fallback means the chain stops; broader handlers skipped. The easy mistake: wrong registration order, default mock always wins instead of override.

```
// Suite-level default: registered first, so it runs LAST (broad handler)
test.beforeAll(async ({ context }) => {
  await context.route('**/api/**', route =>
    route.fulfill({ status: 200, body: JSON.stringify({ ok: true }) }));
});

// Per-test override: registered later, so it runs FIRST (specific handler)
test('payment service down', async ({ page }) => {
  await page.route('**/api/payment', route =>
    route.fulfill({ status: 503, body: JSON.stringify({ error: 'down' }) }));
  // /api/payment  -> 503 (specific runs first, returns 503)
  // /api/orders   -> 200 (specific doesn't match, default runs)
  // /api/users    -> 200 (specific doesn't match, default runs)
});

// Deferring to the broader handler: fallback() is the ONLY verb that does this
await page.route('**/api/**', route => {
  if (!route.request().url().includes('/payment')) return route.fallback();
  return route.fulfill({ status: 503, body: '{}' });
});
...
```

Q 88 LEVEL · Mid

What is the difference between route.fulfill, route.continue, route.fallback, and route.abort?

fulfill: **synthetic response**, **chain stops**, **no network call**. continue: request goes to network (with optional overrides), chain stops. fallback: request goes to NEXT handler in chain (or network if none). abort: request fails with network error (DNS, blocked, etc). Expect next: continue overrides modify the network request; fallback overrides modify what the next handler sees. Trap: continue used when fallback was meant, breaks layered mocking.

```
await page.route('**/api/**', async route => {
  const req = route.request();

  // 1. Synthetic response: never touches network
  if (req.url().includes('/api/auth')) {
    return route.fulfill({ status: 200, body: JSON.stringify({ token: 't_mock' }) });
  }

  // 2. Continue to network with modifications
  if (req.url().includes('/api/legacy')) {
    return route.continue({ url: req.url().replace('/legacy/', '/v2/') });
  }

  // 3. Fail the request: simulate network error
  if (req.url().includes('/api/analytics')) {
    return route.abort('blockedbyclient');
  }

  // 4. Pass to the next handler in the chain (layered mocking)
  return route.fallback();
...
```

Q 89 LEVEL · Mid

How do you conditionally mock based on URL, method, headers, or post data?

Dispatch on: request.url(), method(), headers(), postData()/postDataJSON(). One handler for broad pattern; branch by request property inside. REST in one handler: **GET list**, **POST create**, **DELETE 204**, **fallback rest**. Discipline: mock the minimum for this test; fallback everything else. They usually ask next: mock what the test controls; fallback what it doesn't care about. Where this goes wrong: building a complete API mock, drifts from reality, tests lie.

```
await page.route('**/api/users/**', async route => {
  const req = route.request();

  // Dispatch by method
  if (req.method() === 'GET') {
    return route.fulfill({ status: 200, body: JSON.stringify([{ id: 1, name: 'Alice' }]) });
  }
  if (req.method() === 'POST') {
    const body = req.postDataJSON();        // dispatch by post data shape
    if (!body?.email) {
      return route.fulfill({ status: 400, body: JSON.stringify({ error: 'email required' }) });
    }
    return route.fulfill({ status: 201, body: JSON.stringify({ id: 99, ...body }) });
  }
  if (req.method() === 'DELETE') {
    return route.fulfill({ status: 204 });
  }

  // Auth-token-dependent: check headers
  if (!req.headers()['authorization']) {
...
```

Q 90 LEVEL · Mid to Senior

How do you take a real API response, modify it, and return the modified version?

route.fetch() gets the real response without ending the handler. Parse, mutate fields, fulfill with modified body + original headers. Use for surgical scenario injection: one out-of-stock item, one admin row. Switch to pure mock when replacing most of the response, not worth the realism. The next question: route.fetch waits for real API, slow APIs cost test runtime. What loses the point: rebuilding most of the body via this pattern, loses the realism.

```
// Surgical modification: real user list + one extra synthetic admin row
await page.route('**/api/users', async route => {
  const response = await route.fetch();              // perform the real request
  const json = await response.json() as { users: User[] };

  json.users.unshift({ id: 999_999, email: 'test-admin@local', isAdmin: true });

  await route.fulfill({
    response,                                          // preserve original status + headers
    body: JSON.stringify(json),
  });
});

// Real product catalogue with one product marked out-of-stock for the test
await page.route('**/api/products/sku-42', async route => {
  const r = await route.fetch();
  const product = await r.json();
  product.stock = 0;
  await route.fulfill({ response: r, body: JSON.stringify(product) });
});
```

Q 91 LEVEL · Mid to Senior

What is the difference between context.route and page.route, and when do you use each?

Context.route	Page.route
Use context.route for cross-cutting: env stubs, telemetry, third-party.	Use page.route for test-specific endpoint mocks.

page.route: single Page; context.route: all pages in context (incl. popups).

page.route handlers run BEFORE context.route handlers for that page.

Asked next: popups are new pages in the context, context.route catches them.

The common miss: env stubs in page.route, misses popups and new windows.

```
// In a fixture or beforeAll: cross-cutting concerns at the context
test.beforeEach(async ({ context }) => {
  // Block analytics across every page in the context
  await context.route('**/analytics.example.com/**', route => route.abort());
  // Stub a third-party script every page loads
  await context.route('**/cdn.example.com/widget.js', route =>
    route.fulfill({ body: '/* stubbed */', contentType: 'application/javascript' }));
});

// Inside a test: page-specific override
test('checkout failure', async ({ page }) => {
  await page.route('**/api/payment', route =>
    route.fulfill({ status: 503 }));            // runs BEFORE context.route
  // ... test continues
});

// Note: browser.route() does NOT exist, highest scope is context
```

Q 92 LEVEL · Mid

How do you simulate slow networks, offline mode, and connection failures?

Offline: context.setOffline(true), PWA service-worker testing. Slow: setTimeout in route handler (per-request) OR CDP throttle (full context). Failure: route.abort('failed'|'aborted'|'connectionreset'|'timedout'). Each tests a different recovery: cache (offline), timeout/loading (slow), error UI (abort). Follow-up: setTimeout in route handler is right for timeout-handling tests. Avoid: conflating offline / slow / failed, they exercise different code paths.

```
// Offline mode: PWA service-worker test
await context.setOffline(true);
// ... verify cached content renders, offline indicator shows
await context.setOffline(false);

// Slow network on a specific endpoint: test loading-state and timeout
await page.route('**/api/search', async route => {
  await new Promise(r => setTimeout(r, 5_000));   // 5s artificial delay
  await route.continue();
});

// Full-context throttle via CDP (chromium only)
const cdp = await context.newCDPSession(page);
await cdp.send('Network.emulateNetworkConditions', {
  offline: false, latency: 400, downloadThroughput: 50 * 1024, uploadThroughput: 20 * 1024,
});

// Specific connection error: test the error UI
await page.route('**/api/payment', route => route.abort('connectionreset'));
```

Q 93 LEVEL · Mid to Senior

What is routeFromHAR, and when is it the right tool for offline testing?

page.routeFromHAR replays a recorded session offline. Two phases: record with { update: true }, then replay. Captures the entire session: **APIs**, **CDN**, **third-party**, **analytics**. Use cases: **frozen regression**, **air-gapped CI**, **historical incident reproduction**. Expect next: re-record on a schedule, or never (for frozen incident repros). The easy mistake: HARs drift, stale HAR = tests against fictional behaviour.

```
// Record phase: run once against the real backend
await page.routeFromHAR('fixtures/checkout-flow.har', {
  update: true,                  // capture this session into the HAR
  updateMode: 'full',
  updateContent: 'embed',
});
// ... run the test: all requests recorded into checkout-flow.har

// Playback phase: offline, deterministic
await page.routeFromHAR('fixtures/checkout-flow.har');
// All recorded responses served from disk; no network calls

// Combine: replay HAR by default, override specific request for a scenario
await page.routeFromHAR('fixtures/checkout-flow.har');
await page.route('**/api/payment', route =>           // overrides recorded response
  route.fulfill({ status: 503 }));
```

Q 94 LEVEL · Senior

How do you mock GraphQL operations specifically, given they all hit the same endpoint?

Single endpoint /graphql: URL routing useless; dispatch on operationName. Switch on postDataJSON().operationName: mock specific ops, fallback rest. Variable-based dispatch via postDataJSON().variables for parametrised ops. GraphQL's advantage: operations named explicitly, dispatch is discoverable. They usually ask next: subscriptions use WebSocket, page.routeWebSocket, not page.route. Trap: URL-routing GraphQL, every operation hits the same URL.

```
await page.route('**/graphql', async route => {
  const body = route.request().postDataJSON() as {
    operationName: string;
    variables?: Record<string, unknown>;
  };

  switch (body.operationName) {
    case 'GetUsers':
      return route.fulfill({
        status: 200,
        body: JSON.stringify({ data: { users: [{ id: 1, name: 'Alice' }] } }),
      });

    case 'GetUser':
      // Variable-based dispatch
      const userId = body.variables?.id;
      const user = userId === '1'
        ? { id: 1, name: 'Alice', isAdmin: true }
        : { id: userId, name: 'User', isAdmin: false };
      return route.fulfill({ status: 200, body: JSON.stringify({ data: { user } }) });
...
```

Q 95 LEVEL · Senior

What is the right strategy for what to mock and what not to mock?

Rule: mock external dependencies, never the system under test. External: **third-party APIs**, **CDN**, **auth providers**, **rate limits**, **cost**, **PCI**. Mocking your own backend hides bugs that break prod. Three exceptions: cross-team contracts, slow internal at unit level, destructive ops. The next question: mock own service for speed at unit level, never at integration level. Where this goes wrong: extensive mocking loses integration confidence, prod breaks anyway.

```
// CORRECT: mock external third-party (Stripe), keep our backend live
await context.route('**/api.stripe.com/**', route =>
  route.fulfill({ status: 200, body: JSON.stringify({ id: 'pi_mock', status: 'succeeded' }) }));

// Our own /api/checkout is LIVE: we're verifying the integration
// await page.route('**/api/checkout', ...)   // ✗ do NOT mock the SUT

// EXCEPTION: destructive op, mock for safety even when internal
await context.route('**/api/admin/delete-all-users', route =>
  route.fulfill({ status: 200, body: JSON.stringify({ deleted: 0 }) }));

// EXCEPTION: cross-team service with a contract
await context.route('**/billing-service.internal/**', route =>
  route.fulfill({ status: 200, body: mockBillingContract }));
```

Q 96 LEVEL · Mid

page.route (intercept/mock) vs APIRequestContext (direct API call), when do you mock the network vs hit the API directly?

page.route shapes traffic the browser generates; APIRequestContext makes a real call with no browser. Mock with page.route when testing the UI's reaction to a response. Use request/page.request when the API itself is under test, or to seed and tear down state. They are complementary: arrange state via API, control the UI's network via route. Asked next: seed a logged-in user by POSTing to auth via request and reusing storage state. What loses the point: treating them as rivals, or using page.route for setup that a direct call does faster.

```
// page.route: control what the UI receives (testing the front end's reaction)
test('shows error banner on 500', async ({ page }) => {
  await page.route('**/api/orders', route => route.fulfill({ status: 500 }));
  await page.goto('/orders');
  await expect(page.getByRole('alert')).toBeVisible();
});

// APIRequestContext: talk to the backend directly (no browser)
test('order API contract', async ({ request }) => {
  const res = await request.post('/api/orders', { data: { sku: 'A-1', qty: 2 } });
  expect(res.status()).toBe(201);
  expect((await res.json()).orderId).toBeDefined();
});

// Combined: arrange state fast via API, then drive the UI
test('dashboard for a seeded user', async ({ page, request }) => {
  // Seed + log in directly: no clicking through the login form
  await request.post('/api/test/users', { data: { email: 'sdet@local', plan: 'pro' } });
  const login = await request.post('/api/auth/login',
    { data: { email: 'sdet@local', password: 'pw' } });
...
```

Q 97 LEVEL · Mid

How do you block images, CSS, fonts, and third-party resources with page.route to speed up tests, and what are the risks?

- Abort heavy requests by route.request().resourceType(): **image**, **stylesheet**, **font**, **media**, **script**.
- One context-level handler blocks the heavy types and lets HTML + API through.
- Real speed-up on image-heavy pages and slow CI.
- Risk: blocking CSS breaks visibility/layout/screenshot assertions; images break image-load tests.
- Risk: killing a wired-in third-party script can break the app; environment drifts from prod.
- Rule: block on functional/API tests only, never visual; analytics safe, CSS not.

```
// Context-level: block heavy resource types for the whole suite
const BLOCKED = new Set(['image', 'stylesheet', 'font', 'media']);

test.beforeEach(async ({ context }) => {
  await context.route('**/*', route => {
    const type = route.request().resourceType();
    return BLOCKED.has(type) ? route.abort() : route.continue();
  });
});

// Almost always safe: block analytics / trackers by URL
await context.route('**/{analytics,gtm,hotjar,segment}.**', route => route.abort());

// DANGER, do NOT block CSS on a visual / layout-sensitive test:
// no stylesheet -> no layout -> toBeVisible() can fail on a present element,
// and any screenshot assertion is meaningless. Keep blocking off here.
test('checkout layout @visual', async ({ page }) => {
  // (no resource blocking in this test)
  await page.goto('/checkout');
  await expect(page).toHaveScreenshot('checkout.png');
...
```

QUICK QUIZ

1. Which route option makes the real API call and lets you modify the real response?
   A) fulfill
   B) continue
   C) fetch
   D) abort
2. Why block analytics requests in a shared fixture?
   A) Analytics slow tests down
   B) It is required for sharding
   C) Analytics break auto-waiting
   D) Test traffic should not pollute production analytics dashboards
3. Why is route() the best tool for testing a 401 session-expiry redirect?
   A) It is faster than real login
   B) A 401 is hard to trigger reliably without manipulating session tokens
   C) Playwright cannot test real 401s
   D) It avoids using assertions
4. How do you assert the payload the UI sent inside a route handler?
   A) route.response().json()
   B) page.content()
   C) route.request().postDataJSON()
   D) route.fulfill().body()
5. What does page.routeFromHAR let you do after recording a session?
   A) Replay all matching requests offline from the archive
   B) Generate load tests
   C) Convert HAR to JUnit
   D) Record video of the run

Answers: 1:C 2:D 3:B 4:C 5:A

API Testing with Playwright

Q 98 LEVEL · Mid

How do you use Playwright's APIRequestContext for API testing?

request fixture is a full HTTP client with the same expect API. No Axios or Supertest needed: **one framework**, **one runner**, **one report**. Same assertions work on API responses and DOM elements. Follow-up: tests/api for contracts, tests/integration for hybrid flows. 2026 reality: one framework for both layers is the expectation. The common miss: saying you would add Supertest/Axios, signals you skipped the fixture.

```
import { test, expect } from '@playwright/test';

test('GET /api/users returns a paginated list', async ({ request }) => {
  const response = await request.get('/api/users', { params: { page: 1, limit: 10 } });
  expect(response.status()).toBe(200);
  const body = await response.json();
  expect(body.data).toHaveLength(10);
  expect(body.page).toBe(1);
});

test('POST /api/users creates a user', async ({ request }) => {
  const response = await request.post('/api/users', {
    data: { name: 'Test User', email: 'test@example.com', role: 'user' },
  });
  expect(response.status()).toBe(201);
  expect((await response.json()).id).toBeDefined();
});
```

Q 99 LEVEL · Mid

How do you authenticate API requests in Playwright?

Build an APIRequestContext in a fixture; reuse storageState from globalSetup. One auth source feeds both UI and API tests: no separate token mgmt. Dispose the context after each test to prevent session leakage. One context per role (admin, user) with baseURL and headers. Expect next: the admin context seeds data ~10x faster than the UI. Avoid: hand-rolling per-test token management.

```
// src/fixtures/api.fixture.ts
import { test as base, APIRequestContext } from '@playwright/test';

type APIFixtures = { adminAPI: APIRequestContext };

export const test = base.extend<APIFixtures>({
  adminAPI: async ({ playwright }, use) => {
    const context = await playwright.request.newContext({
      baseURL: process.env.API_URL,
      storageState: 'auth/admin-state.json',
      extraHTTPHeaders: { 'Accept': 'application/json' },
    });
    await use(context);
    await context.dispose(); // prevent session leakage
  },
});
```

Q 100 LEVEL · Mid

How do you implement API contract testing with Playwright?

Validate field names, types, allowed values and format, not exact values. TypeScript interfaces make the contract test type-safe. The safety net between frontend and backend teams. Run as the fast first stage: no browser, catches breaking changes early. They usually ask next: zod adds runtime schema validation on top of compile-time types. The easy mistake: asserting exact values for generated fields like IDs.

```
interface UserResponse {
  id: number; name: string; email: string;
  role: 'admin' | 'user' | 'viewer'; createdAt: string;
}

test('GET /api/users/:id matches the contract', async ({ request }) => {
  const response = await request.get('/api/users/1');
  await expect(response).toBeOK();
  const user: UserResponse = await response.json();
  expect(typeof user.id).toBe('number');
  expect(['admin', 'user', 'viewer']).toContain(user.role);
  expect(new Date(user.createdAt).toString()).not.toBe('Invalid Date');
  expect(user.email).toMatch(/^[^\s@]+@[^\s@]+\.[^\s@]+$/);
});
```

Q 101 LEVEL · Mid

How do you combine API setup with UI verification in tests?

Pattern: **API create**, **UI verify**, **API clean up**. ~30s to create via UI vs ~200ms via API: faster and more reliable. The test verifies the UI displays data, not the creation form. The next question: the UI creation flow is tested once, in its own dedicated test. Keeps a large integration suite fast and honest about what it verifies. Trap: doing all setup through the UI, slow and conflates concerns.

```
test('created user appears in the admin list', async ({ request, page }) => {
  // Setup via API: fast and reliable
  const created = await request.post('/api/users', {
    data: { name: 'Test User', email: 'testuser@example.com', role: 'user' },
  });
  const { id } = await created.json();

  // Verify through the UI
  await page.goto('/admin/users');
  const row = page.locator('tbody tr').filter({ hasText: 'testuser@example.com' });
  await expect(row).toBeVisible();
  await expect(row.getByTestId('role-badge')).toHaveText('user');

  // Cleanup via API: no UI teardown
  await request.delete(`/api/users/${id}`);
});
```

Q 102 LEVEL · Mid to Senior

How do you test GraphQL APIs with Playwright?

Query natively via the request fixture: post query + variables, assert on data. All GraphQL hits one endpoint: mock by operation name, not URL. Match the operation in the body; continue for everything else. Asked next: shared operation-name constants fail typecheck on a rename. Keeps mocks in sync and avoids a silently unmatched mock. Where this goes wrong: trying to match GraphQL mocks by URL, matches everything.

```
// Query natively
test('fetch user via GraphQL', async ({ request }) => {
  const response = await request.post('/graphql', {
    data: { query: 'query GetUser($id: ID!) { user(id: $id) { id email } }', variables: { id: '1' } },
  });
  await expect(response).toBeOK();
  expect((await response.json()).data.user.email).toBeDefined();
});

// Mock by operation name (URL is shared)
await page.route('**/graphql', async route => {
  const body = route.request().postDataJSON();
  if (body.query.includes('GetProducts')) {
    await route.fulfill({ status: 200, body: JSON.stringify({ data: { products: [] } }) });
  } else {
    await route.continue();
  }
});
```

Q 103 LEVEL · Mid to Senior

What are the three ways to get an APIRequestContext, and when do you use each?

- request fixture: **standalone**, **fresh per test**, **no browser**, **pure-API tests**.
- page.request: shares cookies/storage with the browser, same session as UI.
- playwright.request.newContext(): **custom baseURL**, **headers**, **storageState**.
- Rule: fixture for pure-API; page.request for browser-auth; newContext for custom.
- Follow-up: page.request when API must see the same session as the UI.
- What loses the point: request fixture when you needed the browser's auth, different session.

```
// 1. request fixture: pure-API, no browser overhead
test('GET /users returns users', async ({ request }) => {
  const res = await request.get('/api/users');
  expect(res.status()).toBe(200);
});

// 2. page.request: shares browser auth
test('UI action reflects in API', async ({ page }) => {
  await page.goto('/profile');
  await page.getByLabel('Display name').fill('New Name');
  await page.getByRole('button', { name: 'Save' }).click();
  // Same session as the browser: sees the just-saved state
  const res = await page.request.get('/api/me');
  expect((await res.json()).displayName).toBe('New Name');
});

// 3. Custom newContext: specific headers/baseURL
const apiCtx = await playwright.request.newContext({
  baseURL: process.env.API_URL,
  extraHTTPHeaders: { 'X-API-Key': process.env.API_KEY!, 'X-Test-Mode': 'true' },
...
```

Q 104 LEVEL · Senior

How do you reuse authentication tokens across many tests without re-authenticating each time?

storageState: long-lived browser auth via globalSetup/setup project. extraHTTPHeaders: token loaded from setup file, UI + pure-API both. Worker-scoped fixture: short-lived tokens, authenticated once per worker. Rule: match auth scope to token lifetime (long/medium/ephemeral). Expect next: refresh on 401 with single in-flight to avoid thundering herd. The common miss: per-test login, 5-8s × 500 tests = +60min of pure auth overhead.

```
// Strategy 1: storageState (long-lived session)
// auth.setup.ts saves: page.context().storageState({ path: 'auth/admin.json' })
// playwright.config.ts, use: { storageState: 'auth/admin.json' }

// Strategy 2: Bearer token via extraHTTPHeaders
const token = JSON.parse(fs.readFileSync('auth/api-token.json', 'utf8')).token;
const apiCtx = await playwright.request.newContext({
  baseURL: process.env.API_URL,
  extraHTTPHeaders: { Authorization: `Bearer ${token}` },
});

// Strategy 3: worker-scoped fixture for short-lived tokens
export const test = base.extend<{}, { sharedToken: string }>({
  sharedToken: [async ({}, use) => {
    const token = await fetchTokenFromAuthService();
    await use(token);
  }, { scope: 'worker' }],
});
```

Q 105 LEVEL · Senior

How do you validate API responses against a JSON Schema or OpenAPI specification?

Catches silent shape changes: **wrong type**, **missing required**, **unknown field**. ajv for JSON Schema (often generated from OpenAPI). zod for TypeScript-first: validator type AND TS type from one source. Generate from OpenAPI when available: test enforces published contract. They usually ask next: no OpenAPI? Use zod schemas as the contract; formalise later. Avoid: field-by-field assertions pass when fields have the wrong type.

```
import { z } from 'zod';

// One source of truth: runtime validator AND TypeScript type
const UserSchema = z.object({
  id:      z.number().int().positive(),
  email:   z.string().email(),
  isAdmin: z.boolean(),
  createdAt: z.string().datetime(),
});
const UsersResponseSchema = z.object({ users: z.array(UserSchema), total: z.number() });
type UsersResponse = z.infer<typeof UsersResponseSchema>;

test('GET /users matches contract', async ({ request }) => {
  const res = await request.get('/api/users');
  expect(res.status()).toBe(200);

  const json: unknown = await res.json();
  const parsed = UsersResponseSchema.parse(json);   // throws on shape mismatch
  expect(parsed.users.length).toBeGreaterThan(0);   // domain assertion after schema OK
});
...
```

Q 106 LEVEL · Mid

How do you upload files via API requests (multipart/form-data)?

multipart option on request.post() handles multipart/form-data correctly. Three value shapes: string (form field), file path (disk), buffer object. Buffer shape: { name, mimeType, buffer } for in-memory files. Discipline: commit fixtures <50KB; generate large/invalid ones at test time. The next question: 10 MB rejection test uses Buffer.alloc, keeps repo small. The easy mistake: hand-built FormData with manual boundary handling, use the option.

```
// File path from disk: the happy path
test('uploads a PDF', async ({ request }) => {
  const res = await request.post('/api/documents', {
    multipart: {
      file: 'tests/fixtures/sample.pdf',         // disk path
      category: 'invoice',                       // regular form field
      notes: 'Q1 expenses',
    },
  });
  expect(res.status()).toBe(201);
});

// Generated buffer: test the rejection of oversized files
test('rejects 10 MB upload with 413', async ({ request }) => {
  const res = await request.post('/api/documents', {
    multipart: {
      file: { name: 'big.pdf', mimeType: 'application/pdf', buffer: Buffer.alloc(10 * 1024 * 1024) },
      category: 'invoice',
    },
  });
...
```

Q 107 LEVEL · Mid to Senior

How do you assert on response time and SLA targets in API tests?

Measure with performance.now() around the request, no built-in timing field. Soft upper bound from SLA, not happy-path observed time. SLA 500ms p95 -> assert <1000ms in tests to absorb CI variance. Track statistics across runs: trend > single-test value. Asked next: percentile-level perf needs k6/Artillery/Locust, not Playwright. Trap: asserting on happy-path times, flakes every time CI is slow.

```
test('GET /products meets SLA', async ({ request }) => {
  const start = performance.now();
  const res = await request.get('/api/products?limit=50');
  const durationMs = performance.now() - start;

  expect(res.status()).toBe(200);
  // SLA: 500ms p95. Test threshold: 1000ms (soft upper bound)
  expect(durationMs).toBeLessThan(1000);

  // For trend tracking: write to a metrics file or reporter
  test.info().annotations.push({ type: 'duration_ms', description: String(durationMs) });
});

// Wrong: asserting near observed happy-path time, flakes on slow CI runs
// expect(durationMs).toBeLessThan(150);

// For real percentile performance testing: k6, Artillery, Locust
```

Q 108 LEVEL · Mid to Senior

How do you test rate limiting (429 responses) and retry logic?

Two halves: API enforces 429; client handles 429, both matter. Enforcement test: loop requests to trigger, assert 429 + Retry-After. Client test: page.route injects 429; assert backoff/cap/UI state. Respect Retry-After in test code: spamming exhausts shared rate limits. Follow-up: exponential backoff via successive Retry-After values 1, 2, 4... Where this goes wrong: fire requests until 429 and stop, doesn't verify recovery.

```
// Half 1: verify the API enforces the rate limit
test('API enforces 100/min rate limit', async ({ request }) => {
  const responses = await Promise.all(
    Array.from({ length: 105 }, () => request.get('/api/search?q=test'))
  );
  const rateLimited = responses.filter(r => r.status() === 429);
  expect(rateLimited.length).toBeGreaterThanOrEqual(5);
  expect(rateLimited[0]!.headers()['retry-after']).toBeDefined();
});

// Half 2: verify the client handles 429 with backoff and recovery
test('app retries after Retry-After on 429', async ({ page }) => {
  let calls = 0;
  await page.route('**/api/data', async route => {
    calls++;
    if (calls === 1) {
      return route.fulfill({ status: 429, headers: { 'retry-after': '2' } });
    }
    return route.fulfill({ status: 200, body: JSON.stringify({ ok: true }) });
  });
...
```

Q 109 LEVEL · Senior

How do you build typed test-data factories that combine branded IDs with automatic cleanup?

Factory returns branded types (UserId, OrderId), no ID-mixup bugs. Fluent builder reads like the scenario: asAdmin().withEmail().create(). Cleanup fixture tracks IDs; deletes in afterEach, even on test failure. Cleanup wrapped in try-catch so one failure doesn't block other deletions. Expect next: faker + worker index for unique data, no parallel collisions. What loses the point: cleanup that skips on failure, orphan data flakes future tests.

```
type UserId = string & { __brand: 'UserId' };

export class UserBuilder {
  private data: Partial<{ email: string; isAdmin: boolean }> = {};
  asAdmin()                   { this.data.isAdmin = true; return this; }
  withEmail(email: string)    { this.data.email = email;  return this; }

  async create(request: APIRequestContext, registry: Set<string>): Promise<UserId> {
    const email = this.data.email ?? `test+${faker.string.uuid()}@local`;
    const res = await request.post('/api/users', { data: { email, ...this.data } });
    expect(res.status()).toBe(201);
    const id = (await res.json()).id as UserId;
    registry.add(id);
    return id;
  }
}

// Cleanup fixture: runs regardless of test outcome
export const test = base.extend<{ cleanup: Set<string>; request: APIRequestContext }>({
  cleanup: async ({ request }, use) => {
...
```

Q 110 LEVEL · Mid to Senior

How do you test cursor-based vs offset-based pagination?

Offset: **page count**, **boundaries**, **out-of-range**, **insertion-shift bug**. Cursor: traversal completeness, invalid cursor -> 400 not 500, opaque rule. NEVER construct cursors in test code: only echo from previous response. Test boundary conditions: **first/last/beyond**, **empty**, **size=1**, **size=max**. They usually ask next: classic offset bug is the same item on two pages after insert. The common miss: offset and cursor tested identically, different failure modes.

```
// Offset-based: catch the insertion-shift bug
test('offset pagination is stable across inserts', async ({ request, cleanup }) => {
  const page1 = await (await request.get('/api/items?page=1&pageSize=20')).json();
  await new ItemBuilder().create(request, cleanup);   // insert between fetches
  const page2 = await (await request.get('/api/items?page=2&pageSize=20')).json();

  const ids1 = new Set(page1.items.map((i: any) => i.id));
  const dupes = page2.items.filter((i: any) => ids1.has(i.id));
  expect(dupes).toEqual([]);                          // no duplicates across pages
});

// Cursor-based: echo cursor, don't construct
test('cursor pagination traverses the full set', async ({ request }) => {
  const seen = new Set<number>();
  let cursor: string | undefined = undefined;
  for (;;) {
    const url = cursor ? `/api/items?cursor=${cursor}&limit=20` : '/api/items?limit=20';
    const json = await (await request.get(url)).json();
    for (const item of json.items) {
      expect(seen.has(item.id)).toBe(false);          // no duplicates
...
```

Q 111 LEVEL · Senior

How do you test a microservice in isolation when it depends on other services?

Stand up the service under test live; mock its downstream dependencies. Intercept via proxy or by pointing HTTP client base URL at a mock server. Tools: WireMock, MSW, or a custom Express stub, controlled responses. Contract tests (Pact, Spring Cloud Contract) prevent mock drift. The next question: shared contract files between consumer and provider keep mocks honest. Avoid: mocking everything without verifying contract, tests pass against fiction.

```
// Service under test runs at http://localhost:8080 (real, live)
// Its downstream user-service is configured via env var to use the mock
//   USER_SERVICE_URL=http://localhost:9000   (WireMock or MSW)

test('billing-service handles user-service 404 gracefully', async ({ request }) => {
  // Configure the downstream mock for this scenario
  await fetch('http://localhost:9000/__admin/mappings', {
    method: 'POST',
    body: JSON.stringify({
      request:  { method: 'GET', urlPattern: '/users/9999' },
      response: { status: 404, body: '{"error":"not found"}' },
    }),
  });

  // Test the service under test: live, real code path
  const res = await request.post('http://localhost:8080/invoices', {
    data: { userId: 9999, amount: 100 },
  });
  expect(res.status()).toBe(404);                 // handles downstream 404
  expect((await res.json()).error).toBe('user not found');
...
```

Q 112 LEVEL · Mid

What does storageState actually store, and how do you save and load it?

storageState saves cookies + localStorage to a JSON file. Save: context.storageState({ path }); load: use { storageState }. Caveat: **sessionStorage is NOT captured**, **it is per-tab**, **not persisted**. Login once, not per test: the biggest single lever for suite speed. Asked next: sessionStorage auth needs addInitScript to seed the token. The easy mistake: assuming storageState covers everything, sessionStorage is the gap.

```
// Save after login: in auth.setup.ts (a setup project)
import { test as setup } from '@playwright/test';

setup('authenticate as user', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill(process.env.USER_EMAIL!);
  await page.getByLabel('Password').fill(process.env.USER_PASSWORD!);
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('/dashboard');
  // Cookies + localStorage written to disk; sessionStorage is NOT
  await page.context().storageState({ path: 'auth/user.json' });
});

// Load: in playwright.config.ts
// use: { storageState: 'auth/user.json' }

// If auth lives in sessionStorage, seed it (storageState won't):
// await context.addInitScript(t => sessionStorage.setItem('token', t), token);
```

Q 113 LEVEL · Mid to Senior

storageState reuse vs logging in per test: what's the trade-off, and when is logging in per test the right call?

Reuse: login once, fast and stable, the default for most tests. Per-test login: slow (5-8s each) and a shared failure point. Log in per test when the login flow is the thing under test. Also when you need a fresh unique session, or the token is very short-lived. Heuristic: reuse by default; log in only when the session is what you verify. Trap: per-test login everywhere ‘for realism’, slow, fragile, no real gain.

```
// Default: reuse saved state (fast, stable)
// playwright.config.ts:  use: { storageState: 'auth/user.json' }
test('dashboard shows the user widgets', async ({ page }) => {
  await page.goto('/dashboard');           // already logged in
  await expect(page.getByTestId('widgets')).toBeVisible();
});

// Exception: the login flow IS the test (no reuse)
test.use({ storageState: { cookies: [], origins: [] } });  // start logged out
test('login rejects a wrong password', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('user@example.com');
  await page.getByLabel('Password').fill('wrong-password');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByRole('alert')).toHaveText(/invalid credentials/i);
});
```

Q 114 LEVEL · Mid to Senior

How do you handle multiple authenticated roles (admin vs standard user) with separate storageState files and projects?

One storageState file per role: auth/admin.json, auth/user.json. One project per role, each with its own use { storageState }. Each role project sets dependencies: ['setup'] so logins run first. Parameterise login over a roles array: new role = new entry, not copy-paste. Two roles in one test: open a second context from the other role's state. Where this goes wrong: copy-pasted per-role setup tests that drift out of sync.

```
// playwright.config.ts: setup project + one project per role
projects: [
  { name: 'setup', testMatch: /auth\.setup\.ts/ },
  {
    name: 'admin',
    testMatch: /.*\.admin\.spec\.ts/,
    use: { storageState: 'auth/admin.json' },
    dependencies: ['setup'],
  },
  {
    name: 'user',
    testMatch: /.*\.user\.spec\.ts/,
    use: { storageState: 'auth/user.json' },
    dependencies: ['setup'],
  },
],

// auth.setup.ts: parameterise over roles, no copy-paste
const roles = [
  { name: 'admin', email: process.env.ADMIN_EMAIL!, file: 'auth/admin.json' },
...
```

Q 115 LEVEL · Mid

How do you test logged-out behaviour and reset or clear authentication state between tests?

Override the shared state: test.use({ storageState: { cookies: [], origins: [] } }). Use it for login redirects, public pages, and unauthenticated guards. Clear mid-test: context.clearCookies() + localStorage, or open a fresh context. Shared storageState is loaded read-only per context, logout doesn't leak. A dedicated logged-out project beats per-test overrides for a whole batch. What loses the point: trying to mutate the shared state file, override to empty instead.

```
// Logged-out test: override the project's shared state to empty
import { test, expect } from '@playwright/test';

test.describe('unauthenticated access', () => {
  test.use({ storageState: { cookies: [], origins: [] } });

  test('redirects to login when not signed in', async ({ page }) => {
    await page.goto('/dashboard');
    await expect(page).toHaveURL(/\/login/);
  });
});

// Clear state mid-test: cookies + localStorage
test('after logout the session is gone', async ({ page, context }) => {
  await page.goto('/dashboard');
  await context.clearCookies();
  await page.evaluate(() => localStorage.clear());
  await page.goto('/dashboard');
  await expect(page).toHaveURL(/\/login/);   // session truly cleared
});
```

QUICK QUIZ

1. Which Playwright fixture provides a full HTTP client with the same expect API as UI tests?
   A) page
   B) browser
   C) context
   D) request
2. What is the elegant way to authenticate API requests that reuses browser auth?
   A) Hardcode a bearer token in each test
   B) Reuse the storageState saved by the setup project
   C) Log in through the UI before every API call
   D) Disable authentication in staging
3. Why run API contract tests as the first stage in CI?
   A) They are fast (no browser) and catch breaking schema changes before UI tests
   B) They need a browser, so run them early
   C) They replace UI tests entirely
   D) They cannot run in parallel
4. In the API-setup-plus-UI-verification pattern, why create the test data via API?
   A) The UI cannot create data
   B) API creation skips validation
   C) It is far faster and keeps the test focused on verifying display, not creation
   D) It avoids using fixtures
5. Why must GraphQL mocks match on operation name rather than URL?
   A) GraphQL uses GET requests
   B) All GraphQL requests share one endpoint, so a URL match catches every query
   C) Operation names are faster to parse
   D) URLs are not available in route handlers

Answers: 1:D 2:B 3:A 4:C 5:B

Assertions and the Expect API

Q 116 LEVEL · Junior to Mid

What are all the web-first assertions in Playwright, and what are soft assertions?

- Web-first assertions retry until condition met or timeout.
- Catalogue: **visibility**, **state**, **content**, **count**, **attribute**, **page**.
- expect.soft collects all failures before throwing.
- Ten-field form: soft assertions report every broken field, not just the first.
- Follow-up: soft for the full picture, hard to halt immediately.
- The common miss: treating one-shot isVisible as an assertion.

```
// All retry automatically
await expect(page.locator('#status')).toHaveText('Order Confirmed');
await expect(page.locator('#items')).toHaveCount(3);
await expect(page).toHaveURL('/dashboard');

// Soft assertions: collect every failure in one run
await expect.soft(page.getByTestId('name')).toHaveText('Alice');
await expect.soft(page.getByTestId('email')).toHaveText('alice@test.com');
await expect.soft(page.getByTestId('role')).toHaveText('Admin');
// all three run; the test reports all failures at once
```

Q 117 LEVEL · Mid

How do you use toHaveScreenshot() for visual regression testing?

Compares to a committed baseline with a configurable maxDiffPixelRatio. mask hides dynamic content (prices, timestamps) to avoid false failures. animations: 'disabled' stops spurious pixel diffs. Baselines live in the repo; update deliberately with update-snapshots. Expect next: **intentional design change**, **regenerate baseline**, **review the diff**. Avoid: screenshotting dynamic data, endless false failures.

```
test('product card matches the design', async ({ page }) => {
  await page.goto('/products/macbook-pro');
  await expect(page.getByTestId('product-card')).toHaveScreenshot('product-card.png', {
    mask: [page.locator('.price-badge'), page.locator('.stock-count')],
    maxDiffPixelRatio: 0.02,
  });
});

await expect(page).toHaveScreenshot('dashboard-full.png', {
  fullPage: true,
  animations: 'disabled',
});
```

Q 118 LEVEL · Mid to Senior

How do you use expect.poll() for eventually consistent systems?

expect.poll calls a function repeatedly and asserts on the result. Right tool for async, eventually consistent systems (queue -> worker -> DB). Progressive backoff: faster than fixed polling, clearer than a while loop. Returns the instant the condition is true: fast and patient as needed. They usually ask next: why not sleep, fixed waits are flaky if short, slow if long. The easy mistake: suggesting a hard sleep for eventual consistency.

```
// Poll the status API until the order is confirmed
await expect.poll(async () => {
  const res = await page.request.get('/api/orders/42');
  return (await res.json()).status;
}, {
  message: 'Order should reach Confirmed within 30s',
  timeout: 30_000,
  intervals: [1_000, 2_000, 5_000, 10_000], // progressive backoff
}).toBe('confirmed');
```

Q 119 LEVEL · Mid

How do you assert on API responses using Playwright's expect?

toBeOK for 2xx; assert exact status and headers where needed. toMatchObject + expect.any() + stringMatching for flexible schema checks. Validate type and format, not exact values for generated fields. A format regex for email beats a literal address, it tests the rule. The next question: unpredictable fields, expect.any for type, stringMatching for format. Trap: asserting exact IDs/timestamps, brittle on every data change.

```
test('API response assertions', async ({ request }) => {
  const response = await request.get('/api/users/1');
  await expect(response).toBeOK();
  expect(response.headers()['content-type']).toContain('application/json');

  const user = await response.json();
  expect(user).toMatchObject({
    id: expect.any(Number),
    name: expect.any(String),
    email: expect.stringMatching(/^[^\s@]+@[^\s@]+\.[^\s@]+$/),
  });
});
```

Q 120 LEVEL · Mid to Senior

How do you use custom matchers to extend the expect API?

expect.extend adds domain-specific assertions: readable and reusable. toBeLoggedIn bundles multi-step checks behind one expressive call. Add the TypeScript declaration for autocomplete and type safety. Login rule changes -> edit one matcher, not dozens of tests. Asked next: worth it for repeated multi-step domain checks, not one-liners. Where this goes wrong: wrapping a single built-in assertion, indirection for no gain.

```
// src/utils/custom-matchers.ts
import { expect } from '@playwright/test';

expect.extend({
  async toBeLoggedIn(page) {
    const onApp = !page.url().includes('/login');
    const hasCookie = await page.evaluate(() => document.cookie.includes('auth_token'));
    const pass = onApp && hasCookie;
    return { pass, message: () => `Expected logged-in. url ok: ${onApp}, cookie: ${hasCookie}` };
  },
});

// TypeScript declaration: autocomplete + type safety
declare global {
  namespace PlaywrightTest {
    interface Matchers<R> { toBeLoggedIn(): R; }
  }
}

await expect(page).toBeLoggedIn();
```

Q 121 LEVEL · Senior

How do you operate visual regression in production: baseline updates, masking, and threshold tuning?

Baselines in git + code review: prevents regression-becomes-new-correct. mask: […] excludes legitimately-changing regions from diff. maxDiffPixels / maxDiffPixelRatio absorb anti-alias + OS/GPU noise. Set the lowest stable threshold; revisit when flaking, don't just raise. Follow-up: **mask timestamps**, **ad slots**, **random dynamic content**. What loses the point: keep-raising-threshold, hides real visual regressions.

```
// Production-grade visual regression
test('product card visual', async ({ page }) => {
  await page.goto('/products/sku-42');

  await expect(page.getByTestId('product-card')).toHaveScreenshot('product-card.png', {
    // 1. Mask legitimately-dynamic regions
    mask: [
      page.getByTestId('timestamp'),
      page.getByTestId('ad-slot'),
    ],
    // 2. Threshold for anti-alias / GPU noise (lowest stable value)
    maxDiffPixels: 50,
    threshold: 0.2,
    // 3. Disable animations for deterministic capture
    animations: 'disabled',
  });
});

// Baseline update workflow:
//   npx playwright test --update-snapshots   -> git diff -> PR -> review -> merge
...
```

Q 122 LEVEL · Mid to Senior

When do you use soft assertions versus fail-fast, and what's the discipline?

Fail-fast is Playwright's default: first failure stops the test. expect.soft continues, collects, reports all failures at end. Right uses: independent dashboard fields, end-of-test verification. Rule: soft for independent assertions, fail-fast for dependent ones. Expect next: soft on login then assertions on dashboard = cascade noise. The common miss: soft everywhere, cascade failures obscure root cause.

```
// GOOD: soft on independent dashboard fields
test('user profile dashboard', async ({ page }) => {
  await page.goto('/profile');
  // Each field is independent; one wrong doesn't invalidate the others
  await expect.soft(page.getByTestId('name')).toHaveText('Anna Banerjee');
  await expect.soft(page.getByTestId('email')).toHaveText('anna@bank.com');
  await expect.soft(page.getByTestId('tenant')).toHaveText('Mumbai');
  await expect.soft(page.getByTestId('role')).toHaveText('Admin');
  // All failures reported at the end of this test
});

// BAD: soft on a dependency causes cascade failures
// await expect.soft(page.getByText('Welcome back')).toBeVisible();  // login succeeded?
// await expect(page.getByTestId('orders').count()).resolves.toBe(5);  // meaningless if login failed

// CORRECT: fail-fast on the precondition
await expect(page.getByText('Welcome back')).toBeVisible();   // fail-fast: stops if login failed
await expect.soft(page.getByTestId('orders').count()).resolves.toBe(5);
```

Q 123 LEVEL · Mid to Senior

How do you use toMatchSnapshot for non-image artifacts like JSON or HTML?

toMatchSnapshot compares any string/Buffer to a stored baseline. Uses: **JSON shape**, **HTML fragments**, **CSV exports**, **generated markdown**. Normalise BEFORE snapshot: **strip timestamps/IDs**, **sort arrays**, **round floats**. Use when entire shape matters; targeted assertions when only fields matter. They usually ask next: normalisation strips non-deterministic fields + sorts unstable arrays. Avoid: snapshotting raw responses with timestamps, constant churn, snapshots stop catching bugs.

```
// JSON snapshot with normalisation
test('GET /users response shape', async ({ request }) => {
  const res  = await request.get('/api/users');
  const json = await res.json() as { users: Array<{ id: string; createdAt: string; email: string }> };

  // Normalise: strip non-deterministic fields, sort by stable key
  const normalised = {
    users: json.users
      .map(({ createdAt, id, ...rest }) => rest)             // strip id, createdAt
      .sort((a, b) => a.email.localeCompare(b.email)),         // stable order
  };

  expect(JSON.stringify(normalised, null, 2)).toMatchSnapshot('users-response.json');
});

// HTML fragment
// expect(await page.locator('[data-testid="summary"]').innerHTML()).toMatchSnapshot('summary.html');

// Update workflow: npx playwright test --update-snapshots -> review diff -> PR
```

Q 124 LEVEL · Mid to Senior

How does the .not negation work, and what's the timing trap with it?

.not retries the same as positive: polls until negation true or timeout. not.toBeVisible waits the FULL timeout if element is currently visible. Right for asserting disappearance; wrong for asserting never-appeared. Prefer asserting the POSITIVE expected state instead of negative absence. The next question: assert success toast + new URL, auto-wait catches the wrong UI. The easy mistake: not.toBeVisible for never-appeared, pays full timeout silently.

```
// RIGHT USE: asserting disappearance after a known action
await page.getByRole('button', { name: 'Dismiss' }).click();
await expect(page.getByRole('alert')).not.toBeVisible();    // waits up to timeout, expected to disappear

// SLOW USE: asserting never-appeared pays full timeout for nothing
// await page.getByRole('button', { name: 'Save' }).click();
// await expect(page.getByRole('alert')).not.toBeVisible(); // 5s wait, every test, no value

// BETTER: assert the positive expected state
await page.getByRole('button', { name: 'Save' }).click();
await expect(page.getByText('Saved successfully')).toBeVisible();
await expect(page).toHaveURL(/\/saved/);
// Auto-waiting catches the bug if the wrong UI appears; no wasted timeout
```

Q 125 LEVEL · Senior

What's the difference between toEqual, toStrictEqual, toMatchObject, and expect.objectContaining?

toEqual: deep equality, ignores undefined props and prototype, default. toStrictEqual: + checks undefined props + prototype, type identity matters. toMatchObject: **partial**, **actual must contain expected**, **can have more**. expect.objectContaining: partial-match inside other matchers / arg matchers. Asked next: prototype matters for class-vs-plain-object distinctions. Trap: toEqual on API responses with fields you don't care about, brittle.

```
const actual = { id: 1, name: 'Anna', email: 'anna@bank.com', createdAt: '2026-01-15T10:00:00Z' };

// toEqual: full deep equality (ignores undefined props)
expect({ a: 1 }).toEqual({ a: 1, b: undefined });        // passes

// toStrictEqual: + checks undefined props + prototype
expect({ a: 1 }).not.toStrictEqual({ a: 1, b: undefined });   // strict: NOT equal
class User { constructor(public id: number) {} }
expect(new User(1)).not.toStrictEqual({ id: 1 });        // class vs plain object

// toMatchObject, partial: actual contains at least expected
expect(actual).toMatchObject({ id: 1, name: 'Anna' });   // ignores email, createdAt

// objectContaining: partial inside other matchers
expect([actual]).toEqual([
  expect.objectContaining({ name: 'Anna' }),             // ignores other fields
]);
```

Q 126 LEVEL · Mid

toHaveText vs toContainText: when do you use each, and what's the exact-match vs substring trap?

toHaveText = full exact string (trimmed); toContainText = substring. Both retry: the difference is what they assert, not the waiting. Exact when the whole value is the contract; substring for dynamic content. Where this goes wrong: toHaveText on text that carries extra words, fails for the wrong reason. Both accept a regex (anchored partial) and an array (per-item assertion). Follow-up: a phrase inside a paragraph, toContainText or anchored regex.

```
// toHaveText: the WHOLE string is the contract
await expect(page.getByRole('heading')).toHaveText('Order Confirmed');
// fails if the heading reads 'Order Confirmed Today'

// toContainText: a phrase inside dynamic content
await expect(page.getByTestId('greeting')).toContainText('Welcome');
// passes for 'Welcome back, Anna'

// Anchored regex: partial but pattern-controlled
await expect(page.getByTestId('total')).toHaveText(/^Total: ₹[\d,]+$/);

// Array form: assert across a list in one call
await expect(page.getByRole('listitem')).toHaveText(['Apples', 'Bananas', 'Cherries']);
await expect(page.getByRole('listitem')).toContainText(['Apple', 'Banana']);
```

Q 127 LEVEL · Mid

toBeVisible vs toBeHidden vs toBeAttached: what does each actually assert, and why isn't toBeHidden just not(toBeVisible)?

Two axes: presence in DOM (attached) vs rendered to the user (visible). toBeAttached = in the DOM at all; toBeVisible = attached AND actually rendered. toBeHidden = deliberate opposite: not in DOM, or in DOM but not rendered. Not just not.toBeVisible: toBeHidden reads positive and counts detached as a pass. not.toBeVisible is a negation: tends to burn the full timeout. Expect next: modal fully removed, assert not.toBeAttached, not toBeHidden.

```
// Presence only: in the DOM, visible or not
await expect(page.getByTestId('lazy-panel')).toBeAttached();

// Rendered to the user: attached AND actually shown
await expect(page.getByRole('dialog')).toBeVisible();

// Hidden: passes whether detached OR present-but-not-rendered
await page.getByRole('button', { name: 'Close' }).click();
await expect(page.getByRole('dialog')).toBeHidden();

// Distinguish 'fully removed' from 'merely hidden'
await expect(page.getByRole('dialog')).not.toBeAttached();   // node gone from DOM
// vs toBeHidden(), which would pass even if a collapsed node lingered
```

Q 128 LEVEL · Mid to Senior

expect.poll vs expect.toPass vs manual polling, when do you reach for each, and what does expect.toPass give you that expect.poll doesn't?

poll retries a single return value into one matcher, one settling value. toPass retries a whole block of assertions until it all passes or times out. Key distinction: poll = one value/one matcher; toPass = arbitrary code, many assertions. toPass can re-run the producing action each attempt (re-fetch, re-click, re-assert). Manual polling: **the fallback to avoid**, **loses intervals**, **timeout**, **messages**; invites sleeps. They usually ask next: only-toPass case, re-fetch then assert several fields, or re-click then verify.

```
// expect.poll: ONE settling value into one matcher
await expect.poll(async () => {
  const res = await page.request.get('/api/orders/42');
  return (await res.json()).status;
}, { timeout: 30_000, intervals: [1_000, 2_000, 5_000] }).toBe('confirmed');

// expect.toPass: a BLOCK of assertions, with the action re-run each attempt
await expect(async () => {
  const res = await page.request.get('/api/orders/42');   // re-fetched every attempt
  expect(res.ok()).toBeTruthy();
  const body = await res.json();
  expect(body.status).toBe('confirmed');
  expect(body.items).toHaveLength(3);                     // several assertions together
}).toPass({ timeout: 30_000, intervals: [1_000, 2_000, 5_000] });

// Manual polling, the fallback to avoid: hard sleeps, no built-in timeout/messages
// while (true) { ...; await page.waitForTimeout(1000); }   // flaky, don't
```

Q 129 LEVEL · Mid to Senior

What does expect.configure do, and when is a pre-configured expect better than passing options at each call?

expect.configure returns a NEW expect instance with its own defaults. Documented options: timeout and soft. slowExpect for one genuinely slow area, instead of raising the global timeout. softExpect for a block of non-blocking presentation checks. Judgement: a named instance documents the exception; a global bump hides regressions. The next question: per-call options for a one-off, configure when the exception repeats.

```
import { expect } from '@playwright/test';

// One slow area gets a longer budget; the rest of the suite stays tight
const slowExpect = expect.configure({ timeout: 10_000 });
await slowExpect(page.getByTestId('quarterly-report')).toBeVisible();

// A block of presentation checks: collect every failure, don't stop at the first
const softExpect = expect.configure({ soft: true });
await softExpect(page.getByRole('heading')).toHaveText('Orders');
await softExpect(page.getByRole('columnheader').first()).toHaveText('Order ID');
await softExpect(page.getByTestId('footer-note')).toContainText('Updated');

// WRONG: raising the global default to fix one slow page
// export default defineConfig({ expect: { timeout: 10_000 } });
// every assertion in the suite now waits longer, and a real
// performance regression elsewhere passes quietly.
```

QUICK QUIZ

1. What does expect.soft() do differently from a normal assertion?
   A) It retries longer
   B) It disables retries
   C) It collects all failures before throwing instead of stopping at the first
   D) It only works on API responses
2. Which two options make toHaveScreenshot usable on a real, dynamic app?
   A) fullPage and timeout
   B) mask and animations: 'disabled'
   C) threshold and retries
   D) clip and scale
3. What is expect.poll() the correct tool for?
   A) Eventually consistent systems where state propagates over time
   B) Asserting on a static DOM element
   C) Replacing web-first assertions
   D) Mocking network requests
4. Why assert email with expect.stringMatching(regex) rather than a literal address?
   A) Regexes run faster
   B) Literal strings are not supported
   C) It validates the format rule, staying resilient to data changes
   D) It avoids using toMatchObject
5. When is writing a custom matcher (e.g. toBeLoggedIn) worth it?
   A) To wrap a single built-in assertion
   B) To replace expect entirely
   C) Only for API responses
   D) When a multi-step domain check repeats across many tests

Answers: 1:C 2:B 3:A 4:C 5:D

Parallel Execution and Sharding

Q 130 LEVEL · Mid

How does Playwright's parallel execution model work?

Files run in parallel; each worker is an isolated Node process + browser. fullyParallel: true parallelises tests within a file too. Fresh context per test, but shared external state still collides. Rule: every test creates and tears down its own data. Asked next: what breaks, shared DB record/account; fix with owned data. What loses the point: claiming isolation magically prevents all interference.

```
// playwright.config.ts
export default defineConfig({
  fullyParallel: true,
  workers: process.env.CI ? 4 : undefined,
});

// Worker-scoped fixture: expensive setup once per worker
const test = base.extend<{}, { adminContext: BrowserContext }>({
  adminContext: [async ({ browser }, use) => {
    const ctx = await browser.newContext({ storageState: 'auth/admin.json' });
    await use(ctx);
    await ctx.close();
  }, { scope: 'worker' }],
});
```

Q 131 LEVEL · Mid

How does sharding work and how do you calculate the optimal shard count?

Sharding splits the suite across CI machines; merge-reports combines results. Formula: total_suite_time / target_time; target ~10 min per shard. 45-min suite gives ~4 shards; with 4 workers each, under ~8 min. Shards are balanced by NUMBER OF TESTS, not execution time. Each shard gets a contiguous block; some may run fewer tests, or none. Follow-up: one slow shard means slow tests clustered; split large slow spec files. The common miss: over-sharding and ignoring CI infrastructure cost.

```
# Run one shard
npx playwright test --shard=1/4

# GitHub Actions matrix
strategy:
  matrix:
    shardIndex: [1, 2, 3, 4]
    shardTotal: [4]
steps:
  - run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
  - uses: actions/upload-artifact@v4
    with: { name: blob-${{ matrix.shardIndex }}, path: blob-report/ }

# After all shards: merge into one report
# npx playwright merge-reports --reporter html ./all-blob-reports
```

Q 132 LEVEL · Mid

How do you ensure test isolation in a parallel environment?

Isolation needs unique data and no shared DB/file/browser state. Anti-pattern: one hardcoded test record every parallel test fights over. Fix: faker for unique data, API setup and teardown per test. Shared test data is the #1 cause of parallel failures. Expect next: a truly non-parallel test, serial mode, but treat it as a smell. Avoid: a hardcoded shared user reused across tests.

```
import { faker } from '@faker-js/faker';

test('user registration', async ({ request, page }) => {
  const email = faker.internet.email(); // unique per run
  const created = await request.post('/api/users', {
    data: { email, name: faker.person.fullName() },
  });
  const { id } = await created.json();

  await page.goto('/users');
  await expect(page.locator('tbody tr').filter({ hasText: email })).toBeVisible();

  await request.delete(`/api/users/${id}`); // teardown
});
```

Q 133 LEVEL · Mid to Senior

How do you handle flaky tests systematically in Playwright?

Process: retries as a net -> track retry rate -> quarantine -> trace -> fix. trace: 'on-first-retry' gives every retried failure a diagnostic artifact. Tag and quarantine tests over a retry-rate threshold so they stop blocking CI. Conviction: no flaky test is truly random. They usually ask next: **root causes**, **missing waitForURL**, **unwaited spinner**, **shared data**, **sleep**, **strict-mode**. The easy mistake: using retries as the fix instead of a safety net.

```
// playwright.config.ts
export default defineConfig({
  retries: process.env.CI ? 2 : 0,
  use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' },
});

// Quarantine a known flaky test while you investigate
test('notification timing @flaky', async ({ page }) => {
  // tracked in JIRA-1234: race in notification service
});

// Common root causes -> fixes:
//  missing waitForURL  -> await page.waitForURL('/dashboard')
//  unwaited spinner    -> spinner.waitFor({ state: 'hidden' })
//  hard sleep          -> replace with an explicit waitFor
```

Q 134 LEVEL · Mid to Senior

How does serial mode work, and when is it justified?

Serial mode runs a describe block sequentially in one worker; failure skips the rest. Opposite of fullyParallel: a deliberate escape hatch. Treat it as a code smell that requires justification. Usual fix: move UI setup to the API so tests are independent and parallel. The next question: legitimate only for a real state transition you can't jump into, rare. Trap: using serial mode to chain UI steps out of convenience.

```
test.describe('Order lifecycle', () => {
  test.describe.configure({ mode: 'serial' });
  let orderId: string;
  test('create order', async ({ page }) => {
    await page.goto('/orders/new');
    await page.getByRole('button', { name: 'Create Order' }).click();
    orderId = await page.getByTestId('order-id').innerText();
  });
  test('approve order', async ({ page }) => {
    await page.goto(`/orders/${orderId}/approve`);
  });
});

// Better: independent test using API setup, runs in parallel
test('full order lifecycle', async ({ request, page }) => {
  const { id } = await (await request.post('/api/orders', { data: { product: 'MacBook Pro' } })).json();
  await request.patch(`/api/orders/${id}`, { data: { status: 'shipped' } });
  await page.goto(`/orders/${id}`);
  await expect(page.getByTestId('order-status')).toHaveText('Shipped');
});
```

Q 135 LEVEL · Senior

How do you pick the optimal worker count, given browser tests are I/O-bound rather than CPU-bound?

Browser tests are I/O-bound: most time waiting on browser/network/app. Workers > cores typically beats workers = cores (6-8 on a 4-vCPU runner). Ceiling: memory (200-400 MB/worker) before CPU, OOM, not slowdown. Discipline: measure 4 vs 6 vs 8 on the same suite; pick where time plateaus. Asked next: too many workers -> wall-clock plateaus OR flake increases. Where this goes wrong: workers = cores from Node.js intuition, doesn't apply to I/O-bound.

```
// playwright.config.ts: measure before locking in
export default defineConfig({
  workers: process.env.CI
    ? Number(process.env.PW_WORKERS ?? 6)              // empirically tuned for this runner
    : '50%',                                            // local: half the cores
  fullyParallel: true,
});

// Tuning workflow: measure on the actual runner
//   PW_WORKERS=4 npx playwright test  ->  total time T4
//   PW_WORKERS=6 npx playwright test  ->  total time T6
//   PW_WORKERS=8 npx playwright test  ->  total time T8
//   PW_WORKERS=10 npx playwright test ->  total time T10
//
// Pick the lowest count where time plateaus.
// On a 4-vCPU/7GB runner that's often 6-8, not 4.
```

Q 136 LEVEL · Senior

What is a shared-resource bottleneck, and how does it cap parallel test speedup?

Shared infra (DB connections, rate limits, shared accounts) caps parallelism. Diagnostic: runtime plateaus while CPU/RAM not saturated -> resource bottleneck. Fix 1: pool of test accounts indexed via testInfo.parallelIndex (auth). Fix 2: per-test/per-worker tenant isolation (DB row contention). Fix 3: mock the rate-limited external service (throttling). What loses the point: adding workers when the bottleneck is elsewhere, noisier, not faster.

```
// Pool of test accounts: one per worker, no auth contention
export const test = base.extend<{ testAccount: { email: string; password: string } }>({
  testAccount: async ({}, use, testInfo) => {
    const idx = testInfo.parallelIndex;                      // 0, 1, 2, ...
    await use({
      email:    `test-user-${idx}@example.com`,
      password: process.env[`TEST_USER_${idx}_PW`]!,
    });
  },
});

// Per-worker unique data: faker + workerIndex avoids row contention
const email = `test+${faker.string.uuid()}-w${testInfo.workerIndex}@local`;

// Mock the rate-limited external service in internal tests
test.beforeEach(async ({ context }) => {
  await context.route('**/api.stripe.com/**', route =>
    route.fulfill({ status: 200, body: JSON.stringify({ id: 'pi_mock' }) }));
});
```

Q 137 LEVEL · Mid

How do test.describe.serial and test.describe.parallel override parallelism for specific blocks?

describe.serial: tests run in declaration order on one worker. Serial: if one fails, subsequent tests in the block SKIP. describe.parallel: tests run in parallel even when fullyParallel is false. Rule: parallel default; serial only for genuine multi-step dependencies. Follow-up: failure in serial -> subsequent tests skip with clean report. The common miss: describe.serial for shared state, refactor to independent tests instead.

```
// RIGHT USE: multi-step flow with genuine ordering
test.describe.serial('order lifecycle', () => {
  let orderId: string;

  test('create order', async ({ request }) => {
    const r = await request.post('/api/orders', { data: { product: 'X' } });
    orderId = (await r.json()).id;
  });

  test('ship the order', async ({ request }) => {
    await request.patch(`/api/orders/${orderId}`, { data: { status: 'shipped' } });
  });
  // If create-order fails, ship-the-order is skipped automatically
});

// WRONG USE: shared state for convenience (refactor instead)
// test.describe.serial('user tests share login', () => { ... });
// Right fix: a loggedInUser fixture, tests run in parallel

// Force parallel in a sequential-default suite
...
```

Q 138 LEVEL · Senior

What data isolation strategies prevent parallel tests from corrupting each other?

Unique data per test: faker + workerIndex for full uniqueness on natural keys. Worker-scoped tenants: each worker its own tenant ID, org-level isolation. Test-scoped cleanup registry: tracked entities deleted in afterEach. Combination scales: collision-free + isolated + non-accumulating. Expect next: no multi-tenant -> explicit namespacing + critical cleanup. Avoid: shared test data 'for convenience', contention under parallel load.

```
// Layer 1: unique data per test (faker + workerIndex)
test('parallel-safe user creation', async ({ request, cleanup }, testInfo) => {
  const email = `test+${faker.string.uuid()}-w${testInfo.workerIndex}@local`;
  const res = await request.post('/api/users', { data: { email } });
  expect(res.status()).toBe(201);
  cleanup.add((await res.json()).id);
});

// Layer 2: worker-scoped tenant
export const test = base.extend<{}, { workerTenant: string }>({
  workerTenant: [async ({ request }, use) => {
    const r = await request.post('/api/tenants', { data: { name: `worker-${testInfo.workerIndex}` } });
    const id = (await r.json()).id;
    await use(id);
    await request.delete(`/api/tenants/${id}`).catch(() => {});
  }, { scope: 'worker' }],
});

// Layer 3: test-scoped cleanup registry (see S7 Q12)
// cleanup fixture tracks IDs, deletes in afterEach regardless of outcome
```

Q 139 LEVEL · Senior

How do you diagnose when adding parallel workers makes the suite slower, not faster?

Signature: more workers -> same or worse wall-clock time. Check 1: CPU/RAM saturation (compute-bound or memory pressure). Check 2: shared-resource contention (DB, external service, rate limits). Check 3: flake increase (isolation bugs surfaced at higher concurrency). Parallelism is diagnostic about real concurrency, prod will hit the same limits. The easy mistake: lowering workers to hide symptom without finding root cause.

QUICK QUIZ

1. What does fullyParallel: true change about Playwright's execution?
   A) It runs tests on more machines
   B) It forces serial execution
   C) It disables worker isolation
   D) It parallelises tests within a file, not just across files
2. How do you calculate the optimal shard count?
   A) Always use the number of CPU cores
   B) total_suite_time divided by target_time per shard
   C) One shard per test file
   D) Double the worker count
3. What is the most common cause of failures that appear only under parallelism?
   A) Too few workers
   B) A missing reporter
   C) Shared test data that multiple tests fight over
   D) Browser version mismatch
4. In a systematic flakiness process, what role do retries play?
   A) They are the fix for flaky tests
   B) They make tests deterministic
   C) They replace the trace viewer
   D) A safety net while you track, quarantine and root-cause the flakiness
5. When is test.describe.configure({ mode: 'serial' }) genuinely justified?
   A) For a real state transition neither the UI nor the API lets you jump into
   B) Whenever tests are slow
   C) To chain UI steps for convenience
   D) For every describe block

Answers: 1:D 2:B 3:C 4:D 5:A

CI/CD Integration

Q 140 LEVEL · Mid

How do you set up Playwright in GitHub Actions with sharding?

Matrix strategy shards the suite across runners. fail-fast: **false**, **all shards finish**, **full failure picture**. if: always() on uploads, keep traces/reports when tests fail. merge-reports job combines blob reports into one HTML report. Follow-up: why merge, four shard reports are unreadable; one is the deliverable. Trap: leaving fail-fast at default, hiding other shards' failures.

```
jobs:
  test:
    strategy:
      fail-fast: false              # all shards finish
      matrix: { shardIndex: [1,2,3,4], shardTotal: [4] }
    steps:
      - run: npm run typecheck       # cheap checks gate the run
      - run: npm run lint
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
      - if: always()                 # keep diagnostics on failure
        uses: actions/upload-artifact@v4
        with: { name: blob-${{ matrix.shardIndex }}, path: blob-report/ }
  merge-reports:
    needs: test
    if: always()
    steps:
      - run: npx playwright merge-reports --reporter html ./all-blob-reports
```

Q 141 LEVEL · Mid

How do you configure Playwright for Jenkins CI?

Jenkinsfile stages; JUnit XML feeds Jenkins' built-in test view. Use the official Playwright Docker image: deps pre-installed, version-pinned. --ipc=host is the key gotcha: without it Chromium crashes silently. Expect next: why, default /dev/shm too small; --ipc=host shares host IPC. post-always block publishes results and archives the report on failure. Where this goes wrong: installing browsers manually and fighting missing system libs.

```
pipeline {
  agent {
    docker {
      image 'mcr.microsoft.com/playwright:v1.45.0-jammy'
      args '--ipc=host'   // without this, Chromium crashes silently
    }
  }
  environment { CI = 'true'; BASE_URL = credentials('staging-base-url') }
  stages {
    stage('Install')   { steps { sh 'npm ci' } }
    stage('Typecheck') { steps { sh 'npm run typecheck' } }
    stage('Test') {
      steps { sh 'npx playwright test --reporter=junit,html' }
      post { always {
        junit 'results/junit.xml'
        archiveArtifacts artifacts: 'playwright-report/**', fingerprint: true
      } }
    }
  }
}
```

Q 142 LEVEL · Junior to Mid

How do you implement test tagging and selective test execution in CI?

Tag with @-labels; select with --grep, exclude with --grep-invert. One npm script per tag, each mapped to a CI trigger. @smoke ~20 critical tests under 3 min on PRs; @regression nightly. They usually ask next: PR runs smoke for speed; cron runs full regression nightly. Balances fast PR feedback with full nightly coverage. What loses the point: running the whole suite on every PR, slow CI erodes trust.

```
test('@smoke @critical login flow', async ({ page }) => { /* ... */ });
test('@regression user management', async ({ page }) => { /* ... */ });

// package.json
{
  "scripts": {
    "test:smoke":      "playwright test --grep @smoke",
    "test:regression": "playwright test --grep @regression",
    "test:not-slow":   "playwright test --grep-invert @slow"
  }
}

# workflow: smoke on PRs, full regression nightly
on:
  pull_request:       # smoke only
  schedule: [{ cron: '0 2 * * *' }]  # regression at 2am
```

Q 143 LEVEL · Mid

How do you manage test environment secrets in CI?

Inject secrets at runtime from the CI store: never commit or log them. requireEnv throws at startup with a clear message if a var is missing. Turns a cryptic mid-test crash into an actionable config error. Commit .env.example (names, no values) as configuration documentation. The next question: missing var, requireEnv names it at startup, not undefined mid-test. The common miss: hardcoding credentials anywhere, even staging.

```
// src/config/env.ts
function requireEnv(name: string): string {
  const value = process.env[name];
  if (!value) {
    throw new Error(`Required env var ${name} is not set. Check .env or CI secrets.`);
  }
  return value;
}

export const config = {
  baseURL:       requireEnv('BASE_URL'),
  adminEmail:    requireEnv('ADMIN_EMAIL'),
  adminPassword: requireEnv('ADMIN_PASSWORD'),
} as const;

# GitHub Actions injects from repository secrets
# env:
#   ADMIN_PASSWORD: ${{ secrets.ADMIN_PASSWORD }}
```

Q 144 LEVEL · Mid to Senior

How do you implement quality gates in CI using Playwright results?

A custom reporter tallies results and exits non-zero past a threshold. Gates: pass rate > 95%, flaky rate < 5%. Turns test results into a deployment decision, not a passive report. Creates accountability: cannot ship with a degraded suite. Stakeholder framing: the suite is a quality signal, not just a report. Avoid: treating the report as output nobody acts on.

```
import { Reporter, TestCase, TestResult } from '@playwright/test/reporter';

class QualityGateReporter implements Reporter {
  private passed = 0; private failed = 0; private flaky = 0;
  onTestEnd(_t: TestCase, r: TestResult) {
    if (r.status === 'passed') this.passed++;
    else if (r.status === 'failed') this.failed++;
    else if (r.status === 'flaky') this.flaky++;
  }
  onEnd() {
    const total = this.passed + this.failed + this.flaky;
    const passRate = (this.passed / total) * 100;
    const flakyRate = (this.flaky / total) * 100;
    if (passRate < 95) { console.error('GATE FAILED: pass rate < 95%'); process.exit(1); }
    if (flakyRate > 5) { console.error('GATE FAILED: flaky rate > 5%'); process.exit(1); }
  }
}
```

Q 145 LEVEL · Mid to Senior

How do you design a professional tagging and filtering strategy in CI/CD?

Tags are a CI contract: trigger + time budget + quality signal. Four tiers: @smoke (PR, <3 min), @critical (merge to main), @regression (nightly), @slow (excluded from PR). Discipline: enforce the budget, audit and split @smoke when it creeps. Prefer the v1.42+ tag option: queryable, no test-name pollution. Asked next: keep smoke fast by treating the budget as non-negotiable. The easy mistake: letting smoke bloat until PR feedback is slow and distrusted.

```
// v1.42+ tag option (preferred: queryable, clean report names)
test('user can log in', { tag: ['@smoke', '@critical'] }, async ({ page }) => { /* ... */ });
test('report generation', { tag: ['@regression', '@slow'] }, async ({ page }) => { /* ... */ });

// package.json
{
  "scripts": {
    "test:smoke":   "playwright test --grep @smoke",
    "test:release": "playwright test --grep '@smoke|@critical|@regression'"
  }
}

# triggers: smoke on PR, critical on main, release-gate on release/*
# each tier enforces its own time budget
```

Q 146 LEVEL · Senior

How do you balance shards so one shard doesn't dominate the wall-clock time?

Default sharding = contiguous blocks balanced by TEST COUNT, not by time. fullyParallel: true, distribute by test, not file (the simple fix). Duration-aware bin-packing for serious imbalance, ~20 lines of CI script. Wall-clock time = duration of slowest shard: imbalance dominates pipeline. Follow-up: measure imbalance as ratio slowest/fastest; >2x = balance pays. Trap: adding shards without balancing them, just more idle workers.

```
// 1. Test-level distribution: the cheap fix
// playwright.config.ts
export default defineConfig({
  fullyParallel: true,
  workers: process.env.CI ? 4 : undefined,
});

// 2. Duration-aware shard assignment: the precise fix
// scripts/balanced-shards.ts (run in CI before tests)
//   1. Read previous run's report.json
//   2. For each test, compute average duration over N recent runs
//   3. Sort tests descending by duration
//   4. For each test, assign to the shard with the lowest current total
//   5. Emit a per-shard test-list file consumed via --grep-invert / --shard-config
//
// Result: shard durations within ~10% of each other instead of 4 vs 18 minutes
```

Q 147 LEVEL · Mid

How do you cache Playwright browser binaries and node_modules in CI without breaking determinism?

node_modules: cache key on hash of package-lock.json. Browser binaries: cache key on Playwright version. Include OS in the cache key: Ubuntu 22 binaries fail on Ubuntu 24. Use restore-keys conservatively: loose matches produce stale binaries. Expect next: debug cache failures by invalidating the key for one run. Where this goes wrong: loose restore-keys, wrong cache hit = inscrutable failures.

```
# .github/workflows/test.yml: two-layer caching
- name: Cache node_modules
  uses: actions/cache@v4
  with:
    path: node_modules
    key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
    # NO restore-keys: exact match or rebuild

- run: npm ci

- name: Get Playwright version
  id: pw
  run: |
    echo "version=$(node -p \"require('@playwright/test/package.json').version\")" >> "$GITHUB_OUTPUT"

- name: Cache Playwright browser binaries
  uses: actions/cache@v4
  with:
    path: ~/.cache/ms-playwright
    key: ${{ runner.os }}-playwright-${{ steps.pw.outputs.version }}
...
```

Q 148 LEVEL · Mid to Senior

When do you use the official Playwright Docker image versus a custom one, and how do you keep image versions deterministic?

Official image: Node + Playwright + browsers preinstalled, fast CI start. Custom image extends official when extra system deps or fixtures needed. ALWAYS pin to specific tag matching @playwright/test version. :latest causes silent dev-vs-CI drift on Microsoft's image rebuild. They usually ask next: custom image extending official handles dev+CI extras consistently. What loses the point: :latest in production CI, works until Microsoft pushes a new build.

```
# Custom Dockerfile extending the official image, pinned version
FROM mcr.microsoft.com/playwright:v1.47.0-jammy

# Additional system deps the official image doesn't have
RUN apt-get update && apt-get install -y --no-install-recommends \\
      postgresql-client imagemagick \\
    && rm -rf /var/lib/apt/lists/*

# Company certificate for internal HTTPS endpoints
COPY certs/company-ca.crt /usr/local/share/ca-certificates/
RUN update-ca-certificates

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npm", "test"]

# WRONG: silent dev/CI drift
# FROM mcr.microsoft.com/playwright:latest
```

Q 149 LEVEL · Mid to Senior

How do you decide which browsers to run in CI and when each pays its way?

Tiered: Chromium on PR; +Firefox/WebKit on main; all on release. Chromium catches ~90% of issues at lowest cost, PR-friendly default. Measure browser-specific failure data over 6 months, empirical decision. Cross-browser coverage is infrastructure: decide by data, not defaults. The next question: justify dropping via failure-rate data; document the policy. The common miss: all three on every PR, wastes time/money on unused coverage.

```
# .github/workflows/test.yml: tiered browser matrix
jobs:
  test:
    strategy:
      matrix:
        # PR builds: Chromium only, fast feedback
        # Push to main: + Firefox + WebKit
        # Release: all three explicitly
        browser: ${{ github.event_name == 'pull_request'
                      && fromJSON('["chromium"]')
                      || fromJSON('["chromium","firefox","webkit"]') }}
    steps:
      - run: npx playwright test --project=${{ matrix.browser }}

# Decision data: measure before defaulting
#   grep "webkit" reports/*.json | grep "failed"     -> frequency
#   if < 1/quarter:       move WebKit to main-only, document the policy
#   if weekly or more:    PR-level WebKit pays its way
```

Q 150 LEVEL · Senior

What is a merge queue, and how does it combine with retry-on-flake to ship safely?

Merge queue serialises merges; tests run against actual main tip. Catches merge-race bugs: A+B pass individually, fail together. Retry-on-flake (retries: 2) absorbed by queueing, no dev wait penalty. Track retry-needed tests in a dashboard: the flake-fix backlog. Asked next: dashboard turns invisible flake debt into prioritised work. Avoid: uncapped retries with no tracking, silent degradation.

```
# .github/workflows/test.yml: retry-on-flake + merge queue
on:
  pull_request:
  merge_group:                 # GitHub native merge queue

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: npx playwright test --retries=2 --reporter=json,html
      # JSON output drives the retry-needed dashboard

# Dashboard query (parse JSON reports)
#   for each test where result.retries > 0:
#     count occurrences over the last 30 days
#     sort descending -> prioritised flake-fix backlog
#     owner = team-from-CODEOWNERS for that test file
```

Q 151 LEVEL · Senior

In a monorepo, how do you run only the tests affected by a code change?

Path-based: directory-convention mapping; git diff finds affected tests. Build-graph (Nx/Turborepo/Bazel): follows real imports, more precise. ALWAYS run full suite on main-branch merge as a backstop. Path-based misses indirect deps; graph-based can miss dynamic imports. Follow-up: graph more reliable; path easier without new build tool. The easy mistake: selective testing without the backstop, indirect-dep bugs ship.

```
# Path-based filtering on PRs (no new build tool required)
- name: Compute affected tests
  id: affected
  run: |
    BASE=$(git merge-base origin/main HEAD)
    CHANGED=$(git diff --name-only "$BASE" HEAD)
    # Map source paths to test paths via convention
    echo "$CHANGED" | sed -E 's#^apps/([^/]+)/.*#tests/\1/#; s#^libs/([^/]+)/.*#tests/**/*.spec.ts#' \\
      | sort -u > affected.txt

- run: npx playwright test $(cat affected.txt)

# Build-graph alternative (more precise)
- run: npx nx affected --target=test --base=origin/main

# BACKSTOP: full suite on main-branch merges, catches what selective misses
# trigger: on: push: branches: [main]
```

Q 152 LEVEL · Senior

How do you tier CI feedback (smoke vs regression vs full e2e) to balance speed with coverage?

Three tiers with budgets: smoke <2min, regression <15min, full nightly. Smoke (10-20 tests): critical paths, auth/home/checkout reach payment. Regression: @critical + @regression on PR-ready and main merges. Full e2e: @slow + cross-browser + mobile, nightly schedule. Expect next: smoke is 10-20 tests max, over 2 min and it isn't smoke any more. Trap: one tier (full suite on every PR), too slow for iteration, too narrow for safety.

```
# Tier 1: smoke (runs on every commit, every push, every draft PR)
name: smoke
on: [push, pull_request]
jobs:
  smoke:
    timeout-minutes: 3                    # hard ceiling, not 2 (small buffer)
    steps:
      - run: npx playwright test --grep @smoke --reporter=line

# Tier 2: regression (PR ready or main merge)
name: regression
on:
  pull_request:
    types: [ready_for_review, synchronize]
  push:
    branches: [main]
jobs:
  regression:
    timeout-minutes: 18                   # 15 + small buffer
    steps:
...
```

Q 153 LEVEL · Mid to Senior

How do you manage trace, video, and screenshot artifacts in CI without exploding storage costs?

Generate conditionally: trace on-first-retry, video retain-on-failure. Retain selectively: PR 7d, main 30d, release indefinite (promoted to long-term). Upload conditionally: if-failure pattern, no upload on green builds. Quantify from YOUR measured trace size x tests x runs per day, not a quoted figure. They usually ask next: 7d for PRs; promote release artifacts to long-term archive. Where this goes wrong: trace: 'on' + indefinite retention, cost grows with PR volume forever.

```
// playwright.config.ts: conditional generation
export default defineConfig({
  use: {
    trace:     'on-first-retry',
    video:     'retain-on-failure',
    screenshot: 'only-on-failure',
  },
});

# .github/workflows/test.yml: conditional upload + tiered retention
- name: Upload trace/video/screenshots (failures only)
  if: failure()
  uses: actions/upload-artifact@v4
  with:
    name: playwright-failure-${{ github.run_id }}
    path: |
      test-results/**
      playwright-report/**
    retention-days: ${{ github.ref == 'refs/heads/main' && 30 || 7 }}

...
```

QUICK QUIZ

1. Why set fail-fast: false on a GitHub Actions sharding matrix?
   A) So all shards finish and you get the full failure picture, not just the first
   B) To make the job run faster
   C) To disable retries
   D) To reduce CI cost
2. What is the key Jenkins-plus-Docker gotcha for Playwright?
   A) Setting node-version
   B) Caching the browsers
   C) Using JUnit output
   D) --ipc=host, without which Chromium crashes silently on a too-small /dev/shm
3. What is the purpose of the requireEnv helper for secrets?
   A) It encrypts the secrets
   B) It fails fast with a clear message if a required variable is missing
   C) It commits secrets to the repo
   D) It logs all variables
4. Which two thresholds make sensible Playwright quality gates?
   A) Pass rate > 50%, flaky rate > 20%
   B) Any failures block deployment
   C) Pass rate > 95% and flaky rate < 5%
   D) Pass rate > 80% only
5. Why prefer the v1.42+ tag option over inline @tags in the test title?
   A) It runs faster
   B) It supports more tags
   C) It is the only way to tag tests
   D) It is queryable programmatically and does not pollute the test name in reports

Answers: 1:A 2:D 3:B 4:C 5:D

Debugging and Tracing

Q 154 LEVEL · Junior to Mid

How do you use the Playwright Inspector for local debugging?

Inspector: step-through GUI with a live selector picker. --debug pauses before the first action; --slow-mo to watch in real time. page.pause() at the failing line freezes the page for inspection. codegen scaffolds locators by clicking through a flow. The next question: upgrade codegen's CSS selectors to semantic getByRole. What loses the point: implying you debug by sprinkling console.log.

```
# Pause before the first action
npx playwright test login.spec.ts --debug

# Watch a run in real time
npx playwright test --headed --slow-mo=500

# Pause at an exact point
test('debug this', async ({ page }) => {
  await page.goto('/login');
  await page.pause();             // Inspector opens, browser frozen here
  await page.getByLabel('Email').fill('admin@test.com');
});

# Generate locators interactively, then upgrade to getByRole
npx playwright codegen https://example.com
```

Q 155 LEVEL · Mid

How does the Playwright Trace Viewer work, and how do you use it for CI debugging?

Records screenshots, DOM snapshots, network, console, source location. Diagnose CI failures without reproducing locally. trace: 'on-first-retry', full trace when it fails, minimal storage. View via show-trace or trace.playwright.dev (no install). Asked next: **on-first-retry over on**, **same diagnostics**, **far less storage**. Quantify: ~60 min reproduce-and-log vs ~5 min scrubbing a trace.

```
// playwright.config.ts
export default defineConfig({
  use: { trace: 'on-first-retry' },   // not 'on', storage cost
});

// View locally, or upload the zip to trace.playwright.dev
// npx playwright show-trace trace.zip

// Attach data to the trace for correlation
test('order flow', async ({ page }, testInfo) => {
  const res = await page.request.get('/api/cart');
  await testInfo.attach('cart-api-response', {
    body: await res.text(), contentType: 'application/json',
  });
});
```

Q 156 LEVEL · Mid

How do you capture and analyse console logs and network failures in tests?

Listeners: console (messages), requestfailed (network), pageerror (exceptions). Assert no console errors: catches silent JS failures functional tests miss. Collect error messages into an array; assert empty with messages in the failure. Fixture attaches network failures to the report for instant diagnosis. Follow-up: surface the failing request via a requestfailed fixture. The common miss: only checking the visible UI and ignoring the console.

```
// Capture failures in real time
page.on('requestfailed', req =>
  console.error(`Network fail: ${req.method()} ${req.url()}, ${req.failure()?.errorText}`));
page.on('pageerror', err => console.error('Uncaught:', err.message));

// Assert no console errors during a flow
test('no console errors on dashboard', async ({ page }) => {
  const errors: string[] = [];
  page.on('console', m => { if (m.type() === 'error') errors.push(m.text()); });
  await page.goto('/dashboard');
  expect(errors, `Console errors: ${errors.join(', ')}`).toHaveLength(0);
});
```

Q 157 LEVEL · Junior to Mid

What is test.step() and how does it improve tracing and reporting?

Groups actions under named, collapsible steps in report and trace. 20 raw actions become a few readable logical steps. See which step failed without reading every action line. Wrap multi-action page-object methods in steps so the trace narrates itself. Expect next: it does not change execution, purely organisational. Avoid: dismissing test.step as cosmetic.

```
test('checkout flow', async ({ cartPage, checkoutPage }) => {
  await test.step('Add items to cart', async () => {
    await cartPage.navigate();
    await cartPage.addItem('MacBook Pro');
  });
  await test.step('Complete checkout', async () => {
    await checkoutPage.fillShipping({ name: 'Alice', address: '123 Main St' });
    await checkoutPage.placeOrder();
  });
  await test.step('Verify confirmation', async () => {
    await expect(checkoutPage.confirmationHeading).toBeVisible();
  });
});

// In a page object: the step name appears in the trace
async completeCheckout(d: CheckoutDetails): Promise<void> {
  await test.step(`Complete checkout for ${d.name}`, async () => {
    await this.fillShipping(d); await this.placeOrder();
  });
}
```

Q 158 LEVEL · Junior to Mid

How do you debug Playwright tests in VS Code?

VS Code extension: **test explorer**, **inline running**, **breakpoint debugging**. Pauses at the breakpoint with the browser open, code + state together. PWDEBUG=1 opens the Inspector; PWDEBUG=console for verbose logging. DEBUG=pw:api logs every API call with timing. They usually ask next: diagnose retries/slowness with DEBUG=pw:api. The easy mistake: describing only console.log debugging.

```
// .vscode/launch.json
{
  "configurations": [{
    "name": "Debug Playwright Test",
    "type": "node",
    "request": "launch",
    "program": "${workspaceFolder}/node_modules/.bin/playwright",
    "args": ["test", "--debug", "${file}"],
    "env": { "PWDEBUG": "1" },
    "console": "integratedTerminal"
  }]
}

# PWDEBUG=1: opens the Inspector
# PWDEBUG=console: verbose logging
# DEBUG=pw:api, logs every API call with timing
```

Q 159 LEVEL · Mid

How do you use page.evaluate() for advanced debugging?

Runs JS in the browser context and returns the result to Node. Escape hatch: inspect app state, or set up state the UI cannot. Real use: read React fibre/component props to find a state bug. Firm rule: setup and debugging only, never assertions. The next question: **why not assert**, **it is one-shot**, **no retry**; use expect instead. Trap: asserting on an evaluate result and reintroducing flakiness.

```
// Inspect application state at the point of failure
const appState = await page.evaluate(() => ({
  cart: JSON.parse(localStorage.getItem('cart') || '{}'),
  flags: (window as any).__FEATURE_FLAGS__,
}));
console.log('State at failure:', JSON.stringify(appState, null, 2));

// Set up state the UI cannot
await page.evaluate((token: string) => {
  localStorage.setItem('auth_token', token);
}, 'test-token-12345');

// Rule: setup/debugging only, assertions use the retrying expect API
```

Q 160 LEVEL · Mid

What is Playwright UI Mode, and when do you use it?

--ui: **watch mode**, **inline traces**, **live preview**, **time-travel scrubber**. Default workflow for writing and debugging tests; auto-reruns on save. Tool map: --debug (breakpoints), --ui (dev), --headed (visual check). Trace Viewer for CI post-mortems; page.pause() for mid-test inspection. Asked next: --ui for active development, --debug for targeted breakpoints. Where this goes wrong: treating every debug tool as interchangeable.

```
# Watch mode with inline traces: default dev workflow
npx playwright test --ui
npx playwright test login.spec.ts --ui

# Choosing the right tool for the moment:
#   --debug      -> step-through Inspector for targeted debugging
#   --ui         -> watch mode with inline traces and live preview
#   --headed     -> quick visual check in a normal window
#   trace viewer -> CI post-mortem and failure analysis
#   page.pause() -> targeted mid-test inspection
```

Q 161 LEVEL · Mid

How do PWDEBUG and DEBUG=pw:api change Playwright's runtime behaviour for diagnosis?

PWDEBUG=1: headed + Inspector + no timeouts + auto-pause, live debug. PWDEBUG=console: lighter, exposes playwright object in DevTools console. DEBUG=pw:api: logs every API call with timing, understand failed runs. DEBUG=pw:protocol: raw CDP/WebSocket, only for framework/browser-level bugs. Discipline: match the variable to the question (api-level usually answers it). What loses the point: reaching for protocol-level when api-level would have answered.

```
# Live debugging on my machine: full inspector, no timeouts
PWDEBUG=1 npx playwright test tests/checkout.spec.ts

# Headed run with playwright object in DevTools console
PWDEBUG=console npx playwright test --headed
#   In DevTools: > playwright.locator('button[name="Save"]').highlight()

# Investigate a CI failure: see what auto-waiting actually did
DEBUG=pw:api npx playwright test tests/failing.spec.ts 2>&1 | tee debug.log
#   Log shows: 'click' -> waiting for 'visible' -> waiting for 'stable' -> ...

# Last resort: raw protocol traffic for framework/browser-level issues
DEBUG=pw:protocol npx playwright test tests/strange.spec.ts
```

Q 162 LEVEL · Mid to Senior

How do you use the trace viewer's time-travel features beyond opening a trace zip?

Action timeline + DOM scrubbing: before/after at every action. Network tab: requests/responses with full headers/body, status filter. Console tab: browser console messages including background async errors. Source tab: call-site source, critical for diagnosing helper/POM failures. Workflow: failing action -> DOM scrub -> network + console for cause. The common miss: opening trace as a screenshot gallery, not using DOM/network/console.

Q 163 LEVEL · Mid

How do you use page.pause() with Chrome DevTools for mid-test inspection?

page.pause() stops the test; browser stays live for inspection. Full Chrome DevTools available: **DOM**, **network**, **console**, **application**, **source**. Closest thing to debugging in production: interact, not just observe. Discipline: at the line you want to inspect; removed before merging. Follow-up: Inspector's Record button captures interactive steps as code. Avoid: leaving page.pause in committed code, CI hangs forever.

```
test('debug a flaky checkout step', async ({ page }) => {
  await page.goto('/checkout');
  await page.getByLabel('Card number').fill('4111 1111 1111 1111');
  await page.getByRole('button', { name: 'Place order' }).click();

  await page.pause();   // <- test pauses here, browser stays live
  // In the Inspector window:
  //   - Step over each command
  //   - Click Record to capture new actions as Playwright code
  //
  // In Chrome DevTools attached to the browser:
  //   - Elements: inspect why the success toast isn't appearing
  //   - Network: did the /api/orders call return 200?
  //   - Console: any uncaught errors? Try page.evaluate equivalents
  //   - Application > Local Storage: is the auth token still set?

  await expect(page.getByText('Order placed')).toBeVisible();
});

// .eslintrc.json: prevent page.pause from being merged
...
```

Q 164 LEVEL · Junior to Mid

What is codegen, how do you run it, and how do you treat its output as a starting point rather than final tests?

codegen records clicks/typing into locators and actions, a scaffold. Run: npx playwright codegen <url>; --target, --save-storage, Record button. Output is a draft: **CSS selectors**, **no real assertions**, **no fixtures/POM**. Upgrade pass: CSS -> getByRole/getByLabel for resilience. Add expect assertions, lift setup into fixtures, fold into the page object. The easy mistake: **committing codegen output as-is**, **brittle**, **assertion-free tests**.

```
# Record a flow: opens a browser plus the Inspector
npx playwright codegen https://example.com

# Record already logged in, and save the session for reuse
npx playwright codegen --save-storage=auth.json https://example.com
npx playwright codegen --load-storage=auth.json https://example.com/account

// What codegen generates (a draft, note the CSS, no assertions):
await page.click('#nav > ul > li:nth-child(2) > a');
await page.fill('input[name="q"]', 'playwright');

// After the upgrade pass, semantic locators + a real assertion:
await page.getByRole('link', { name: 'Products' }).click();
await page.getByRole('searchbox').fill('playwright');
await expect(page.getByRole('heading', { name: /results/i })).toBeVisible();
```

QUICK QUIZ

1. What does page.pause() do during local debugging?
   A) Adds a fixed sleep
   B) Captures a screenshot
   C) Disables auto-waiting
   D) Freezes the test at that line and opens the Inspector to inspect page state
2. Why configure trace: 'on-first-retry' rather than trace: 'on'?
   A) It captures more data
   B) It is required for CI
   C) It gives a full trace exactly when a test fails and retries, without wasting storage on passing runs
   D) It disables screenshots
3. Why add a test that asserts no console errors occurred?
   A) JS errors are often invisible to functional tests, the page looks fine but throws in the background
   B) Console errors always fail tests automatically
   C) It speeds up the suite
   D) It replaces network mocking
4. What does test.step() change about a test?
   A) It changes execution order
   B) Nothing functional: it groups actions into named, collapsible steps in the report and trace
   C) It adds retries
   D) It runs steps in parallel
5. What is the firm rule for using page.evaluate()?
   A) Use it for setup and debugging, never for assertions, it is a one-shot snapshot
   B) Use it for all assertions
   C) Never use it
   D) Use it only in CI

Answers: 1:D 2:C 3:A 4:B 5:A

Reporting

Q 165 LEVEL · Junior to Mid

How do you configure Playwright's built-in HTML reporter?

HTML report: **results**, **screenshots**, **traces**, **video for review**. Run multiple reporters: html (humans), junit (CI), json (tooling), github (PR). github reporter annotates the PR inline: no download needed. open: 'never' in CI or Playwright tries to open a browser and hangs. Expect next: map reporter to consumer, html/junit/github. Trap: leaving the reporter trying to open a browser in CI.

```
// playwright.config.ts
export default defineConfig({
  reporter: [
    ['html',  { outputFolder: 'playwright-report', open: 'never' }], // never in CI
    ['junit', { outputFile: 'results/junit.xml' }],                 // CI parsing
    ['json',  { outputFile: 'results/results.json' }],              // custom tooling
    ['github'],                                                     // PR annotations
    ['dot'],                                                        // minimal console
  ],
});

// npx playwright show-report
```

Q 166 LEVEL · Mid

How do you integrate Allure reporting with Playwright?

allure-playwright adds categorisation, trends, environment info, metadata. Killer feature: history-trend chart, pass rate over time across builds. Correlate a test going flaky with the deployment that caused it. allure.step + labels make reports readable for non-technical stakeholders. They usually ask next: worth it for history trends or stakeholder-readable reports. Where this goes wrong: adding Allure with no reason beyond 'it looks nicer'.

```
# npm install --save-dev allure-playwright allure-commandline

// playwright.config.ts
reporter: [['allure-playwright', { detail: true, outputFolder: 'allure-results' }]]

import { allure } from 'allure-playwright';
test('checkout flow', async ({ page }) => {
  allure.label('feature', 'Checkout');
  allure.label('severity', 'critical');
  allure.link('https://jira.example.com/PROJ-123', 'JIRA Ticket');
  await allure.step('Navigate to checkout', async () => { await page.goto('/checkout'); });
  await allure.step('Place order', async () => {
    await page.getByRole('button', { name: 'Place Order' }).click();
  });
});
# npx allure generate allure-results --clean -o allure-report
```

Q 167 LEVEL · Mid

How do you create a custom reporter for Slack or Teams notifications?

Implement the Reporter interface; post to a webhook in onEnd. Payload: pass rate, first few failures by name, link to the report. Pushes failures to where the team already looks, no manual CI polling. Guard: if (!process.env.SLACK_WEBHOOK_URL) return, no-op locally. The next question: the env-var guard stops it spamming during development. What loses the point: a reporter that fires on local runs too.

```
import { Reporter, TestCase, TestResult, FullResult } from '@playwright/test/reporter';

class SlackReporter implements Reporter {
  private failures: string[] = [];
  private passed = 0;
  onTestEnd(test: TestCase, result: TestResult) {
    if (result.status === 'passed') this.passed++;
    else if (result.status === 'failed') this.failures.push(test.title);
  }
  async onEnd(_result: FullResult) {
    if (!process.env.SLACK_WEBHOOK_URL) return; // no-op locally
    const total = this.passed + this.failures.length;
    const passRate = ((this.passed / total) * 100).toFixed(1);
    const text = `Pass rate ${passRate}%, failures: ${this.failures.slice(0, 5).join(', ') || 'none'}`;
    await fetch(process.env.SLACK_WEBHOOK_URL, {
      method: 'POST', body: JSON.stringify({ text }),
    });
  }
}
export default SlackReporter;
```

Q 168 LEVEL · Mid

How do you run multiple reporters at once, and what's the right combination for production?

reporter accepts an array: multiple reporters run against the same events. Production stack: list + html + json + junit + custom. Each reporter serves a different audience: **dev**, **engineer**, **tool**, **stakeholder**. Cost is minimal: reporters are event subscribers, not extra test passes. Asked next: **stacking is essentially free**, **one test run**, **many outputs**. The common miss: one-reporter-fits-all, output that serves nobody well.

```
// playwright.config.ts: layered reporters for different audiences
export default defineConfig({
  reporter: [
    ['list'],                                                  // dev terminal
    ['html', { outputFolder: 'playwright-report', open: 'never' }],   // engineer investigation
    ['json', { outputFile: 'reports/results.json' }],           // dashboards/analysis
    ['junit', { outputFile: 'reports/junit.xml' }],             // CI tool integration
    ['./reporters/slack-on-failure.ts'],                        // stakeholder notification
    ['./reporters/jira-traceability.ts'],                       // requirements coverage
  ],
});

// Stack is essentially free: all reporters subscribe to the same events
```

Q 169 LEVEL · Mid

How do you operate the HTML report in production: attachments, hosting, and trace embedding?

Attachments via testInfo.attach(): inline for images, downloadable otherwise. Static hosting: **S3**, **Netlify**, **GitHub Pages**, **internal Nginx**. Trace links embedded per failure: one click from report to trace timeline. Upload + link from PR comment: minimise friction to investigation. Follow-up: **attach API bodies**, **CSVs**, **log dumps**, **custom diagnostic data**. Avoid: HTML report kept local-only, only the test author looks at it.

```
// Attach custom diagnostic data: visible in HTML report
test('order flow', async ({ request }, testInfo) => {
  const res = await request.post('/api/orders', { data: { product: 'X' } });
  const body = await res.json();
  await testInfo.attach('order-response.json', {
    body: JSON.stringify(body, null, 2),
    contentType: 'application/json',
  });
  expect(res.status()).toBe(201);
});

# .github/workflows/test.yml: upload + link from PR comment
- if: always()
  uses: actions/upload-artifact@v4
  with: { name: html-report, path: playwright-report/, retention-days: 7 }
- if: always()
  uses: peter-evans/create-or-update-comment@v4
  with:
    issue-number: ${{ github.event.pull_request.number }}
    body: |
...
```

Q 170 LEVEL · Mid to Senior

How do test annotations turn tests into living documentation?

testInfo.annotations: metadata attached to tests, flows through all reporters. Use 1: **requirements traceability**, **REQ-IDs**, **coverage dashboard**. Use 2: classification, severity/category/owner for filtering and routing. Use 3: **non-fatal failure context**, **console warnings**, **slow trends**. Discipline: structured types via helper function, queryable data. The easy mistake: free-form annotation strings, unqueryable rubbish dump.

```
// Helper enforces vocabulary: keeps annotations queryable
type Annotation =
  | { type: 'requirement'; description: string }       // REQ-1234
  | { type: 'owner';       description: string }       // 'auth-team'
  | { type: 'severity';    description: 'P0' | 'P1' | 'P2' }
  | { type: 'jira';        description: string };      // PROJ-456

function annotate(testInfo: TestInfo, a: Annotation): void {
  testInfo.annotations.push(a);
}

test('admin can delete user', async ({ page }, testInfo) => {
  annotate(testInfo, { type: 'requirement', description: 'REQ-1234' });
  annotate(testInfo, { type: 'owner',       description: 'auth-team' });
  annotate(testInfo, { type: 'severity',    description: 'P0' });
  annotate(testInfo, { type: 'jira',        description: 'PROJ-789' });
  // ... test body
});

// Custom reporter aggregates: REQ coverage report, owner dashboards, P0 alerts
```

Q 171 LEVEL · Senior

How do you build a PR-comment reporter that posts results back to the originating pull request?

Custom reporter on onEnd posts to GitHub API. Idempotency: **marker string in body**, **find-or-create**, **update not append**. Failure detail: first 3-5 failures with trace link, not the full list. Cross-link to HTML report + trace artifact: one click to investigation. Principle: the comment is entry point, not summary, details live in the link. Trap: verbose comment with full stack traces, noise everyone scrolls past.

```
// reporters/pr-comment.ts
import type { Reporter, TestCase, TestResult } from '@playwright/test/reporter';
import { Octokit } from '@octokit/rest';

const MARKER = '<!-- playwright-results -->';

export default class PrCommentReporter implements Reporter {
  private failed: { test: TestCase; result: TestResult }[] = [];

  onTestEnd(test: TestCase, result: TestResult): void {
    if (result.status === 'failed' || result.status === 'timedOut') {
      this.failed.push({ test, result });
    }
  }

  async onEnd(): Promise<void> {
    const prNumber = Number(process.env.GITHUB_PR_NUMBER);
    if (!prNumber) return;
    const summary = this.failed.slice(0, 5).map((f, i) =>
      `${i + 1}. **${f.test.title}**: ${f.result.error?.message?.split('\n')[0]?.slice(0, 200)}`
...
```

Q 172 LEVEL · Senior

How do you export test metadata to an observability platform for long-term trend analysis?

Trend questions need observability: **slowing tests**, **flake hotspots**, **deploy impact**. Custom reporter emits per-test structured events to Datadog/Honeycomb/etc. Fields: **test_name**, **duration_ms**, **status**, **retries**, **browser**, **annotation tags**. Queries: p95 duration, flake rate per owner, deploy impact on failure rate. Flake dashboards from S10 are queries against this metadata, not manual tallies. Where this goes wrong: free-form log lines, data exists but is unqueryable, insights never extracted.

```
// reporters/observability.ts: emit structured events per test
import type { Reporter, TestCase, TestResult } from '@playwright/test/reporter';

export default class ObservabilityReporter implements Reporter {
  onTestEnd(test: TestCase, result: TestResult): void {
    const owner    = test.annotations.find(a => a.type === 'owner')?.description ?? 'unknown';
    const severity = test.annotations.find(a => a.type === 'severity')?.description ?? 'P3';
    const event = {
      timestamp:    new Date().toISOString(),
      test_name:    test.titlePath().join(' > '),
      project:      test.parent.project()?.name,
      duration_ms:  result.duration,
      status:       result.status,
      retries:      result.retry,
      owner,                              // queryable tag
      severity,                           // queryable tag
      ci_commit:    process.env.GITHUB_SHA,
    };
    // ship to Datadog/Honeycomb/Prometheus/etc.: or jsonl file ingested by a cron
    fetch('https://obs.internal/v1/events', { method: 'POST', body: JSON.stringify(event) });
...
```

QUICK QUIZ

1. Why set open: 'never' on the HTML reporter in CI?
   A) To hide the report
   B) To speed up tests
   C) Otherwise Playwright tries to launch a browser to show the report and hangs the pipeline
   D) To disable screenshots
2. What is the value of running multiple reporters at once?
   A) Each serves a different audience: html for humans, junit for CI, github for PR annotations
   B) It runs tests faster
   C) It reduces flakiness
   D) Only one reporter is allowed
3. What is Allure's standout capability over the built-in HTML report?
   A) Faster generation
   B) The history-trend chart: pass rate over time, correlating flakiness with deployments
   C) It needs no setup
   D) It blocks deployment
4. What payload makes a Slack test-failure notification useful?
   A) Just 'tests failed'
   B) Only the total test count
   C) The entire HTML report inline
   D) Pass rate, the first few failures by name, and a link to the full report
5. Why guard a Slack reporter with if (!process.env.SLACK_WEBHOOK_URL) return?
   A) So it is a no-op locally and only fires in CI: local runs do not spam the channel
   B) To encrypt the webhook
   C) To speed up the reporter
   D) To require the webhook everywhere

Answers: 1:C 2:A 3:B 4:D 5:A

Configuration and Fixtures

Q 173 LEVEL · Mid

How do fixtures work internally in Playwright?

Fixtures are DI: setup, yield with use(), then teardown. Playwright resolves the dependency graph automatically. Teardown runs in reverse order: page closed before user deleted. yield gives try/finally semantics: teardown runs even on throw. Expect next: **teardown on failure**, **runs regardless**, **reverse order**. What loses the point: calling a fixture just 'setup' and missing the teardown guarantee.

```
export const test = base.extend<{ testUser: User; authedPage: Page }>({
  testUser: async ({ request }, use) => {
    const res = await request.post('/api/users', { data: { email: `t-${Date.now()}@x.com` } });
    const user = await res.json();
    await use(user);                        // test runs here
    await request.delete(`/api/users/${user.id}`); // teardown, always runs
  },
  authedPage: async ({ page, testUser }, use) => { // depends on testUser
    await page.goto('/login');
    await page.getByLabel('Email').fill(testUser.email);
    await page.getByRole('button', { name: 'Sign in' }).click();
    await page.waitForURL('/dashboard');
    await use(page); // closed before testUser is deleted (reverse order)
  },
});
```

Q 174 LEVEL · Mid

What are worker-scoped fixtures and when do you use them?

- Worker-scoped: created once per worker, shared across its tests.
- Default is function scope (fresh per test).
- Quantify: 50 setups vs 1, ~98s saved per worker for a 2s setup.
- Pairing: worker-scoped context, function-scoped page.
- They usually ask next: don't share the page, it leaks state and breaks isolation.
- The common miss: making the Page worker-scoped.

```
// Worker-scoped: expensive context, once per worker
export const test = base.extend<{}, { adminContext: BrowserContext }>({
  adminContext: [async ({ browser }, use) => {
    const ctx = await browser.newContext({ storageState: 'auth/admin.json' });
    await use(ctx);
    await ctx.close(); // when the worker finishes
  }, { scope: 'worker' }],
});

// Function-scoped page off the shared context, fresh per test
export const test2 = test.extend<{ adminPage: Page }>({
  adminPage: async ({ adminContext }, use) => {
    const page = await adminContext.newPage();
    await use(page);
    await page.close();
  },
});
```

Q 175 LEVEL · Mid to Senior

How do you compose multiple fixture files in a large framework?

Each file extends the previous test object; index re-exports the final one. Tests import from one index and get every fixture. Each domain team owns its fixture file; compose at the index. Adding a fixture is a one-file change touching no test files. The next question: composition is additive, tests declare only what they use. Avoid: a single monolithic fixture file that becomes a conflict magnet.

```
// page.fixtures.ts
export const pageTest = base.extend<{ loginPage: LoginPage }>({
  loginPage: ({ page }, use) => use(new LoginPage(page)),
});

// api.fixtures.ts: extends the page test
import { pageTest } from './page.fixtures';
export const apiTest = pageTest.extend<{ userAPI: UserAPI }>({
  userAPI: ({ request }, use) => use(new UserAPI(request)),
});

// index.ts: single composed export
export const test = apiTest;
export { expect } from '@playwright/test';

// any test: one import, all fixtures
import { test, expect } from '@fixtures/index';
```

Q 176 LEVEL · Mid

What is the difference between beforeAll/afterAll and fixtures, and when do you use each?

Hooks run per describe block; fixtures run per test/worker via DI. Fixtures guarantee teardown even if setup throws; afterAll may not. Fixtures are composable across files; hooks are local to the block. Default to fixtures for anything that creates a resource. Asked next: beforeAll fits one-time read-only setup with no cleanup. The easy mistake: beforeAll + shared mutable variables for resource setup.

```
// Fragile: afterAll may not run if beforeAll throws midway
test.describe('admin flow', () => {
  let userId: number;
  test.beforeAll(async ({ request }) => {
    userId = (await (await request.post('/api/users', { data: {} })).json()).id;
  });
  test.afterAll(async ({ request }) => { await request.delete(`/api/users/${userId}`); });
});

// Robust: fixture teardown ALWAYS runs
export const test = base.extend<{ adminUser: User }>({
  adminUser: async ({ request }, use) => {
    const user = await (await request.post('/api/users', { data: { role: 'admin' } })).json();
    await use(user);
    await request.delete(`/api/users/${user.id}`); // guaranteed
  },
});
```

Q 177 LEVEL · Junior to Mid

How do you use test.use() for per-test and per-describe configuration?

test.use overrides config per test/describe without changing global config. Sets storageState, viewport, locale, timezoneId per scope. Group by role; set storageState once at the block level. More maintainable than repeating storageState per test. Follow-up: organise tests in role-based describe blocks with test.use. Trap: duplicating storageState in every test.

```
// Per-describe: role-based blocks
test.describe('Admin panel', () => {
  test.use({ storageState: 'auth/admin-state.json' });
  test('can view all users', async ({ page }) => { /* ... */ });
  test('can delete users', async ({ page }) => { /* ... */ });
});

// Mobile viewport block
test.describe('Mobile checkout', () => {
  test.use({ viewport: { width: 390, height: 844 } });
  test('checkout works on mobile', async ({ page }) => { /* ... */ });
});

// Locale + timezone block
test.describe('Date formatting', () => {
  test.use({ locale: 'de-DE', timezoneId: 'Europe/Berlin' });
  test('dates display in German format', async ({ page }) => { /* ... */ });
});
```

Q 178 LEVEL · Mid

How do you implement environment-specific configuration in fixtures?

Env fixture exposes a typed config: **baseURL**, **apiURL**, **isCI**, **environment enum**. Tests switch behaviour by environment without code changes. Destructive tests check environment and test.skip() in production. Explicit, reviewable guard beats relying on developer memory. Expect next: read-only smoke runs everywhere; destructive skips in prod. Where this goes wrong: scattering environment branching across individual tests.

```
interface EnvConfig {
  baseURL: string; apiURL: string; isCI: boolean;
  environment: 'local' | 'staging' | 'production';
}

export const test = base.extend<{ envConfig: EnvConfig }>({
  envConfig: async ({}, use) => {
    const environment = (process.env.TEST_ENV || 'staging') as EnvConfig['environment'];
    await use({ baseURL: process.env.BASE_URL!, apiURL: process.env.API_URL!, isCI: !!process.env.CI, env...
  },
});

test('destructive: delete user', async ({ page, envConfig }) => {
  test.skip(envConfig.environment === 'production', 'No destructive tests in production');
  // ... safe to create/delete data
});
```

Q 179 LEVEL · Senior

How does Playwright resolve the fixture dependency graph, and what happens with a circular dependency?

Topological sort of the dependency graph: deps resolved bottom-up. Graph built from destructured argument lists. Circular deps caught at TEST DISCOVERY time, not runtime, fail-fast safety net. Flow rule: atomic fixtures (page, request) -> composed -> test-specific. Extract shared piece into a third fixture to break a circular dep. What loses the point: bidirectional deps, usually convenience helpers that grew responsibilities.

```
// Atomic -> composed -> test-specific (one direction)
type MyFixtures = {
  authenticatedPage: Page;          // composed
  adminPage:         Page;          // composed
  userBuilder:       UserBuilder;   // test-specific
};

export const test = base.extend<MyFixtures>({
  authenticatedPage: async ({ page }, use) => {                    // depends on page
    await page.goto('/login'); /* sign in */ await use(page);
  },
  adminPage: async ({ authenticatedPage }, use) => {               // depends on authenticatedPage
    await authenticatedPage.goto('/admin'); await use(authenticatedPage);
  },
  userBuilder: async ({ request }, use) => {                        // depends on request
    await use(new UserBuilder(request));
  },
});

// CIRCULAR (Playwright errors at discovery time):
...
```

Q 180 LEVEL · Mid to Senior

What are auto-fixtures, and when are they the right pattern?

- auto: true, runs for every test in scope without being requested.
- Right uses: observation (tracer, console-error guard) and guaranteed cleanup.
- Wrong uses: changing behaviour (locale, viewport), use test.use instead.
- Rule: **invisible**, **observe or guard**, **never change**.
- They usually ask next: debug by grep'ing 'auto: true', the implicit invocation is the cost.
- The common miss: auto-fixtures that modify behaviour, surprise behaviour from invisible code.

```
// GOOD: console-error guard (observes; fails the test on browser error)
export const test = base.extend<{ failOnConsoleError: void }>({
  failOnConsoleError: [async ({ page }, use, testInfo) => {
    const errors: string[] = [];
    page.on('console', msg => { if (msg.type() === 'error') errors.push(msg.text()); });
    await use();
    if (errors.length > 0) {
      testInfo.fail(true, `Console errors:\n${errors.join('\n')}`);
    }
  }, { auto: true }],   // <- runs for every test, no destructure needed
});

// BAD: auto-fixture that CHANGES behaviour (use test.use instead)
// localeFixture: [async ({ context }, use) => {
//   await context.addInitScript(() => Object.defineProperty(navigator, 'language', { value: 'fr-FR' }));
//   await use();
// }, { auto: true }],   // <- surprise! every test runs in French
```

Q 181 LEVEL · Senior

How do you handle teardown failures in fixtures so they don't mask the original test failure?

Teardown errors mask the original test failure if uncaught. Wrap each cleanup op in try-catch: log with context, continue. Principle: one cleanup failure should not prevent other cleanups. Exception: resource leaks affecting subsequent tests SHOULD still throw. Judgement: fail loudly on leaks; fail quietly on dead-data cleanup. Avoid: swallowed test failure because teardown threw, dev chases wrong bug.

```
export const test = base.extend<{ cleanup: Set<string>; request: APIRequestContext }>({
  cleanup: async ({ request }, use, testInfo) => {
    const ids = new Set<string>();
    await use(ids);                                    // - test runs here -

    for (const id of ids) {
      try {
        await request.delete(`/api/users/${id}`);
      } catch (err) {
        // Log, don't throw: don't mask the original test failure
        console.warn(`[cleanup] ${testInfo.title}, delete user ${id} failed:`, err);
      }
      // Loop continues to the next ID even if this one failed
    }
  },
});

// EXCEPTION, resource leak: STILL throw so it's loud
// const ctx = await browser.newContext();
// await use(ctx);
...
```

Q 182 LEVEL · Senior

What is fixture scope leakage, and how do you diagnose state surviving where it shouldn't?

Three patterns: **worker fixture mutating state**, **module-level var**, **shared paths**. Diagnose: **isolate the test**, **reverse the order**, **leakage is order-dependent**. Rule: worker-scoped fixtures immutable after setup; vary state per test via test-scope. Module-level vars outside fixtures are NEVER cleaned up between tests. The next question: leakage is order-dependent; flake is order-independent. The easy mistake: mutable worker-scoped state, order-of-run determines correctness.

```
// LEAKAGE PATTERN: worker fixture with mutable shared state
export const test = base.extend<{}, { cache: Map<string, string> }>({
  cache: [async ({}, use) => {
    const c = new Map<string, string>();        // shared across all tests in worker
    await use(c);
  }, { scope: 'worker' }],
});
// test A writes cache.set('user', 'A'); test B reads cache.get('user'), sees 'A'.
// Order-dependent. Order-dependent = leakage.

// DIAGNOSE
// 1. Run the suspect test alone: npx playwright test some.spec.ts --grep "failing test"
// 2. Reverse order: --workers=1 with tests in reversed order
// 3. Inspect for mutable worker fixtures or module-level vars

// CORRECT: test-scoped state, fresh per test
// const myFixture = async ({}, use) => { const x = new Map(); await use(x); };
```

Q 183 LEVEL · Mid

How does defineConfig give you type safety, and what happens without it?

defineConfig applies PlaywrightTestConfig type, typos become compile errors. Without it: 'retires: 2' compiles fine, silently disables retries. Project use blocks typed against the same interface, inheritance enforced. Extract fragments: baseUse: PlaywrightTestConfig['use'] spread per project. Asked next: retires-vs-retries is a real incident; months without retries. Trap: plain-object config, typos compile and silently disable features.

```
import { defineConfig, devices, PlaywrightTestConfig } from '@playwright/test';

// Reusable typed fragment: changes here propagate to every project
const baseUse: PlaywrightTestConfig['use'] = {
  baseURL: process.env.BASE_URL,
  trace: 'on-first-retry',
  screenshot: 'only-on-failure',
  testIdAttribute: 'data-test',
};

export default defineConfig({
  retries: process.env.CI ? 2 : 0,                  // ✓ typo here = compile error
  // retires: 2                                     // ✗ would compile WITHOUT defineConfig
  projects: [
    { name: 'chromium', use: { ...baseUse, ...devices['Desktop Chrome'] } },
    { name: 'firefox',  use: { ...baseUse, ...devices['Desktop Firefox'] } },
    { name: 'mobile',   use: { ...baseUse, ...devices['Pixel 7'] } },
  ],
});
```

Q 184 LEVEL · Mid to Senior

How do you use projects to model different test data environments?

Projects are general: **environment**, **role**, **locale**, **feature-flag matrices**. Same test file runs across cartesian product of projects. Each project: **own use**, **storageState**, **testMatch**, **no test duplication**. Filter aggressively with --project in CI: don't run full matrix on every PR. Follow-up: PR -> chromium-admin-en; main -> role matrix; nightly -> full. Where this goes wrong: treating projects as browser-only, missing the general-purpose power.

```
// playwright.config.ts: projects model role + locale matrix
export default defineConfig({
  projects: [
    { name: 'setup', testMatch: /.*\.setup\.ts/ },
    {
      name: 'admin-en',
      dependencies: ['setup'],
      use: { storageState: 'auth/admin.json', locale: 'en-US' },
    },
    {
      name: 'admin-fr',
      dependencies: ['setup'],
      use: { storageState: 'auth/admin.json', locale: 'fr-FR' },
    },
    {
      name: 'viewer-en',
      dependencies: ['setup'],
      use: { storageState: 'auth/viewer.json', locale: 'en-US' },
    },
    // 6+ more projects as roles × locales grows
...
```

Q 185 LEVEL · Mid

How do you configure outputDir and write parallel-safe file paths from tests?

Playwright auto-namespaces its own artifacts by test ID under outputDir. testInfo.outputPath('file') for paths Playwright manages, per-test scope. testInfo.parallelIndex for explicit worker-level namespacing. Filesystem is NOT context-isolated: OS lives outside the browser process. Expect next: context isolates browser state; filesystem stays shared. What loses the point: hardcoded paths like /tmp/output, parallel workers collide silently.

```
test('generates a CSV report', async ({ page }, testInfo) => {
  await page.goto('/reports');
  const rows = await page.locator('.row').allTextContents();

  // SAFE: per-test scoped path managed by Playwright
  const csvPath = testInfo.outputPath('report.csv');
  fs.writeFileSync(csvPath, rows.join('\n'));
  await testInfo.attach('report.csv', { path: csvPath });

  // ALSO SAFE: explicit worker namespacing
  const sharedPath = path.join(testInfo.outputDir, `worker-${testInfo.parallelIndex}`, 'shared.csv');
  fs.mkdirSync(path.dirname(sharedPath), { recursive: true });
  fs.writeFileSync(sharedPath, '...');
});

// WRONG: silent collision under parallel execution
// fs.writeFileSync('/tmp/output.csv', data);
```

Q 186 LEVEL · Senior

What are the override semantics of test.use across config, project, describe, and test scopes?

- 4 levels (specific wins): config -> project -> describe -> per-test.
- Options merge SHALLOWLY: nested fields don't merge, full redeclare needed.
- Place the broadest setting at config; override progressively as scope narrows.
- test.use must be at describe-level or above: inside a test it doesn't apply.
- They usually ask next: test.use inside a test body errors (or silently fails in older versions).
- The common miss: expecting nested options to merge, they don't; redeclare in full.

```
// 1. Config-level (top-level use): applies to all projects
export default defineConfig({
  use: { viewport: { width: 1280, height: 720 }, locale: 'en-US' },
  projects: [
    // 2. Project-level: overrides config-level for this project
    { name: 'mobile', use: { viewport: { width: 390, height: 844 } } },
  ],
});

// 3. Describe-level: overrides project for tests inside
test.describe('French locale block', () => {
  test.use({ locale: 'fr-FR' });
  test('renders in French', async ({ page }) => { /* ... */ });
});

// 4. Per-test: overrides describe-level for this one test
test('this one needs a custom viewport', async ({ page }) => {
  // test.use({ viewport: { width: 800 } });  // ✗ WRONG, must be outside the test
});
test.use({ viewport: { width: 800, height: 600 } });   // ✓ applies to next test
```

Q 187 LEVEL · Mid to Senior

globalSetup/globalTeardown vs project dependencies (setup projects), which do you use for auth and one-time setup, and why are setup projects usually better?

One-line contrast: globalSetup runs OUTSIDE the runner; setup project runs INSIDE it as a real test. globalSetup: no fixtures, no trace, no report line, failures are raw stack traces. Setup project: page/request fixtures, traces + screenshots on failure, report line, retries. Setup projects compose: different main projects depend on different auth setups (admin vs viewer). Default for auth/one-time browser setup: a setup project via dependencies. The next question: globalSetup still fits browser-less env work, seed a DB, start a mock server, set env vars. Avoid: doing the login in globalSetup, no trace the day auth breaks.

```
// == OPTION A: globalSetup (outside the runner) ==
// playwright.config.ts
export default defineConfig({ globalSetup: require.resolve('./global-setup') });

// global-setup.ts: no fixtures, no trace; drive the browser by hand
async function globalSetup() {
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto('https://app/login'); /* ...sign in... */
  await page.context().storageState({ path: 'auth/admin.json' });
  await browser.close();                 // a failure here = raw stack trace, no report line
}
export default globalSetup;

// == OPTION B (usually better): setup project via dependencies ==
// auth.setup.ts, a REAL test: gets fixtures, trace, retries, a report line
setup('authenticate as admin', async ({ page }) => {
  await page.goto('/login'); /* ...sign in... */
  await page.context().storageState({ path: 'auth/admin.json' });
});
...
```

QUICK QUIZ

1. How does Playwright run fixture teardown when a test throws?
   A) Teardown is skipped on failure
   B) Teardown runs in setup order
   C) Only the last fixture tears down
   D) Teardown still runs, in reverse dependency order (try/finally semantics)
2. When is a worker-scoped fixture the right choice?
   A) For data that must be fresh per test
   B) Never: it breaks isolation
   C) Only for the Page object
   D) For expensive, stateful setup (e.g. an authenticated context) shared across a worker's tests
3. What is the decisive advantage of fixtures over beforeAll/afterAll?
   A) Fixtures run faster
   B) beforeAll cannot do setup
   C) Teardown is guaranteed even if setup throws, and fixtures compose across files
   D) Fixtures avoid using the API
4. What is test.use() the correct tool for?
   A) Overriding config (storageState, viewport, locale) per test or describe block
   B) Changing the global config file
   C) Importing fixtures
   D) Running tests in serial
5. How does an environment fixture keep destructive tests out of production?
   A) It blocks all network calls
   B) It exposes the environment so tests call test.skip() when it is production
   C) It deletes production data first
   D) It runs only read-only tests automatically

Answers: 1:D 2:D 3:C 4:A 5:B

Advanced Playwright Patterns

Q 188 LEVEL · Mid

How do you implement the Builder pattern for test data?

Fluent API: chainable methods returning this, build() and create(). faker seeds defaults; tests override only what matters. One line communicates intent ('I need an admin') vs ten lines of setup. Schema change -> update the builder once, all tests stay correct. Asked next: single source of truth for test data, like typed contracts. The easy mistake: duplicating faker data setup across every test.

```
import { faker } from '@faker-js/faker';

export class UserBuilder {
  private user = { name: faker.person.fullName(), email: faker.internet.email(), role: 'user' };
  withRole(role: string): this { this.user.role = role; return this; }
  asAdmin():  this { this.user.role = 'admin';  return this; }
  asViewer(): this { this.user.role = 'viewer'; return this; }
  build() { return { ...this.user }; }
  async create(request: APIRequestContext) {
    return (await request.post('/api/users', { data: this.user })).json();
  }
}

// Tests state only what matters
const admin  = await new UserBuilder().asAdmin().create(request);
const viewer = await new UserBuilder().asViewer().create(request);
```

Q 189 LEVEL · Mid

How do you implement retry logic for unstable external dependencies?

withRetry wraps a transient operation with attempts, delay, backoff. Not the same as Playwright test retries: operation-level vs whole-test. For external systems whose latency/failures aren't app bugs. Exponential backoff returns on success; fixed sleep is flaky or slow. Follow-up: Playwright retries re-run the whole test, too coarse here. Trap: conflating the two retry levels or using a fixed sleep.

```
export async function withRetry<T>(
  operation: () => Promise<T>,
  { attempts = 3, delayMs = 1000, backoff = 2, onRetry }: {
    attempts?: number; delayMs?: number; backoff?: number;
    onRetry?: (attempt: number, error: Error) => void;
  } = {},
): Promise<T> {
  for (let attempt = 1; attempt <= attempts; attempt++) {
    try { return await operation(); }
    catch (error) {
      if (attempt === attempts) throw error;
      onRetry?.(attempt, error as Error);
      await new Promise(r => setTimeout(r, delayMs * backoff ** (attempt - 1)));
    }
  }
  throw new Error('unreachable');
}

const code = await withRetry(() => emailClient.getLatestCode('user@test.com'), { attempts: 5, delayMs: 20...
```

Q 190 LEVEL · Mid

How do you implement a test data cleanup strategy?

Registry collects delete fns as resources are created; runs in reverse. Works even on partial-setup failure: only created resources are cleaned. try/catch per deletion so one failure doesn't block the rest. Prevents orphaned data that accumulates into cross-test interference. Expect next: fail mid-setup -> only resources created so far are deleted. Where this goes wrong: cleanup that only runs on the happy path.

```
export const test = base.extend<{ cleanup: (fn: () => Promise<void>) => void }>({
  cleanup: async ({}, use) => {
    const fns: Array<() => Promise<void>> = [];
    await use(fn => fns.push(fn));
    for (const fn of fns.reverse()) {
      try { await fn(); } catch (e) { console.warn('Cleanup failed:', e); }
    }
  },
});

test('complex setup with cleanup', async ({ request, cleanup }) => {
  const user = await (await request.post('/api/users', { data: {} })).json();
  cleanup(() => request.delete(`/api/users/${user.id}`));
  const order = await (await request.post('/api/orders', { data: { userId: user.id } })).json();
  cleanup(() => request.delete(`/api/orders/${order.id}`));
  // if the test fails here, both deletions still run in reverse
});
```

Q 191 LEVEL · Mid to Senior

How do you implement visual component testing with Playwright?

Mounts a component in a real browser without the full app. Assert rendering and interaction with the normal Playwright API. Fills the unit-to-E2E gap: unit mocks the DOM, E2E needs the whole stack. Real rendering/CSS/events without full-app overhead. They usually ask next: use it for isolated components (design system), not full journeys. What loses the point: positioning it as a replacement for E2E.

```
// playwright-ct.config.ts
import { defineConfig } from '@playwright/experimental-ct-react';
export default defineConfig({ testDir: './src', use: { ctPort: 3100 } });

// Button.spec.tsx
import { test, expect } from '@playwright/experimental-ct-react';
import { Button } from './Button';

test('renders with label and variant', async ({ mount }) => {
  const component = await mount(<Button label="Submit Order" variant="primary" />);
  await expect(component).toContainText('Submit Order');
  await expect(component).toHaveClass(/btn-primary/);
});

test('calls onClick when clicked', async ({ mount }) => {
  let clicked = false;
  const component = await mount(<Button label="Click" onClick={() => { clicked = true; }} />);
  await component.click();
  expect(clicked).toBe(true);
});
```

Q 192 LEVEL · Mid to Senior

How do you implement a Page Object for a complex SPA with dynamic routing?

Handle URL patterns, query params, and async content in the page object. Build filter query strings with typed URLSearchParams, one-method edits. URL changes instantly; content loads async: wait for real content. waitForURL on the pattern, then wait for a content element. The next question: why wait after the URL changes, SPA renders data asynchronously. The common miss: asserting right after navigation, racing the data load.

```
export class ProductPage extends BasePage {
  readonly productTitle = this.page.getByRole('heading', { level: 1 });

  async navigateToProduct(productId: string): Promise<void> {
    await this.page.goto(`/products/${productId}`);
    await this.page.waitForURL(`**/products/${productId}`);
    await this.productTitle.waitFor({ state: 'visible' }); // content loads async
  }

  async navigateWithFilters(f: { category?: string; sort?: 'price' | 'rating' | 'newest' }): Promise<void> {
    const params = new URLSearchParams();
    if (f.category) params.set('category', f.category);
    if (f.sort) params.set('sort', f.sort);
    await this.page.goto(`/products?${params.toString()}`);
    await this.page.locator('#product-grid').waitFor({ state: 'visible' });
  }
}
```

Q 193 LEVEL · Mid to Senior

What's the difference between Object Mother, Builder, and Factory patterns for test data?

Object Mother: named pre-configured instances, reads as scenarios. Builder: fluent composition for multi-dimensional variation. Factory function: **simplest**, **options-bag**, **fine for small/flat data**. Object Mother explodes combinatorially with many dimensions. Rule: **factory for small**, **builder for complex**, **Mother for canonical scenarios**. Avoid: Object Mother only, named-scenario list grows until unnavigable.

```
// FACTORY: simplest, fine for small flat data
export function createUser(opts: Partial<UserData> = {}): UserData {
  return { email: faker.internet.email(), role: 'user', ...opts };
}

// BUILDER: fluent composition for complex objects
export class UserBuilder {
  private data: Partial<UserData> = {};
  asAdmin()                    { this.data.role = 'admin'; return this; }
  withEmail(email: string)     { this.data.email = email;  return this; }
  inTenant(tenantId: string)   { this.data.tenantId = tenantId; return this; }
  build(): UserData            { return { ...defaults, ...this.data }; }
}

// OBJECT MOTHER: named canonical scenarios layered on builder
export const UserMother = {
  aValidAdmin:        () => new UserBuilder().asAdmin().build(),
  aLockedOutCustomer: () => new UserBuilder().withStatus('locked').build(),
  aTrialNearExpiry:   () => new UserBuilder().onTrial().withTrialDaysRemaining(1).build(),
};
...
```

Q 194 LEVEL · Senior

What's the Meszaros taxonomy of test doubles, and why does precise vocabulary matter?

Dummy: passed but never used, fills parameter slots. Stub: **returns canned answers**, **state**, **no behaviour verification**. Spy: stub + records calls, assert arguments after the fact. Mock: pre-programmed expectations, fails test if not called as expected. Fake: **working impl with shortcuts**, **in-memory DB**, **localhost SMTP**. Asked next: stub the auth response (state) vs mock the email send (behaviour).

```
// STUB: controls returned state, no verification of call
await page.route('**/api/user', route =>
  route.fulfill({ status: 200, body: JSON.stringify({ id: 1, role: 'admin' }) }));

// SPY: records the calls made to it; assert on them after the test action
const calls: string[] = [];
await page.route('**/api/audit', route => {
  calls.push(route.request().postData() ?? '');
  route.fulfill({ status: 200 });
});
// ... perform action ...
expect(calls).toHaveLength(1);
expect(JSON.parse(calls[0]!).event).toBe('user_login');

// MOCK: expectation that the call happens with specific shape (sinon-style)
// emailService.expects('send').once().withArgs(sinon.match({ to: 'anna@bank.com' }));

// FAKE: working implementation with shortcuts (testcontainers)
// const pg = await new PostgreSqlContainer('postgres:17').withTmpFs({ '/var/lib/postgresql/data': 'rw' }...
```

Q 195 LEVEL · Mid to Senior

What is the Specification pattern, and when does it cleaner than chained assertions?

Spec encapsulates a business rule as a composable predicate. Composes via .and(), .or(), .not(): complex rules from simple ones. Strength: **shared definition**, **change rule once**, **every test updates**. Weakness: machinery overhead for simple checks. Rule: reach when rule has 3+ conditions and a name in the domain. The easy mistake: over-using for trivial checks, class hierarchy for one expect.

```
// Specification encapsulates a domain rule
interface Spec<T> { isSatisfiedBy(item: T): boolean; }

class And<T> implements Spec<T> {
  constructor(private a: Spec<T>, private b: Spec<T>) {}
  isSatisfiedBy(item: T): boolean { return this.a.isSatisfiedBy(item) && this.b.isSatisfiedBy(item); }
}

const OrderIsConfirmed: Spec<Order>     = { isSatisfiedBy: o => o.status === 'confirmed' };
const OrderHasItems: Spec<Order>        = { isSatisfiedBy: o => o.items.length > 0 };
const OrderHasShippingAddr: Spec<Order> = { isSatisfiedBy: o => Boolean(o.shippingAddress) };

// Composed, named domain rule
const OrderReadyToShip = new And(OrderIsConfirmed, new And(OrderHasItems, OrderHasShippingAddr));

// Test reads as intent, not implementation details
test('shipping fulfilment picks up ready orders', async ({ request }) => {
  const order = await (await request.get('/api/orders/123')).json();
  expect(OrderReadyToShip.isSatisfiedBy(order)).toBe(true);
});
```

Q 196 LEVEL · Senior

What is the Saga pattern in test orchestration, and when do you need it?

Saga = multi-step flow + compensating action per step + reverse unwind on failure. Use cases: **payment-checkout-shipment**, **multi-system**, **no-transaction services**. Declare compensation AT creation, not at end: throw mid-test still unwinds. Walks compensations in reverse for only the steps that completed. Follow-up: **differs from cleanup fixtures**, **ordered**, **step-tied**, **not flat**. Trap: confusing Saga with serial mode, different concerns.

```
class Saga {
  private compensations: Array<() => Promise<void>> = [];
  async run<T>(step: () => Promise<T>, compensate: (result: T) => Promise<void>): Promise<T> {
    const result = await step();
    this.compensations.unshift(() => compensate(result));   // unshift = reverse order
    return result;
  }
  async unwind(): Promise<void> {
    for (const c of this.compensations) { try { await c(); } catch { /* keep going */ } }
  }
}

test('full checkout flow with reliable unwind', async ({ request }) => {
  const saga = new Saga();
  try {
    const order = await saga.run(
      () => request.post('/api/orders', { data: { product: 'X' } }).then(r => r.json()),
      (o) => request.delete(`/api/orders/${o.id}`).then(() => undefined),
    );
    const charge = await saga.run(
...
```

Q 197 LEVEL · Senior

How do you apply hexagonal architecture (ports and adapters) to test code itself?

Tests depend on ports (interfaces describing domain operations). Adapters implement ports for environments: **HTTP**, **browser**, **SQL**. Same test logic across environments: swap adapter at fixture level. Real example: REST -> GraphQL migration, new adapter, unchanged tests. Discipline: ports describe operations, not implementation details. Where this goes wrong: port leaks implementation, abstraction becomes renamed adapter.

```
// PORT: describes domain operations the test needs
interface UserRepository {
  findByEmail(email: string): Promise<User | null>;
  create(user: NewUser): Promise<User>;
  delete(id: string): Promise<void>;
}

// ADAPTERS: environment-specific implementations
class HttpUserRepository implements UserRepository { /* uses APIRequestContext */ }
class GraphQLUserRepository implements UserRepository { /* same operations via GraphQL */ }
class SqlUserRepository implements UserRepository { /* direct DB for verification */ }
class PlaywrightUserRepository implements UserRepository { /* via browser UI */ }

// Fixture chooses the adapter for this test run
export const test = base.extend<{ users: UserRepository }>({
  users: async ({ request }, use) => {
    await use(new HttpUserRepository(request));   // swap to GraphQL/SQL without changing tests
  },
});

...
```

Q 198 LEVEL · Senior

Beyond toHaveScreenshot, what is the production playbook for visual regression at scale?

toHaveScreenshot baseline approach: **free**, **in-repo**, **fine for stable pages**. Cross-OS flake: pin to mcr.microsoft.com/playwright Docker image. Dynamic content: mask (paints pink) or clip (capture only stable region). Baseline-update burden: Applitools or Percy for AI grouping plus semantic diffs. Rule: toHaveScreenshot for stable pages; AI tools when burden bottlenecks. Pitfall: enabling visual tests everywhere -- noise, distrust, abandonment.

```
// toHaveScreenshot with mask plus clip for noise control
test('checkout page visual regression', async ({ page }) => {
  await page.goto('/checkout');
  await expect(page.getByRole('heading', { name: 'Checkout' })).toBeVisible();

  await expect(page).toHaveScreenshot('checkout.png', {
    fullPage: true,
    mask: [
      page.locator('[data-testid="timestamp"]'),
      page.locator('[data-testid="user-avatar"]'),
    ],
    maxDiffPixelRatio: 0.01,
    animations: 'disabled',
  });
});

// Pin visual baselines to the Playwright Docker image
// Update baselines from inside the image, not from your laptop:
//   docker run -it --rm -v $(pwd):/work mcr.microsoft.com/playwright:v1.45.0-jammy \
//     bash -c 'cd /work && npx playwright test --update-snapshots --project=visual'
...
```

QUICK QUIZ

1. What problem does the Builder pattern solve for test data?
   A) Tests state only the fields that matter; schema changes update one builder
   B) It makes tests run in parallel
   C) It removes the need for the API
   D) It replaces fixtures
2. How does withRetry differ from Playwright's built-in test retries?
   A) They are identical
   B) withRetry retries one operation inside a test; Playwright retries the whole test
   C) withRetry only works in CI
   D) Playwright retries individual operations
3. Why use a cleanup registry instead of deleting data at the end of the test?
   A) It is faster
   B) It runs cleanup in setup order
   C) It avoids using the API
   D) It deletes only the resources created so far, even if the test fails mid-setup
4. What gap does Playwright Component Testing fill?
   A) It replaces end-to-end tests
   B) It tests only APIs
   C) Between unit (mocks the DOM) and E2E (needs the whole stack), real rendering, isolated
   D) It is only for visual screenshots
5. Why must an SPA page object wait for content after navigation, not just the URL?
   A) URLs are unreliable
   B) It speeds up the test
   C) Playwright cannot read URLs
   D) In an SPA the URL changes instantly but content loads asynchronously

Answers: 1:A 2:B 3:D 4:C 5:D

AI for SDET Engineers

Q 199 LEVEL · Mid

How is AI transforming test automation in 2026?

Five areas: **generation**, **self-healing**, **maintenance**, **defect prediction**, **visual AI**. Framing: accelerator, not replacement. AI scaffolds; you review every line and upgrade CSS to getByRole. Expect next: the SDET job is prompt engineering + evaluating the output. Generation is cheap; judgement on locators/awaits/coverage is the value. What loses the point: sounding either threatened by AI or naive about it.

Q 200 LEVEL · Mid

How do you use LLMs for test case generation from requirements?

Prompt quality decides everything; vague prompts give happy-path only. Explicit acceptance criteria drive ~80% coverage. Supply page objects; ask for the test file only, using project fixtures. Engineer owns the 20%: **auth setup**, **API assertions**, **data strategy**. They usually ask next: the LLM misses cross-cutting setup and negative/edge coverage. The common miss: committing generated tests unreviewed.

```
/* Effective test-generation prompt
 User Story: As a user I can log in with email and password.
 Acceptance Criteria:
  - Valid credentials redirect to /dashboard
  - Invalid password shows "Invalid email or password"
  - Empty email shows "Email is required"
  - Account locks after 5 failed attempts ("Account locked")
 Page Objects: LoginPage (navigate, login, getErrorMessage)
 Generate Playwright TypeScript tests using @fixtures/pages.fixture.
 Cover positive, negative, and edge cases. Output only the test file. */
```

Q 201 LEVEL · Mid

How do you implement self-healing locators in a Playwright framework?

Try fallback strategies in order; return the one that resolves uniquely. Log a warning identifying what healed. Framing: a maintenance strategy, not a permanent fix. Each heal is technical debt: the warning becomes a ticket. The next question: doesn't hide breakage because the heal is logged and scheduled. Avoid: using healing as a reason never to fix locators.

```
export async function healingLocator(
  page: Page, strategies: Array<() => Locator>, description: string,
): Promise<Locator> {
  for (let i = 0; i < strategies.length; i++) {
    const locator = strategies[i]();
    if (await locator.count() === 1) {
      if (i > 0) console.warn(`[HEALING] Primary failed for "${description}"; used fallback ${i + 1}. Upd...
      return locator;
    }
  }
  throw new Error(`All locator strategies failed for: ${description}`);
}

const submit = await healingLocator(page, [
  () => page.getByRole('button', { name: 'Submit Order' }), // primary
  () => page.getByTestId('submit-order-btn'),               // fallback 1
  () => page.locator('[data-action="submit-order"]'),       // fallback 2
], 'Submit Order button');
```

Q 202 LEVEL · Mid to Senior

How do you use AI for test maintenance and failure analysis?

Feed the LLM three inputs: **error message**, **test code**, **recent diff**. Best for strict-mode violations and post-UI-change locator failures. The diff lets the model reason about cause (a new button was added). A VS Code tool can cut locator fix time from ~20 min to ~3. Asked next: the LLM infers cause by correlating error with diff. The easy mistake: auto-applying AI fixes instead of reviewing them.

```
/* AI-assisted failure-analysis prompt
 ERROR: strict mode violation: locator('button') resolved to 8 elements
 TEST CODE: await page.locator('button').click();
 RECENT DIFF:
   + <button class="btn-secondary">Cancel</button>
   + <button class="btn-primary">Submit Order</button>
 PAGE OBJECTS: CheckoutPage (submitButton, cancelButton)
 Suggest the correct fix using Playwright best practices. */

// AI output (reviewed, then applied):
// await page.getByRole('button', { name: 'Submit Order' }).click();
// Reason: a new Cancel button made locator('button') ambiguous.
```

Q 203 LEVEL · Mid to Senior

How do you evaluate and validate AI-generated test code?

Review against a checklist targeting AI's predictable mistakes. Five checks: **locator quality**, **awaits**, **assertion type**, **independence**, **negative cases**. ESLint auto-catches awaits (no-floating-promises) and assertion type. Manual review focuses on independence and negative coverage. Net: ~15 min review vs ~45 writing, ~3x gain, bar held. Trap: trusting AI output by default.

Q 204 LEVEL · Senior

What is Playwright MCP, and how does it change the developer experience around test authoring?

MCP = Anthropic's standard for AI access to external tools. Playwright MCP server exposes browser ops to AI clients (Claude, Cursor). AI drives real browser, queries live DOM, emits selector-correct code. Uses: **scaffold first test**, **diagnose failures**, **generate variations**. Discipline: **starting points**, **not finished work**, **still need review**. Where this goes wrong: treating MCP-generated tests as production-ready, skip review at cost.

```
// .mcp/playwright.config.json: register Playwright MCP server with the AI client
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp", "--browser", "chrome", "--headless"]
    }
  }
}

// Developer workflow:
//   1. Prompt: "Write a Playwright test for the checkout flow on staging.acme.com.
//               Use storageState 'auth/user.json'. Cover happy path + card decline."
//   2. AI uses MCP to navigate to the URL, inspect the actual DOM, find selectors.
//   3. AI emits TypeScript code with selectors verified against the live page.
//   4. ALWAYS review: selectors, auto-waiting, negative cases, independence.
//   5. Add the cases the AI missed; commit.
//
// What actually saves time: the first draft. What still needs you: the polish.
```

Q 205 LEVEL · Mid to Senior

What's the structure of a prompt that reliably produces high-quality Playwright tests?

Five sections: **role**, **constraints**, **domain**, **example output**, **task**. Constraints do the most work: stop the model defaulting to common patterns. Without constraints: output mixes Playwright/Cypress/Selenium/Jest idioms. Example output format teaches the model your team's style. Codify conventions in a reusable prompt template, don't retype each time. What loses the point: free-form prompts, inconsistent output, no leverage from prior work.

````
# prompts/playwright-test.md: reusable team prompt template

## Role & context
You are an SDET writing **Playwright TypeScript** tests using @playwright/test.
Tests run with `fullyParallel: true` on Chromium, Firefox, WebKit.

## Constraints (MUST follow)
- Locators: only `getByRole`, `getByLabel`, `getByTestId`, no CSS, no XPath.
- Assertions: only web-first (`toBeVisible`, `toHaveText`, etc), never `isVisible()` in expects.
- Auto-waiting: never `page.waitForTimeout`. Use `expect.toHaveURL` or `waitForResponse`.
- Test independence: each test creates its own data via the `userBuilder` fixture.
- One scenario per test; describe blocks group related tests.

## Domain context
App: acme.com. Stripe-hosted payment widget at iframe `[title="Payment"]`.
Test users: storageState at `auth/{role}.json`. Roles: admin, user, viewer.

## Example output
```ts
test('admin removes a user', async ({ adminPage, userBuilder }) => {
...
````
````

Q 206 LEVEL · Senior

What is AI-driven exploratory testing, and where does it fit in a test strategy?

Agent + browser + goal: explores autonomously, reports findings. Covers unknown unknowns: **input combos**, **paths**, **feature interactions**. Complements scripted regression: doesn't replace it. Place: discovery before scripting, reproducible bugs become scripted tests. Follow-up: every finding validated by human reproduction. The common miss: replacing scripted regression with exploratory, lose deterministic coverage.

Q 207 LEVEL · Senior

When is LLM-as-judge a valid assertion technique, and how do you keep it from being arbitrary?

LLM-as-judge: model evaluates output against a rubric, not a fixed value. Use when output is semantically variable but rule-bound (AI responses, dynamic email). Rubric with explicit criteria: **'mentions one benefit**, **no superlatives'**, **not 'good'**. Calibrate: hand-label 50, test 80%+ agreement before CI use. 2026 growth case: testing apps that themselves use AI. Avoid: **LLM-as-judge for deterministic outputs**, **expensive**, **non-deterministic**, **wrong**.

```
// LLM-as-judge with an explicit rubric (anthropic SDK)
import Anthropic from '@anthropic-ai/sdk';
const client = new Anthropic();

async function judgeChatbotResponse(userQuery: string, botResponse: string): Promise<boolean> {
  const prompt = `
Rubric (return JSON):
  - on_topic: response addresses the user's question
  - factual: no claims contradicting our docs
  - polite: no aggressive or dismissive tone
  - no_personal_data: no leaked PII

User query: ${userQuery}
Bot response: ${botResponse}

Return JSON: {"on_topic": bool, "factual": bool, "polite": bool, "no_personal_data": bool}`;
  const result = await client.messages.create({
    model: 'claude-opus-4-7', max_tokens: 200,
    messages: [{ role: 'user', content: prompt }],
  });
...
```

Q 208 LEVEL · Senior

How do you protect a test suite from AI hallucinations and AI-generated bugs?

Four-layer defence: **TypeScript compile**, **ESLint**, **schema validation**, **human review**. TypeScript catches non-existent APIs (hallucinated methods/properties). Schema validation catches data-shape hallucinations (wrong field names). Human checklist: selectors exist? APIs match OpenAPI? data shape valid? Expect next: TypeScript catches most API hallucinations; schemas catch most data ones. The easy mistake: AI verifying its own work, same hallucination risk as generation.

```
// Defence layer 1: strict TypeScript, hallucinated APIs fail at compile time
// tsconfig.json: "strict": true, "noImplicitAny": true, "strictNullChecks": true
// await page.getByRole('button').clickIt();  <- TS2339: clickIt does not exist on Locator

// Defence layer 2: ESLint catches framework idiom hallucinations
// .eslintrc.json:
//   "plugins": ["playwright"],
//   "rules": {
//     "playwright/no-wait-for-timeout": "error",
//     "playwright/no-conditional-in-test": "warn",
//     "playwright/prefer-web-first-assertions": "error",
//     "playwright/missing-playwright-await": "error",
//   }

// Defence layer 3: schema validation of test data, hallucinated shapes fail at runtime
const UserRequestSchema = z.object({ email: z.string().email(), role: z.enum(['admin','user']) });
const body = UserRequestSchema.parse(aiGeneratedBody);  // throws on shape mismatch

// Defence layer 4: human-review checklist (PR template)
//   [ ] All getByRole/getByLabel/getByTestId selectors verified against live page
...
```

QUICK QUIZ

1. What is the professional framing for AI in test automation?
   A) An accelerator paired with the expertise to evaluate its output
   B) A full replacement for SDETs
   C) A tool to avoid
   D) Only useful for visual testing
2. What most improves LLM-generated test coverage?
   A) A longer prompt
   B) Asking for an explanation
   C) Explicit acceptance criteria in the prompt
   D) Using GPT over Claude
3. How should a healed self-healing locator be treated?
   A) As a permanent fix: no action needed
   B) As technical debt: the logged warning becomes a ticket to fix the primary
   C) As a test failure
   D) As a reason to delete the test
4. Which three inputs let an LLM diagnose a failing test's root cause?
   A) Screenshot, video, trace
   B) Config, fixtures, and reporter
   C) Test name, author, and date
   D) Error message, test code, and the recent diff
5. Which AI-generated-test mistakes can ESLint catch automatically?
   A) Test independence and negative cases
   B) Locator naming and comments
   C) Missing awaits (no-floating-promises) and assertion type (prefer-web-first-assertions)
   D) None: all review is manual

Answers: 1:A 2:C 3:B 4:D 5:C

Test Design Techniques

Q 209 LEVEL · Junior to Mid

How do you apply Equivalence Partitioning in Playwright tests?

Partition inputs into groups that behave identically; test one each. Numbers sell it: ~5 partition tests vs 121 exhaustive, equal confidence. Parameterised test with the partition name in the title, gaps visible. Justifies coverage to stakeholders without exhaustive testing. They usually ask next: any value in a partition works since it behaves identically. Trap: testing many values inside one partition for no extra coverage.

```
const ageCases = [
  { age: -1,  expect: 'error',   partition: 'below-min' },
  { age: 25,  expect: 'success', partition: 'valid' },
  { age: 121, expect: 'error',   partition: 'above-max' },
];
for (const { age, expect: outcome, partition } of ageCases) {
  test(`age validation, ${partition}: ${age}`, async ({ page }) => {
    await page.goto('/profile');
    await page.getByLabel('Age').fill(String(age));
    await page.getByRole('button', { name: 'Save' }).click();
    if (outcome === 'success') await expect(page.locator('.success-message')).toBeVisible();
    else await expect(page.locator('.field-error')).toBeVisible();
  });
}
```

Q 210 LEVEL · Junior to Mid

How do you apply Boundary Value Analysis in Playwright?

Test min-1/min/min+1 and max-1/max/max+1: the boundary neighbours. Catches > vs >= off-by-one that mid-range values miss. One-liner: the boundary is where the condition changes, where bugs live. Pairs with partitioning: classes from EP, edges hardened by BVA. The next question: **why both**, **EP covers classes**, **BVA covers the transitions**. Where this goes wrong: testing only mid-range values and skipping the edges.

```
// Password length: min 8, max 64
const bva = [
  { len: 7,  valid: false, label: 'below-min' },
  { len: 8,  valid: true,  label: 'min-boundary' },
  { len: 9,  valid: true,  label: 'min-plus-one' },
  { len: 64, valid: true,  label: 'max-boundary' },
  { len: 65, valid: false, label: 'above-max' },
];
for (const { len, valid, label } of bva) {
  test(`password BVA: ${label} (${len})`, async ({ page }) => {
    await page.goto('/register');
    await page.getByLabel('Password').fill('A'.repeat(len - 2) + '1!');
    await page.getByRole('button', { name: 'Register' }).click();
    if (valid) await expect(page.locator('.password-error')).toBeHidden();
    else await expect(page.locator('.password-error')).toBeVisible();
  });
}
```

Q 211 LEVEL · Mid

How do you implement Decision Table testing in Playwright?

Map condition combinations to outcomes; each row is one test. Covers all meaningful combinations without combinatorial explosion. Communication: PMs verify coverage from the table directly. New rule -> add a row and a test; coverage tracks requirements. Asked next: **keeping coverage in sync**, **one row per rule**, **gaps are obvious**. What loses the point: testing combinations ad hoc and silently missing one.

```
// premium + >$100 = 20% | premium + <=$100 = 10%
// regular + >$100 = 5%  | regular + <=$100 = 0%
const discountTable = [
  { premium: true,  total: 150, discount: '20%' },
  { premium: true,  total: 50,  discount: '10%' },
  { premium: false, total: 150, discount: '5%' },
  { premium: false, total: 50,  discount: '0%' },
];
for (const { premium, total, discount } of discountTable) {
  test(`discount: ${premium ? 'premium' : 'regular'} + $${total} = ${discount}`, async ({ page, request }...
    await new UserBuilder().withRole(premium ? 'premium' : 'user').create(request);
    await page.goto(`/checkout?total=${total}`);
    await expect(page.locator('#discount-display')).toHaveText(discount);
  });
}
```

Q 212 LEVEL · Mid

How do you apply State Transition testing in Playwright?

Test valid transitions and that invalid ones are rejected (right error + 422). Discipline: draw the diagram, then cover every transition and terminal state. Invalid-transition tests verify rule enforcement, not the happy path. Catches the API allowing a jump the UI prevents, a real security gap. Follow-up: why test invalid transitions, data-integrity enforcement. The common miss: testing only valid transitions.

```
test('order follows valid transitions', async ({ request, page }) => {
  const order = await new OrderBuilder().asDraft().create(request);
  await request.post(`/api/orders/${order.id}/submit`);
  await page.goto(`/orders/${order.id}`);
  await expect(page.locator('#order-status')).toHaveText('Submitted');
});

test('cannot ship a draft order: invalid transition', async ({ request }) => {
  const order = await new OrderBuilder().asDraft().create(request);
  const res = await request.post(`/api/orders/${order.id}/ship`);
  expect(res.status()).toBe(422);
  expect((await res.json()).error).toContain('Cannot ship an order in Draft status');
});
```

Q 213 LEVEL · Mid to Senior

How do you design a test pyramid for a Playwright-based framework?

Distribution: **~30% unit**, **30% component**, **30% API**, **10% E2E**. Lower layers are faster/cheaper; push coverage down the pyramid. Reserve E2E for revenue-critical journeys (login, checkout, core CRUD). ~95% coverage with a CI run in minutes, not an hour. Expect next: the 10% E2E is the flows that hit revenue if broken. Avoid: an inverted pyramid that is mostly slow E2E tests.

Q 214 LEVEL · Mid to Senior

How do you apply pairwise testing to features with many input parameters?

Parameter explosion: 5×4×3×4 = 240 combinations, infeasible to test all. Pairwise covers every value-pair across any two parameters, 240 -> ~20. NIST research: most multi-param defects are 1- or 2-way, not higher. Tools: AllPairs (classic), PICT (Microsoft), pict-node (TS-integrated). Generate matrix -> run as parameterised tests, data-driven loop. The easy mistake: **pairwise instead of BVA/EP**, **complementary**, **not replacement**.

```
// pict-node generates the pairwise matrix for a multi-parameter feature
import { pict } from 'pict-node';

const cases = await pict({
  model: [
    { key: 'browser', values: ['chrome', 'firefox', 'safari', 'edge', 'opera'] },
    { key: 'plan',    values: ['free', 'basic', 'pro', 'enterprise'] },
    { key: 'country', values: ['US', 'UK', 'IN'] },
    { key: 'payment', values: ['card', 'paypal', 'bank', 'crypto'] },
  ],
  // generates ~20 cases covering all pairs (vs 240 for full cartesian)
});

for (const c of cases) {
  test(`signup: ${c.browser}/${c.plan}/${c.country}/${c.payment}`, async ({ page }) => {
    await page.goto(`/signup?plan=${c.plan}&country=${c.country}`);
    await selectPaymentMethod(page, c.payment);
    await page.getByRole('button', { name: 'Complete signup' }).click();
    await expect(page.getByText('Welcome')).toBeVisible();
  });
...
```

Q 215 LEVEL · Senior

What is risk-based test design, and how do you decide where to invest test coverage?

Risk = probability of failure × impact of failure, allocate effort accordingly. Four-quadrant matrix: high/low impact × high/low probability. Inputs: incident history (strongest), code churn, business criticality, complexity. Quantify each input from real data: **git log**, **bug tracker**, **complexity tools**. They usually ask next: incident history is the strongest signal, past predicts future. Trap: rationalising boring areas as low-risk, evidence not vibes.

Q 216 LEVEL · Mid to Senior

What is scenario-based testing, and how does it differ from feature-based testing?

Feature: per-component tests; scenario: end-to-end user journeys. Scenarios catch integration bugs: features miss them. Derive from real journeys: **analytics**, **support tickets**, **research**, **sales calls**. Strategy: features for breadth, 5-10 scenarios for critical journeys. The next question: 5-10 scenarios, more becomes maintenance burden. Where this goes wrong: one without the other, miss edge cases OR integration bugs.

```
// FEATURE TEST: isolated, comprehensive for one capability
test.describe('search feature', () => {
  test('returns matching results', async ({ page }) => { /* ... */ });
  test('handles empty query', async ({ page }) => { /* ... */ });
  test('paginates beyond 20 results', async ({ page }) => { /* ... */ });
  test('filters by category', async ({ page }) => { /* ... */ });
});

// SCENARIO TEST: end-to-end journey across multiple features
test('new-customer journey: browse -> signup -> buy -> review', async ({ page, userBuilder }) => {
  // 1. Browse as anonymous
  await page.goto('/');
  await page.getByPlaceholder('Search').fill('headphones');
  await page.getByRole('button', { name: 'Search' }).click();
  await page.getByRole('link', { name: /Sony WH/ }).click();
  // 2. Sign up (auth feature integration)
  await page.getByRole('button', { name: 'Buy now' }).click();
  const user = await userBuilder.unsavedDetails();
  await page.getByLabel('Email').fill(user.email);
  await page.getByLabel('Password').fill(user.password);
...
```

QUICK QUIZ

1. What is the core idea of Equivalence Partitioning?
   A) Test every possible input value
   B) Group inputs that behave identically and test one value per group
   C) Test only boundary values
   D) Randomly sample inputs
2. Why does Boundary Value Analysis test min-1, min, and min+1?
   A) To test more values for thoroughness
   B) To satisfy code coverage
   C) Because the middle is never buggy
   D) Because off-by-one errors (> vs >=) live exactly at the boundary
3. What is the main benefit of a decision table beyond coverage?
   A) It makes business-rule coverage traceable and verifiable by product managers
   B) It runs tests faster
   C) It removes the need for assertions
   D) It avoids parameterised tests
4. Which state transition tests are often the most valuable?
   A) Valid transitions only
   B) Invalid transitions: they verify the app enforces business rules
   C) Terminal states only
   D) Transitions the UI never shows
5. What is a sound test-pyramid distribution for a Playwright framework?
   A) Mostly E2E tests for maximum confidence
   B) 50% E2E, 50% unit
   C) ~30% unit, 30% component, 30% API, 10% E2E (critical journeys only)
   D) Only E2E and unit, no API

Answers: 1:B 2:D 3:A 4:B 5:C

QA Fundamentals and Test Strategy

Q 217 LEVEL · Mid to Senior

How do you define a test strategy for a new product?

Strategy = what/how/who/when: **scope**, **risk**, **types**, **envs**, **criteria**, **tooling**. Risk matrix: rate every feature on probability x business impact. Effort proportional to risk: payment gets 50 tests, about page gets 2. Layer on test types, environment plan, pyramid, entry/exit criteria. Asked next: the risk matrix IS the budget justification. What loses the point: proposing to test everything equally.

Q 218 LEVEL · Junior to Mid

What is the difference between verification and validation in QA?

Verification: building it right (conforms to spec), reviews, unit tests. Validation: building the right thing (meets user needs), UAT, beta. Wrong-formula calculator passes verification, fails validation. Playwright tests are verification; UAT is validation. Follow-up: automated tests provide verification, not validation. The common miss: implying green tests prove the product is the right product.

Q 219 LEVEL · Mid

How do you measure and report test coverage to stakeholders?

Frame coverage as requirement, risk, and code coverage. Build a traceability matrix from requirement-ID tags on tests. Report shows requirements covered, failing, and with NO test. Present as a requirements dashboard, not a code-coverage percent. Expect next: the untested requirement is the most actionable item. Avoid: reporting a raw code-coverage number to product.

```
// Requirement-ID tags create the traceability matrix
test('user can log in with valid credentials', { tag: ['@REQ-101', '@smoke'] }, async ({ page }) => {
  // maps to REQ-101
});
test('invalid password shows error', { tag: ['@REQ-102', '@regression'] }, async ({ page }) => {
  // maps to REQ-102
});

// Generate a coverage report from the tags:
//   npx playwright test --reporter=json | node scripts/coverage-report.js
//   -> REQ-101: covered | REQ-102: covered | REQ-103: NOT COVERED
```

Q 220 LEVEL · Mid

What is shift-left testing and how do you implement it?

Move testing earlier so defects are found when cheapest to fix. Spans stages: **requirements**, **design**, **development**, **integration**, **release**. Highest-impact practice: write test cases from criteria before dev starts. Cost: ~0 pre-code vs ~100x in production. They usually ask next: prove it with a metric (e.g. ~40% fewer production defects). The easy mistake: reciting the buzzword with no concrete practice.

Q 221 LEVEL · Mid

How do you handle regression testing in an agile environment?

Three tiers: smoke per PR (~3min), targeted per sprint (~15min), full per release (~45min). Targeted tier runs the areas changed, by changed files or labels. Automate tier selection so developers never choose which tests run. Balances fast PR feedback with release confidence. The next question: tier chosen automatically from PR labels/changed files. Trap: one-size regression, always-full (slow) or always-smoke (thin).

```
# Tier 1: smoke on every PR (~3 min)
npx playwright test --grep @smoke --project=chromium

# Tier 2: targeted regression per sprint (~15 min)
npx playwright test --grep "@checkout|@payment" --project=chromium

# Tier 3: full regression on release candidates (~45 min)
npx playwright test --project=chromium --project=firefox --project=webkit

// Automated tier selection in CI
if (pr.labels.includes('release-candidate'))      runFullRegression();
else if (pr.labels.includes('payment-change'))    runTargeted(['@payment', '@checkout']);
else                                              runSmoke();
```

Q 222 LEVEL · Senior

How do you design a test environment strategy that supports parallel teams without contention?

Ephemeral per PR: **max isolation**, **max cost**, **best for highest-stakes teams**. Shared staging + tenant isolation: best balance, default for most teams. Dedicated per team: between ephemeral and shared, good for regulated orgs. Match strategy to team size and budget: transitions are themselves the work. Asked next: default recommendation is shared + tenant isolation. Where this goes wrong: one shared environment past team size, contention storms, scheduling pain.

```
# Environment strategy by team size / budget

# Strategy A: ephemeral per PR (Vercel/Render/K8s namespace)
# .github/workflows/preview.yml
deploy_preview:
  steps:
    - run: kubectl create namespace pr-${{ github.event.number }}
    - run: helm install acme ./chart --namespace pr-${{ github.event.number }} \
        --set imageTag=${{ github.sha }}
    - run: BASE_URL=https://pr-${{ github.event.number }}.staging.acme.com npx playwright test
    - if: always()
      run: helm uninstall acme --namespace pr-${{ github.event.number }}

# Strategy B: shared staging with tenant isolation (S9 Q9 pattern)
test.use({
  storageState: 'auth/per-worker.json',
});
test.beforeEach(async ({ request }, testInfo) => {
  await request.post('/api/test/tenant', { data: { id: `tenant-w${testInfo.workerIndex}-t${testInfo.testI...
});
...
```

Q 223 LEVEL · Senior

What quality metrics actually matter, and which are vanity metrics that mislead leadership?

- Outcome metrics: defect escape rate (top), MTTD, MTTR, change failure, stability.
- Vanity metrics: test count, coverage %, tests-passing, don't put on leadership dash.
- Discipline: outcomes to leadership, activity to your own team.
- Defect escape rate is the single most important metric, direct test effectiveness.
- Follow-up: coverage is a floor check, not a quality measure.
- What loses the point: leadership rewarding vanity metrics, incentivises noise over signal.

```
// Outcome dashboard for leadership
{
  defectEscapeRate:    bugsReachingProduction / totalReleasesThisQuarter,
  mttdMinutes:         avg(timeFromDefectStartToAlert),               // monitoring quality
  mttrMinutes:         avg(timeFromAlertToFix),                       // operational responsiveness
  changeFailureRate:   incidentCausingDeploys / totalDeploys,         // DORA
  testStabilityRate:   passesFirstTry / totalTestRuns,                // suite trustworthiness
}

// Activity dashboard for the team (NOT for leadership)
{
  testCount:            totalTestsInSuite,        // direction-of-travel only
  codeCoverage:         percentLinesHit,           // floor check, not quality measure
  passingTests:         passedThisRun,             // baseline, not achievement
  flakeRate:            retriedTests / totalRuns,  // tracks toward stability
}

// Anti-pattern: putting test count on the executive dashboard
// Result: someone optimises for the metric, mass-produces brittle tests
// Outcome: dashboard goes up, defect escape stays flat, leadership confused
```

Q 224 LEVEL · Senior

How do you scale QA expertise when you're the only SDET across multiple teams?

Solo SDET = leverage problem; can't write every test, so enable devs to. Move 1: reusable framework (fixtures, POs, helpers), tests in minutes not hours. Move 2: **pair-programming**, **patterns learned by doing**, **not documentation**. Move 3: code review on test PRs, teaching at scale via specific feedback. Move 4: documented patterns library, async teaching for what pairing doesn't. The common miss: **SDET as bottleneck**, **writes/reviews everything**, **role doesn't scale**.

```
// The reusable framework that enables developers to write tests in minutes
//
// /tests/support/                          <- the SDET builds and owns this
//   fixtures.ts                           authenticated pages, test data builders
//   page-objects/
//     CheckoutPage.ts, AccountPage.ts     consistent patterns per area
//   matchers.ts                            custom expect.extend matchers
//   builders/
//     userBuilder.ts, orderBuilder.ts     fluent test-data construction
//
// /tests/features/                         <- developers write tests here
//   checkout.spec.ts
//   account.spec.ts
//
// Developer's typical test, made trivial by the framework:
test('user can update billing address', async ({ accountPage, userBuilder }) => {
  const user = await userBuilder.withAddress().create();
  await accountPage.gotoAs(user);
  await accountPage.updateAddress({ city: 'Chennai' });
  await expect(accountPage.savedConfirmation).toBeVisible();
...
```

QUICK QUIZ

1. What anchors a risk-based test strategy?
   A) A risk matrix rating each feature on probability x business impact
   B) Testing every feature equally
   C) Code coverage targets
   D) The number of available testers
2. Which question does validation (not verification) answer?
   A) Are we building the product right?
   B) Are we building the right product?
   C) Is the code well-formatted?
   D) Do the unit tests pass?
3. How should test coverage be reported to product stakeholders?
   A) As a branch-coverage percentage
   B) As the total number of tests
   C) As a requirement traceability matrix (which requirements have tests)
   D) As lines of test code
4. What is the most impactful shift-left practice?
   A) Running more tests in production
   B) Increasing the timeout
   C) Adding retries in CI
   D) Writing test cases from acceptance criteria before development starts
5. How is regression best handled in an agile team?
   A) Run the full suite on every PR
   B) Tiered: smoke per PR, targeted per sprint, full per release, selected automatically
   C) Run only smoke before releases
   D) Manual regression each sprint

Answers: 1:A 2:B 3:C 4:D 5:B

Mobile Testing with Playwright

Q 225 LEVEL · Junior to Mid

How do you test mobile web applications with Playwright?

Device descriptors set viewport, user agent, DPR, touch (test.use or project). Scope upfront: mobile web emulation, NOT native app testing. Native apps -> Appium or Detox; Playwright owns responsive mobile web. Run mobile projects nightly; catches hidden-behind-nav and tiny-tap-target bugs. Expect next: native apps need Appium/Detox. Avoid: claiming Playwright tests native mobile apps.

```
import { test, expect, devices } from '@playwright/test';

test.use({ ...devices['iPhone 14'] });
test('mobile checkout', async ({ page }) => {
  await page.goto('/checkout');
  await page.getByRole('button', { name: 'Menu' }).tap();
  await expect(page.locator('.mobile-nav')).toBeVisible();
  await expect(page.locator('.desktop-nav')).toBeHidden();
});

// playwright.config.ts: mobile projects for nightly regression
// projects: [
//   { name: 'iPhone 14', use: { ...devices['iPhone 14'] } },
//   { name: 'Pixel 7',   use: { ...devices['Pixel 7'] } },
// ]
```

Q 226 LEVEL · Mid

How do you test touch gestures and mobile-specific interactions?

tap() and simple touchscreen taps are reliable. No built-in swipe or pinch: touchscreen only has tap(). Swipe = build Touch objects, then dispatchEvent touchstart/touchmove/touchend. The easy mistake: a tap plus mouse.move fires no touch events, so the swipe silently does nothing. Real devices via BrowserStack/Sauce for gesture-heavy apps (remote connection). Rule of thumb: emulation ~90%, real devices for the gesture-heavy 10%. They usually ask next: move to real devices for maps, galleries, drawing tools. Trap: claiming emulation handles every gesture flawlessly.

```
// tap: the mobile click; reliable
await page.getByRole('button', { name: 'Add to Cart' }).tap();
await page.touchscreen.tap(200, 300);

// Swipe: there is NO touchscreen.swipe(). You dispatch the whole
// touchstart/touchmove/touchend run yourself, which is why it is verbose.
// A tap plus a mouse.move fires no touch events at all and does nothing.
async function swipeLeft(page: Page, el: Locator): Promise<void> {
  const box = await el.boundingBox();
  if (!box) throw new Error('Element has no bounding box');
  const y = box.y + box.height / 2;
  const target = await el.elementHandle();

  const touchAt = (x: number) => page.evaluateHandle(
    ({ target, x, y }) => new Touch({ identifier: 1, target, clientX: x, clientY: y }),
    { target, x, y });

  const start = await touchAt(box.x + box.width * 0.8);
  const end = await touchAt(box.x + box.width * 0.2);
  await el.dispatchEvent('touchstart',
...
```

Q 227 LEVEL · Mid

How do you test responsive design breakpoints?

Parameterise across mobile/tablet/desktop viewports; assert layout at each. Add a visual snapshot per viewport to catch CSS regressions. Catches overflow, broken tablet nav, unreadable small-screen text. Run in nightly, not per-PR: too slow but high value. The next question: why not per-PR, too slow; nightly is the right home. Where this goes wrong: a single fixed viewport or manual visual checks.

```
const viewports = [
  { name: 'mobile',  width: 375,  height: 812 },
  { name: 'tablet',  width: 768,  height: 1024 },
  { name: 'desktop', width: 1440, height: 900 },
];
for (const v of viewports) {
  test(`nav layout at ${v.name} (${v.width}px)`, async ({ page }) => {
    await page.setViewportSize({ width: v.width, height: v.height });
    await page.goto('/');
    if (v.width < 768) {
      await expect(page.locator('.hamburger-menu')).toBeVisible();
      await expect(page.locator('.desktop-nav')).toBeHidden();
    } else {
      await expect(page.locator('.desktop-nav')).toBeVisible();
      await expect(page.locator('.hamburger-menu')).toBeHidden();
    }
    await expect(page).toHaveScreenshot(`nav-${v.name}.png`);
  });
}
```

Q 228 LEVEL · Senior

What does Playwright's device emulation actually emulate, and where does it differ from a real device?

Emulation sets: **viewport**, **UA**, **DPR**, **touch flag**, **isMobile from devices object**. Doesn't change: **rendering engine**, **CPU/RAM**, **real hardware**, **iOS-specific quirks**. Catches: **layout**, **responsive**, **UA-conditional paths**. Misses: **device browser bugs**, **real perf**, **iOS Safari divergences**. Discipline: emulation in CI; BrowserStack/real devices before release. What loses the point: emulation-as-equivalent ships iOS-specific bugs to production.

```
// What emulation sets (devices['iPhone 15'] approximation)
test.use({
  viewport:         { width: 393, height: 852 },
  deviceScaleFactor: 3,
  isMobile:          true,
  hasTouch:          true,
  userAgent:         'Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/...',
});

// What it does NOT set / cannot set:
//   - Real iOS Safari rendering (Playwright WebKit ≠ iOS Safari)
//   - CPU and memory constraints of a real phone
//   - Real touch hardware (events are synthesised)
//   - iOS-specific bugs in font rendering, video, scroll behaviour
//   - OS features: push notifications, biometrics, deep links

// Right CI strategy:
//   - Emulation in PR + main pipelines (fast, deterministic)
//   - BrowserStack/SauceLabs or real-device lab on release branches
//   - Smoke test on actual iPhone Safari before customer-visible deploy
```

Q 229 LEVEL · Mid

How do you simulate mobile network conditions (slow 3G, intermittent connectivity, offline)?

CDP Network.emulateNetworkConditions: throttle bandwidth + latency (Chromium only). Slow 3G profile: **500 kbps down**, **500 kbps up**, **400 ms latency**. context.setOffline(true): full network drop for PWA and error-state testing. Intermittent: toggle offline during test, or random-fail route handler. Discipline: test slow-network for any mobile-user feature. The common miss: testing on fast wifi only, ships features that break on real cellular.

```
// Slow 3G simulation via CDP
test('checkout works on slow 3G', async ({ page, context }) => {
  const cdp = await context.newCDPSession(page);
  await cdp.send('Network.emulateNetworkConditions', {
    offline: false,
    downloadThroughput: (500 * 1024) / 8,   // 500 kbps in bytes/s
    uploadThroughput:   (500 * 1024) / 8,
    latency: 400,                            // ms RTT
  });
  // Run the checkout flow under slow-3G conditions; assert loading states render
});

// Offline scenario
test('PWA serves cached content offline', async ({ page, context }) => {
  await page.goto('/dashboard');
  await context.setOffline(true);
  await page.reload();
  await expect(page.getByText('You're offline')).toBeVisible();
  await expect(page.getByTestId('cached-content')).toBeVisible();
  await context.setOffline(false);
...
```

Q 230 LEVEL · Mid

How do you handle mobile-specific browser APIs: geolocation, permissions, device orientation?

context.setGeolocation({ latitude, longitude }) for location-aware features. context.grantPermissions for geolocation, notifications, camera, clipboard. Orientation via page.setViewportSize: swap dimensions to simulate rotation. Set permissions per test, not globally: prompt-rejection paths need coverage. Asked next: rotation handler tested via setViewportSize, layout/resize events fire. Avoid: global granted permissions, prompt-rejection path never tested.

```
// Geolocation + permission for store locator
test.use({
  geolocation: { latitude: 19.0760, longitude: 72.8777 },   // Mumbai
  permissions: ['geolocation'],
});

test('store locator shows nearest store', async ({ page }) => {
  await page.goto('/stores');
  await expect(page.getByTestId('nearest-store')).toContainText('Mumbai');
});

// Test the permission-denied path explicitly
test('app handles location denial gracefully', async ({ page, context }) => {
  await context.clearPermissions();   // no permissions granted
  await page.goto('/stores');
  await page.getByRole('button', { name: 'Use my location' }).click();
  await expect(page.getByText('Allow location access in settings')).toBeVisible();
});

// Rotation via viewport swap
...
```

Q 231 LEVEL · Senior

How do you test a Progressive Web App's service worker, offline mode, and install flow?

SW registration: poll navigator.serviceWorker.ready until 'activated'. Offline: setOffline(true) + assert cached pages render from SW cache. Cache management: **test the update path**, **deploy serves new version**, **not stale**. Install prompt: can't trigger natively, but test the handler via page.evaluate. Discipline: cache-update path is the PWA failure most likely to ship. The easy mistake: only testing the happy install path, stale-cache regressions ship.

```
test('service worker registers and activates', async ({ page }) => {
  await page.goto('/');
  await page.waitForFunction(async () => {
    const reg = await navigator.serviceWorker.ready;
    return reg.active?.state === 'activated';
  });
});

test('app works offline', async ({ page, context }) => {
  await page.goto('/');
  await page.waitForFunction(() => navigator.serviceWorker.ready);
  await context.setOffline(true);
  await page.reload();
  await expect(page.getByRole('heading', { name: 'Welcome' })).toBeVisible();
});

test('cache-update path serves new version after deploy', async ({ page }) => {
  await page.goto('/');
  await page.evaluate(async () => {
    const reg = await navigator.serviceWorker.ready;
...
```

Q 232 LEVEL · Mid to Senior

Where does Playwright's mobile-web scope end, and when do you reach for Appium or native tools?

Playwright = mobile web + web-view portions of hybrid apps. Appium = native apps + native-shell portions of hybrid apps. Detox/XCUITest/Espresso = deep native, when Appium's overhead matters. Three artifact types: mobile web, hybrid, native, each gets the right tool. Rule: match tool to artifact, not team's framework preference. Trap: forcing Playwright into native space via wrappers, worst of both.

```
// Mobile WEB: Playwright (responsive site, PWA, hybrid web-view)
test.use({ ...devices['iPhone 15 Pro'] });
test('mobile checkout flow', async ({ page }) => {
  await page.goto('https://m.acme.com/checkout');
  // ... web test, same patterns as desktop with mobile viewport
});

// NATIVE APP: Appium (iOS Simulator / Android Emulator)
// (different tooling, different test runner: NOT Playwright)
//
// const driver = await remote({
//   capabilities: {
//     platformName: 'iOS', 'appium:deviceName': 'iPhone 15', 'appium:app': '/path/Acme.app'
//   },
// });
// await driver.$('~Sign In').click();

// HYBRID APP: Playwright for the web-view portion + Appium for native chrome
//   Two test suites, shared API contracts, different tools for what each does best.
//
...
```

QUICK QUIZ

1. What kind of mobile testing does Playwright provide?
   A) Browser-based mobile web emulation (viewport, UA, DPR, touch)
   B) Native iOS and Android app testing
   C) Real-device cloud testing only
   D) Performance profiling of native apps
2. What is the honest limitation of touch-gesture testing in Playwright?
   A) tap() does not work
   B) Touch is unsupported entirely
   C) Complex gestures (swipe, pinch) via dispatchEvent are verbose and fragile
   D) Gestures only work on desktop
3. Where in CI should parameterised responsive-breakpoint tests run?
   A) On every PR
   B) They should not be automated
   C) Only on release candidates
   D) In the nightly build: too slow for every PR but high value
4. Roughly what share of mobile web testing does emulation cover well?
   A) About 50%: real devices needed for the rest
   B) 100%, real devices are never needed
   C) About 90%: real devices needed for the gesture-heavy 10%
   D) About 10%: mostly real devices
5. Which mobile-specific bugs does device emulation most commonly surface?
   A) Database deadlocks and API timeouts
   B) Elements hidden behind the mobile nav and touch targets too small to tap
   C) Memory leaks in native code
   D) Server-side rendering errors

Answers: 1:A 2:C 3:D 4:C 5:B

Microservices and API Architecture

Q 233 LEVEL · Mid to Senior

How do you test microservices with Playwright?

Three levels: contract (per PR), integration (on change), E2E (release). Contract tests validate the API shape, not the implementation. Contract tests are the highest-value investment, fail fast on breaks. Pact for consumer-driven contracts in complex environments. Follow-up: the per-service contract test catches a break first. Where this goes wrong: testing everything through the UI/E2E.

```
// Contract test: validates a service in isolation
test('User Service: GET /users/:id contract', async ({ request }) => {
  const res = await request.get(`${process.env.USER_SERVICE_URL}/users/1`);
  await expect(res).toBeOK();
  expect(await res.json()).toMatchObject({
    id: expect.any(Number),
    email: expect.stringMatching(/@/),
    role: expect.stringMatching(/^(admin|user|viewer)$/),
  });
});

// Integration test: validates a service interaction
test('Order Service references User Service', async ({ request }) => {
  const { id: userId } = await (await request.post(`${process.env.USER_SERVICE_URL}/users`, { data: { ema...
  const order = await request.post(`${process.env.ORDER_SERVICE_URL}/orders`, { data: { userId, items: [{...
  expect(order.status()).toBe(201);
});
```

Q 234 LEVEL · Mid

How do you test the API gateway and authentication middleware?

Gateway tests cover routing, auth, rate limiting, CORS, via request fixture. Framing: security tests disguised as functional tests. 401 verifies auth applied; 429 verifies abuse protection; CORS for SPAs. Misconfigured CORS causes silent, hard-to-diagnose browser failures. Expect next: test the gateway separately, it's the single enforcement point. What loses the point: treating gateway tests as plain routing checks.

```
test('unauthenticated request returns 401', async ({ request }) => {
  const res = await request.get('/api/protected-resource');
  expect(res.status()).toBe(401);
});

test('rate limiting returns 429 past the threshold', async ({ request }) => {
  const responses = await Promise.all(Array.from({ length: 101 }, () => request.get('/api/public-endpoint...
  expect(responses.filter(r => r.status() === 429).length).toBeGreaterThan(0);
});

test('CORS header present for an allowed origin', async ({ request }) => {
  const res = await request.get('/api/data', { headers: { Origin: 'https://allowed-origin.com' } });
  expect(res.headers()['access-control-allow-origin']).toBe('https://allowed-origin.com');
});
```

Q 235 LEVEL · Mid to Senior

How do you test event-driven microservices with message queues?

Verify events are published and consumers process them. Pattern: trigger the synchronous action, poll for the async effect. expect.poll with backoff (500ms/1s/2s/5s) to the expected value. A too-long wait should fail loudly: a slow pipeline is a defect. They usually ask next: **validates the whole chain**, **publish**, **queue**, **consume**, **persist**. The common miss: a fixed sleep before checking the downstream effect.

```
test('order placement triggers inventory update', async ({ request }) => {
  const initial = (await (await request.get('/api/inventory/PROD-001')).json()).quantity;

  await request.post('/api/orders', { data: { items: [{ productId: 'PROD-001', quantity: 2 }] } });

  await expect.poll(async () => {
    return (await (await request.get('/api/inventory/PROD-001')).json()).quantity;
  }, {
    message: 'Inventory should decrease by 2 after the order',
    timeout: 15_000,
    intervals: [500, 1000, 2000, 5000],
  }).toBe(initial - 2);
});
```

Q 236 LEVEL · Senior

How do you test gRPC services from Playwright, given Playwright is HTTP-focused?

Playwright is HTTP/HTTPS only: no native gRPC support. Option 1: gRPC-Web gateway, Playwright hits it as HTTP/2. Option 2: @grpc/grpc-js client in a fixture for pure gRPC. Option 3: **hybrid**, **Playwright drives UI**, **gRPC client verifies state**. Rule: match the test protocol to how production callers actually use it. Avoid: testing gRPC-Web when production traffic is pure gRPC, misses protocol bugs.

```
// Approach 1: gRPC-Web gateway, Playwright as HTTP client
test('order service via gRPC-Web', async ({ request }) => {
  const res = await request.post('/grpc.OrderService/CreateOrder', {
    headers: { 'content-type': 'application/grpc-web+proto' },
    data: encodeOrderRequest({ product: 'X', qty: 1 }),
  });
  expect(res.status()).toBe(200);
});

// Approach 2: dedicated gRPC client inside a Playwright fixture
import * as grpc from '@grpc/grpc-js';
import { OrderServiceClient } from './generated/order_grpc_pb';

export const test = base.extend<{ orderClient: OrderServiceClient }>({
  orderClient: async ({}, use) => {
    const client = new OrderServiceClient('order-service:50051', grpc.credentials.createInsecure());
    await use(client);
    client.close();
  },
});
...
```

Q 237 LEVEL · Mid to Senior

How do you use distributed tracing to diagnose test failures across microservices?

Trace ID via traceparent header propagates across all services. Test emits a generated trace ID; failure surfaces it in annotations + report. Engineer queries Jaeger/Honeycomb by trace ID: full call tree in one query. 10x faster diagnosis on multi-service failures vs grepping logs. Discipline: **generate at test level**, **log on failure**, **surface in HTML report**. The easy mistake: logging trace IDs but not linking where engineers look during investigation.

```
// Generate a trace ID per test, propagate via traceparent
import { randomBytes } from 'crypto';

function newTraceparent(): string {
  const traceId = randomBytes(16).toString('hex');
  const spanId  = randomBytes(8).toString('hex');
  return `00-${traceId}-${spanId}-01`;   // W3C format
}

export const test = base.extend<{ tracedRequest: APIRequestContext; traceId: string }>({
  traceId: async ({}, use) => { await use(newTraceparent()); },
  tracedRequest: async ({ traceId, playwright }, use) => {
    const ctx = await playwright.request.newContext({
      extraHTTPHeaders: { traceparent: traceId },
    });
    await use(ctx);
  },
});

test('cross-service order flow', async ({ tracedRequest, traceId }, testInfo) => {
...
```

Q 238 LEVEL · Senior

How do you test resilience patterns: circuit breakers, retries, and timeouts?

Test each pattern in isolation: **breaker**, **retry**, **timeout**, **separate scenarios**. Circuit: N failures -> breaker opens -> fast-fail -> cooldown -> recovery. Retries: count calls + measure delays, verify exponential backoff numbers. Timeouts: slow downstream + assert caller aborts at the right time. Discipline: specific numeric behaviour, not 'the system is resilient'. Trap: vague resilience claims, production reveals breaker takes 10 min to recover.

```
// Circuit breaker test: specific behaviour assertions
test('breaker opens after 5 failures, fast-fails until 30s cooldown', async ({ page }) => {
  let calls = 0;
  await page.route('**/api/downstream', route => {
    calls++;
    return route.fulfill({ status: 503 });
  });

  // 5 failing calls open the breaker
  for (let i = 0; i < 5; i++) {
    await page.evaluate(() => fetch('/api/downstream'));
  }
  expect(calls).toBe(5);

  // 6th call fast-fails without hitting downstream
  const start = Date.now();
  const status = await page.evaluate(async () => (await fetch('/api/downstream')).status);
  const elapsed = Date.now() - start;
  expect(elapsed).toBeLessThan(100);     // fast-fail
  expect(status).toBe(503);
...
```

Q 239 LEVEL · Senior

What is chaos engineering applied to test environments, and how do you implement it?

Deliberate failure injection: **pod kills**, **network drops**, **latency injection**. Tools: Chaos Mesh, Litmus (K8s); Toxiproxy (network); Gremlin (commercial). Fixture-injected chaos: **bring up fault**, **run test**, **remove fault**. Tiering: release-gate + nightly only, NOT every PR (too slow/brittle). Resilience claims need evidence: chaos engineering produces it. Where this goes wrong: chaos tests on every PR, brittle, slow, eventually abandoned.

```
// Fixture that injects chaos via Chaos Mesh (Kubernetes CRD API)
export const test = base.extend<{ chaos: ChaosFault }>({
  chaos: async ({}, use) => {
    const fault: ChaosFault = {
      kind: 'PodChaos',
      target: 'app=payment-service',
      action: 'pod-kill',
    };
    await applyChaosMesh(fault);          // CRD applied
    await use(fault);
    await removeChaosMesh(fault);          // teardown
  },
});

test('order flow degrades gracefully when payment-service is down', async ({ page, chaos, request }) => {
  // chaos fixture already killed payment-service
  await page.goto('/checkout');
  await page.getByRole('button', { name: 'Place Order' }).click();

  // System degrades gracefully:
...
```

Q 240 LEVEL · Senior

How do you tier contract testing in CI: fast consumer-driven vs slow end-to-end verification?

Tier 1: consumer contracts in PR pipelines (seconds), unit-test-fast. Tier 2: provider verification on provider PRs (minutes), backward compat. Tier 3: real-service e2e on integration branches, race/state-dependent bugs. Rule: tier-1 gates every PR; tier-3 gates merge to main. The next question: Pact = consumer-driven contracts shared via broker. What loses the point: only tier-3 e2e, slow, expensive, bugs found too late to fix cheaply.

```
# CI tiering for contract tests

# Tier 1: consumer PR pipeline (seconds)
on: pull_request
jobs:
  consumer-contracts:
    runs-on: ubuntu-latest
    steps:
      - run: npm test -- --grep '@contract'   # Pact-based unit tests

# Tier 2: provider PR (minutes), verify against published consumer contracts
on: pull_request                              # in the provider's repo
jobs:
  provider-verification:
    runs-on: ubuntu-latest
    steps:
      - run: |
          pact-broker can-i-deploy --pacticipant=auth-service \\
            --version=$GITHUB_SHA --to-environment=production

...
```

QUICK QUIZ

1. Which microservices test level catches a breaking change first?
   A) E2E tests through the UI
   B) Manual exploratory testing
   C) Per-service contract tests: before integration tests even run
   D) Load tests
2. Why are API-gateway tests best thought of as security tests?
   A) They verify auth enforcement, rate limiting and CORS, the system's perimeter
   B) They run faster than other tests
   C) They only check routing
   D) They replace contract tests
3. What pattern tests an event-driven flow through a message queue?
   A) A fixed sleep then a single check
   B) Trigger the action, then expect.poll for the async downstream effect
   C) Assert immediately after the trigger
   D) Mock the entire queue
4. What does Pact provide in a microservices environment?
   A) Load testing
   B) A message queue
   C) Browser automation
   D) Consumer-driven contract testing across services
5. How should contract tests validate a response?
   A) Assert exact values for every field
   B) Compare against a screenshot
   C) Only check the status code
   D) Validate the contract, types, formats, allowed values, not the implementation

Answers: 1:C 2:A 3:B 4:D 5:D

Security and Performance Testing

Q 241 LEVEL · Mid

How do you implement security testing with Playwright?

Functional tests with a security lens, using the standard API. XSS: payload not executed and shown as escaped text. Auth bypass: **clear cookies/storage**, **hit protected route**, **expect redirect**. Sensitive data: assert no password/passwordHash in API responses. Asked next: caught a debug endpoint leaking the hashed password pre-prod. The easy mistake: implying it replaces a dedicated pen test or security audit.

```
test('user input is sanitised: XSS prevented', async ({ page }) => {
  const payload = '<script>window.__XSS__ = true;</script>';
  await page.goto('/profile/edit');
  await page.getByLabel('Display Name').fill(payload);
  await page.getByRole('button', { name: 'Save' }).click();
  await page.goto('/profile');
  expect(await page.evaluate(() => (window as any).__XSS__)).toBeUndefined();
  expect(await page.locator('#display-name').textContent()).toContain('<script>');
});

test('password not returned in user API response', async ({ request }) => {
  const user = await (await request.get('/api/users/1')).json();
  expect(user).not.toHaveProperty('password');
  expect(user).not.toHaveProperty('passwordHash');
});
```

Q 242 LEVEL · Mid

How do you measure performance metrics with Playwright?

Capture Web Vitals / Navigation Timing (LCP, TTFB, DOMContentLoaded). Framing: performance-regression tests, NOT load tests. Load testing is k6/JMeter territory: a different question. Set thresholds from the SLA; fail on a regression like a heavy image. Follow-up: load test with k6/JMeter; track trends for gradual degradation. Trap: conflating single-load budgets with load testing.

```
test('dashboard meets the performance budget', async ({ page }) => {
  await page.goto('/dashboard');
  const m = await page.evaluate(() => {
    const nav = performance.getEntriesByType('navigation')[0] as PerformanceNavigationTiming;
    return {
      domContentLoaded: nav.domContentLoadedEventEnd - nav.startTime,
      ttfb: nav.responseStart - nav.requestStart,
      loadComplete: nav.loadEventEnd - nav.startTime,
    };
  });
  expect(m.ttfb).toBeLessThan(500);              // SLA-derived budget
  expect(m.domContentLoaded).toBeLessThan(2000);
  expect(m.loadComplete).toBeLessThan(5000);
});
```

Q 243 LEVEL · Mid to Senior

How do you integrate OWASP ZAP with Playwright for security scanning?

Configure ZAP as Playwright's proxy; tests run normally. ZAP silently records every endpoint/parameter, then active-scans them. Scans for SQLi, XSS, missing headers, insecure cookie flags. Zero test-code change: functional tests map the attack surface. Expect next: **coverage scales**, **more functional tests**, **more scanned endpoints**. Where this goes wrong: treating ZAP as a separate manual effort detached from the suite.

```
// playwright.config.ts: route traffic through ZAP
export default defineConfig({
  use: { proxy: { server: 'http://localhost:8080' } }, // ZAP proxy
});

// Tests run normally; ZAP records every endpoint + parameter.
// After the run, trigger an active scan via the ZAP API:
//   curl -X POST "http://localhost:8080/JSON/ascan/action/scan/" \
//     -d "url=${BASE_URL}&recurse=true&inScopeOnly=true"
//
// ZAP reports: SQL injection, XSS, missing security headers,
// insecure cookie flags: across everything the tests exercised.
```

Q 244 LEVEL · Senior

How do you systematically test authorisation boundaries across user roles?

Authorisation bugs: wrong user could perform an action they shouldn't. Matrix approach: roles × actions × expected outcomes, cover all cells. Per-role storageState files: **admin**, **user**, **viewer**, **anonymous**. Parameterised loop runs every endpoint as every role with expected outcome. Test negative path explicitly: viewer canNOT delete, not just admin can. What loses the point: positive-only testing, broken auth that allows everything still passes.

```
// Parameterised authorisation matrix
const matrix = [
  { role: 'anonymous', file: undefined,          delete: 401, view: 401 },
  { role: 'viewer',    file: 'auth/viewer.json', delete: 403, view: 200 },
  { role: 'user',      file: 'auth/user.json',   delete: 403, view: 200 },
  { role: 'admin',     file: 'auth/admin.json',  delete: 204, view: 200 },
];

for (const { role, file, delete: del, view } of matrix) {
  test.describe(`role: ${role}`, () => {
    test.use({ storageState: file });
    test(`DELETE /api/users/X returns ${del}`, async ({ request }) => {
      const res = await request.delete('/api/users/123');
      expect(res.status()).toBe(del);              // covers BOTH positive + negative
    });
    test(`GET /api/users returns ${view}`, async ({ request }) => {
      const res = await request.get('/api/users');
      expect(res.status()).toBe(view);
    });
  });
...
```

Q 245 LEVEL · Mid to Senior

How do you test for XSS and injection vulnerabilities with input fuzzing?

XSS/injection live in any input flowing to rendered output or executed query. Payload catalogue: **OWASP XSS cheat sheet**, **sqlmap**, **command/NoSQL patterns**. XSS assertion: payload as text + no new script tags created. SQL injection assertion: normal error/400, not 500 with stack trace. Generate fuzzing from OpenAPI schema: every string field is a target. The common miss: one fuzz test per endpoint, vulnerabilities live where you didn't test.

```
// Curated payload catalogue (subset of OWASP cheat sheet)
const xssPayloads = [
  '<script>alert(1)</script>',
  '"><script>alert(1)</script>',
  '<img src=x onerror=alert(1)>',
  'javascript:alert(1)',
  '<svg/onload=alert(1)>',
  '<iframe src="javascript:alert(1)">',
  '<body onload=alert(1)>',
];

for (const payload of xssPayloads) {
  test(`comment field escapes XSS: ${payload.slice(0, 30)}`, async ({ page }) => {
    await page.goto('/posts/1');
    await page.getByLabel('Comment').fill(payload);
    await page.getByRole('button', { name: 'Post' }).click();

    // 1. Payload appears as text, not executed
    await expect(page.getByTestId('comment-body').last()).toHaveText(payload);
    // 2. No new <script> tags created
...
```

Q 246 LEVEL · Mid

How do you verify security headers and cookie flags are correctly set?

Security headers are deployment-config concerns; pass dev, fail in prod. Headers to verify: CSP (specific directives!), HSTS, X-Frame-Options, nosniff, Referrer-Policy. Cookies: **HttpOnly**, **Secure**, **SameSite=Lax or Strict**. Run against every environment: catches LB/proxy stripping drift. They usually ask next: permissive CSP (default-src *) is technically present, provides no protection. Avoid: assert header present without verifying restrictive value.

```
test('production security headers and cookies', async ({ request }) => {
  const res = await request.get(`${process.env.BASE_URL}/`);
  const headers = res.headers();

  // CSP: specific directives, not just present
  const csp = headers['content-security-policy'] ?? '';
  expect(csp).toContain("default-src 'self'");
  expect(csp).not.toContain('unsafe-inline');           // catches permissive CSP
  expect(csp).not.toContain('*');

  // HSTS
  expect(headers['strict-transport-security']).toMatch(/max-age=\d{7,}.*includeSubDomains/);

  // Anti-clickjacking + MIME sniff
  expect(headers['x-frame-options']).toMatch(/DENY|SAMEORIGIN/);
  expect(headers['x-content-type-options']).toBe('nosniff');

  // Cookies (auth flow)
  const setCookie = res.headersArray().find(h => h.name.toLowerCase() === 'set-cookie')?.value ?? '';
  expect(setCookie).toMatch(/HttpOnly/i);
...
```

Q 247 LEVEL · Mid to Senior

How do you measure Core Web Vitals (LCP, INP, CLS) from Playwright tests?

LCP <2.5s, INP <200ms (replaced FID in 2024), CLS <0.1, Google's targets. Measure via PerformanceObserver API in page.evaluate. Throttle network (slow 3G) and CPU for representative environment. Dev-machine numbers are misleading; real users have slower devices/networks. The next question: PerformanceObserver = per-test threshold; Lighthouse = fuller report. The easy mistake: Web Vitals on wired dev box, numbers don't reflect real users.

```
test('home page meets Web Vitals on slow 3G', async ({ page, context }) => {
  // Realistic environment
  const cdp = await context.newCDPSession(page);
  await cdp.send('Network.emulateNetworkConditions', {
    offline: false, latency: 400, downloadThroughput: 500 * 1024 / 8, uploadThroughput: 500 * 1024 / 8,
  });
  await cdp.send('Emulation.setCPUThrottlingRate', { rate: 4 });   // 4x slower CPU

  await page.goto('/');

  const vitals = await page.evaluate(() => new Promise<{ lcp: number; cls: number }>(resolve => {
    let lcp = 0, cls = 0;
    new PerformanceObserver(list => {
      for (const entry of list.getEntries()) {
        if (entry.entryType === 'largest-contentful-paint') lcp = entry.startTime;
        if (entry.entryType === 'layout-shift' && !(entry as any).hadRecentInput) cls += (entry as any).v...
      }
    }).observe({ type: 'largest-contentful-paint', buffered: true });
    setTimeout(() => resolve({ lcp, cls }), 5000);
  }));
...
```

Q 248 LEVEL · Senior

How do you enforce performance budgets in CI without producing constant flake?

Budgets: fixed thresholds that fail CI if exceeded, prevent regression. Soft upper bounds: budget for the variance, not the happy path. Median across N runs (3-5): single bad run shouldn't fail the build. Trend tracking catches chronic drift that absolute thresholds miss. Both needed: budgets for acute regression, trends for chronic. Trap: tight budgets without variance absorption, flake -> team disables them.

```
test('LCP performance budget', async ({ page }) => {
  const samples: number[] = [];
  // Median across 5 runs absorbs CI variance
  for (let i = 0; i < 5; i++) {
    await page.goto('/', { waitUntil: 'networkidle' });
    const lcp = await page.evaluate(() => new Promise<number>(resolve => {
      new PerformanceObserver(list => {
        const entries = list.getEntries();
        resolve(entries[entries.length - 1]!.startTime);
      }).observe({ type: 'largest-contentful-paint', buffered: true });
    }));
    samples.push(lcp);
  }

  samples.sort((a, b) => a - b);
  const median = samples[Math.floor(samples.length / 2)]!;

  // Soft upper bound: target is 1500ms typical, budget is 2500ms (variance absorbed)
  expect(median).toBeLessThan(2500);

...
```

Q 249 LEVEL · Senior

Where does Playwright performance measurement end, and when do you reach for k6, JMeter, or Artillery?

Playwright = single-user perf under realistic conditions (Web Vitals, page load). Load tools = system behaviour under concurrent load (degradation, breaking point). k6: **JS-scriptable**, **modern DX**, **Grafana dashboards**, **best for JS-stack teams**. JMeter: GUI-driven, plugin ecosystem, best for enterprise QA with existing skills. Artillery: YAML or JS, lightweight, best for fast CI smoke load tests. Pitfall: claiming Playwright does load testing, loses senior credibility instantly.

```
// Playwright: single-user perf budget (this is its lane)
test('checkout LCP under 2500ms', async ({ page }) => {
  await page.goto('/checkout');
  const lcp = await page.evaluate(() => new Promise<number>(res => {
    new PerformanceObserver(list => {
      const e = list.getEntries(); res(e[e.length - 1]!.startTime);
    }).observe({ type: 'largest-contentful-paint', buffered: true });
  }));
  expect(lcp).toBeLessThan(2500);
});

# k6, concurrent load testing (different lane, different tool)
# k6/checkout-load.js
# import http from 'k6/http';
# export const options = {
#   stages: [
#     { duration: '2m', target: 100 },
#     { duration: '5m', target: 500 },
#     { duration: '2m', target: 0 },
#   ],
...
```

QUICK QUIZ

1. How are security tests in Playwright best characterised?
   A) A replacement for penetration testing
   B) Load tests
   C) Functional tests with a security lens (XSS, auth bypass, data exposure)
   D) Manual-only checks
2. What does a 'password not in the API response' test catch?
   A) Slow responses
   B) Sensitive-data exposure: e.g. a debug endpoint leaking the password hash
   C) CORS misconfiguration
   D) Rate-limit failures
3. What kind of performance testing does Playwright provide?
   A) Performance-regression tests against a single-page-load budget
   B) Load testing under thousands of concurrent users
   C) Stress testing the database
   D) Network throughput benchmarking
4. Which tool is appropriate for load testing, not Playwright?
   A) ESLint
   B) Pact
   C) Allure
   D) k6 or JMeter
5. Why is integrating OWASP ZAP as a proxy elegant?
   A) It rewrites the tests automatically
   B) It removes the need for assertions
   C) Zero test-code change: functional tests map the attack surface ZAP then scans
   D) It only scans the login page

Answers: 1:C 2:B 3:A 4:D 5:C

Behavioural and Soft Skills

Q 250 LEVEL · Mid

Tell me about a time you improved test suite performance significantly.

Use STAR explicitly: **Situation**, **Task**, **Action**, **Result**. Profile first (JSON reporter): diagnose before acting. Action: **API setup over UI**, **fullyParallel**, **sharding**. Quantify: 45 min -> 8 min (82%), deploys twice-daily -> on-demand. Asked next: how you found the slow tests, profiling. Where this goes wrong: 'I made the tests faster' with no method or number.

Q 251 LEVEL · Mid

How do you handle disagreements with developers about test coverage?

Reframe as a risk conversation, not an opinion debate. Use business impact: ~2% fail x $150 AOV ≈ $3,000/hr vs 2 hrs of tests. Stated as a number, the decision becomes obvious and agreement is fast. Principle: coverage is risk management, not perfectionism. Follow-up: if the math favours the developer, they're right, it cuts both ways. What loses the point: escalating into a who's-right standoff.

Q 252 LEVEL · Mid

How do you onboard a new engineer to a Playwright framework?

Progressive complexity: run -> read -> pair -> solo -> contribute. Read existing tests before writing so the first test matches house style. CONTRIBUTING.md (locator priority, fixtures, review checklist) for self-service. Measure with time-to-first-merged-test (e.g. ~4 days). Expect next: how you know it works, the time-to-first-merged-test metric. The common miss: ad-hoc 'ask anyone' onboarding that doesn't scale.

Q 253 LEVEL · Junior to Mid

How do you communicate test results to non-technical stakeholders?

Translate results into business impact and confidence, not metrics. Never send a raw report link: send a one-paragraph summary. Include pass rate, failure criticality, and a confidence-to-deploy verdict. Explicitly confirm the high-stakes flows (payment, auth, checkout) passed. They usually ask next: they need a deploy/no-deploy call, not test internals. Avoid: dumping a report URL or technical detail on a non-technical audience.

Q 254 LEVEL · Mid

How do you prioritise test automation work in a sprint?

Treat automation as a product with a ranked backlog. P1 blocking failures, P2 new-feature coverage, P3 high-risk gaps, P4 flaky fixes, P5 framework. Allocate a fixed, non-negotiable ~20% of capacity each sprint. Never let coverage debt accumulate: it becomes permanent manual regression. The next question: stop debt piling up via fixed allocation + visible metric in review. The easy mistake: letting automation be the leftover that gets cut when sprints tighten.

Q 255 LEVEL · Mid to Senior

Tell me about a critical bug that escaped your test suite. What did you do, and what changed afterwards?

Pick a real bug; walk diagnosis honestly including what was missed. Identify the category of gap (input space, environment, assumption), not 'need more tests'. Close with the systemic change that prevents the whole category. 'Escapes as data, not shame': information about where strategy has gaps. Asked next: name the specific check now in place for this category. Trap: defensive answers, blame-shifting, or vague 'we added more tests'.

Q 256 LEVEL · Senior

How have you convinced leadership to invest in test infrastructure when they wanted feature delivery instead?

Quantify the current cost: **engineer-hours**, **escapes**, **customer impact in dollars**. Scope the proposal: **specific deliverables**, **concrete timeline**, **measurable outcomes**. Tie to a metric leadership already cares about (velocity, revenue, customer satisfaction). Engineering arguments fail; ROI arguments win: same axis as other investments. Follow-up: revisit after the next incident, cost just became concrete. Where this goes wrong: arguing from principle or technical detail, wrong conversational frame.

Q 257 LEVEL · Senior

How do you handle a critically important test that's flaky and can't be quickly fixed?

Containment: **quarantine queue**, **allow-fail in PR**, **must-pass nightly**. Investigation: **treat as a bug**, **file the ticket**, **assign an owner**, **track to resolution**. Communication: **team knows what's quarantined**, **why**, **who owns it**, **target date**. Quarantine = temporary state with owner + deadline, never a graveyard. Expect next: never delete without a replacement at a different layer. What loses the point: extremes, delete (lose coverage) or keep blocking (lose team trust).

QUICK QUIZ

1. What structure should a 'tell me about a time' answer follow?
   A) STAR, Situation, Task, Action, Result, with a quantified result
   B) A list of technologies used
   C) A chronological diary
   D) Only the result
2. How do you best resolve a coverage disagreement with a developer?
   A) Pull rank as the QA lead
   B) Escalate to the manager immediately
   C) Reframe it as a risk-and-cost conversation backed by numbers
   D) Add the tests without telling them
3. What metric measures onboarding success for a test framework?
   A) Lines of test code written
   B) Number of meetings attended
   C) Time-to-first-merged-test
   D) Total tests in the suite
4. How should you report test results to a non-technical stakeholder?
   A) Send a link to the HTML report
   B) A one-paragraph summary with pass rate, failure criticality, and a confidence-to-deploy verdict
   C) A full stack trace of failures
   D) The raw JSON results
5. What is the key discipline in prioritising automation work?
   A) Doing it only when there is spare time
   B) Letting developers decide each sprint
   C) Always prioritising framework refactors first
   D) A fixed, non-negotiable capacity allocation so coverage debt never accumulates

Answers: 1:A 2:C 3:C 4:B 5:D

Advanced TypeScript for SDETs

Q 258 LEVEL · Mid to Senior

How do you use conditional and template literal types in a Playwright framework?

- Conditional types: type-level logic; infer extracts a type (e.g. Awaited<T>).
- Template literal types pin endpoint shapes: `/api/users/${number}`.
- Catches /api/users/abc at compile time instead of as a runtime 404.
- Apply to every typed API client: enforces the contract for free.
- They usually ask next: prevents the wrong-path 404, fast diagnosis vs runtime hunt.
- The common miss: describing conditional types in the abstract with no application.

```
type APIResponse<T extends 'GET' | 'POST' | 'DELETE'> =
  T extends 'GET'    ? { data: unknown; status: number } :
  T extends 'POST'   ? { data: unknown; id: number; status: number } :
  T extends 'DELETE' ? { success: boolean; status: number } : never;

type Awaited<T> = T extends Promise<infer U> ? U : T;
type LoginResult = Awaited<ReturnType<typeof loginPage.login>>;

// Template literal types pin endpoint shapes at compile time
type UserEndpoint = `/api/users/${number}`;
function fetchUser(endpoint: UserEndpoint) { /* ... */ }
fetchUser('/api/users/42');   // Valid
// fetchUser('/api/users/abc'); // TS error: string not assignable to number
```

Q 259 LEVEL · Mid to Senior

How do you use mapped types for test data generation?

- Mapped types transform every property of a type into a derived shape.
- DataGenerator<T> = one generator function per field of T, fully typed.
- Self-maintaining: add a field to the interface, the compiler names the gap.
- Without it, a new required field silently becomes undefined in test data.
- The next question: new field -> TS immediately flags missing generator.
- Avoid: a plain-object generator that drifts silently out of sync.

```
type DataGenerator<T> = { [K in keyof T]: () => T[K] };

interface User { name: string; email: string; age: number; role: 'admin' | 'user'; }

const userGenerator: DataGenerator<User> = {
  name:  () => faker.person.fullName(),
  email: () => faker.internet.email(),
  age:   () => faker.number.int({ min: 18, max: 80 }),
  role:  () => faker.helpers.arrayElement(['admin', 'user'] as const),
};

function generateUser(overrides?: Partial<User>): User {
  const base = Object.fromEntries(
    Object.entries(userGenerator).map(([k, gen]) => [k, (gen as () => unknown)()]),
  ) as User;
  return { ...base, ...overrides };
}
```

Q 260 LEVEL · Mid to Senior

How do you use decorators for test metadata in TypeScript?

- Decorators attach metadata to test classes/methods; queryable programmatically.
- @TestSuite on class, @TestCase on methods (tags, priority).
- Most Playwright projects: simpler @smoke title + --grep is enough.
- Reach for decorators when a custom orchestrator/platform needs structured metadata.
- Asked next: actually used when string-matching titles is fragile.
- The easy mistake: recommending decorators for every project, overkill.

```
// tsconfig.json: "experimentalDecorators": true
import 'reflect-metadata';  // Reflect.defineMetadata comes from this polyfill
function TestSuite(name: string) {
  return function(ctor: Function) { Reflect.defineMetadata('suite:name', name, ctor); };
}
function TestCase(opts: { tags?: string[]; priority?: 'high' | 'medium' | 'low' }) {
  return function(target: any, key: string) { Reflect.defineMetadata('test:options', opts, target, key); };
}

@TestSuite('Authentication')
class LoginTests {
  @TestCase({ tags: ['@smoke', '@critical'], priority: 'high' })
  async validLoginRedirects() { /* ... */ }

  @TestCase({ tags: ['@regression'], priority: 'medium' })
  async invalidPasswordShowsError() { /* ... */ }
}
```

Q 261 LEVEL · Mid

How do you use discriminated unions for test state management?

Mutually exclusive states modelled as a union with a discriminant field. switch on the discriminant narrows the type to that case's fields. Add a new state: compiler flags every unhandled switch. Same single-source-of-truth idea as Record-by-enum, for state machines. Follow-up: new state -> TS fan-out points at every site to update. Trap: a broad interface with all fields optional, loses narrowing.

```
type TestState =
  | { status: 'pending' }
  | { status: 'running'; startTime: number }
  | { status: 'passed'; duration: number; screenshot?: string }
  | { status: 'failed'; duration: number; error: string; trace: string };

function handleTestResult(state: TestState): string {
  switch (state.status) {
    case 'pending': return 'Waiting to run';
    case 'running': return `Running for ${Date.now() - state.startTime}ms`;
    case 'passed':  return `Passed in ${state.duration}ms`;
    case 'failed':  return `Failed: ${state.error}`; // TS knows .error exists
  }
}
```

Q 262 LEVEL · Mid to Senior

How do you use module augmentation to extend Playwright types?

- Extend @playwright/test's Page/Matchers without modifying the library.
- declare module + runtime impl on Page.prototype / expect.extend.
- Custom methods feel first-class: **autocomplete**, **type safety**, **no fork**.
- Use for page-level utilities and custom matchers (toBeLoggedIn, etc.).
- Expect next: tests use the new methods on ordinary Page, no special import.
- Where this goes wrong: wrapping Page in a custom class everyone must import.

```
// src/types/playwright.d.ts: extend Playwright's types
import { Page } from '@playwright/test';

declare module '@playwright/test' {
  interface Page {
    loginAs(role: 'admin' | 'user' | 'viewer'): Promise<void>;
    clearAllCookiesAndStorage(): Promise<void>;
  }
}
declare global {
  namespace PlaywrightTest {
    interface Matchers<R> {
      toBeLoggedIn(): R;
      toHavePermission(permission: string): R;
    }
  }
}

// Runtime implementation
(Page.prototype as any).loginAs = async function(role: string) {
...
```

Q 263 LEVEL · Senior

How do generic constraints and infer let you build type-safe test helpers that work for any shape?

K extends keyof T: field-name typos become compile errors. infer U inside conditional: extract the unwrapped type from Promise/array. expectFieldEquals(user, 'email', value): third arg type inferred from field. Compiler catches type mismatches before tests run. They usually ask next: highest leverage in data-driven tests + assertion helpers. What loses the point: helpers typed string + any, enforce nothing, typos slip through.

```
// Pattern 1: extends keyof, field-name typos fail at compile time
function expectFieldEquals<T, K extends keyof T>(obj: T, key: K, value: T[K]): void {
  if (obj[key] !== value) throw new Error(`Expected ${String(key)}=${value}, got ${obj[key]}`);
}

interface User { email: string; age: number; }
const user: User = { email: 'a@b.c', age: 30 };

expectFieldEquals(user, 'email', 'a@b.c');   // OK
expectFieldEquals(user, 'email', 30);         // error: number not string
expectFieldEquals(user, 'emial', 'x');        // error: 'emial' not in User keys

// Pattern 2: infer extracts type from a wrapping type
type UnwrapPromise<T> = T extends Promise<infer U> ? U : T;

async function fetchUser(): Promise<User> { /* ... */ return {} as User; }
type Fetched = UnwrapPromise<ReturnType<typeof fetchUser>>;   // Fetched = User

// Test helper using infer to give callers the unwrapped type
async function expectResolves<T>(p: Promise<T>): Promise<T> {
...
```

Q 264 LEVEL · Senior

How does the `satisfies` operator help you keep test config and test data both type-safe and inference-rich?

satisfies checks shape AND preserves narrow literal inference. Colon Type widens literals; no annotation skips shape-checking. Uses: **env-specific config maps**, **fixture option tables**, **narrowing-aware builders**. Compare with as const (freezes but no shape check). Pitfall: colon Type on test config, then literal-typed code downstream breaks. Don't use for mutable bindings that need widened type, annotate those.

```
// BEFORE satisfies: two bad options
// Option A: : Type widens literals
const config1: PlaywrightTestConfig = {
  projects: [{ name: 'chromium', use: { browserName: 'chromium' } }],
};
// config1.projects[0].name is now `string`, not `'chromium'`

// Option B: no annotation, no shape check
const config2 = {
  projects: [{ name: 'chromium', use: { browserName: 'chromium' } }],
};
// Typo in field name? No error. config2.projetcs is silently undefined.

// satisfies: shape verified, literals preserved
const config3 = {
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox',  use: { browserName: 'firefox'  } },
  ],
} satisfies PlaywrightTestConfig;
...
```

QUICK QUIZ

1. What concrete bug do template literal types like `/api/users/${number}` prevent?
   A) A wrong-type URL path: caught at compile time instead of as a runtime 404
   B) Race conditions in tests
   C) Memory leaks
   D) Locator strict-mode violations
2. Why use a mapped type like DataGenerator<T> for test data?
   A) It stays in sync with the interface: a new field becomes a compile error until a generator is added
   B) It makes tests run faster
   C) It replaces faker
   D) It removes the need for fixtures
3. When are TypeScript decorators the right choice over @tag-in-title + --grep?
   A) For every project: they're always better
   B) When a custom orchestrator or platform needs to query structured metadata programmatically
   C) For simple smoke-test filtering
   D) Whenever the suite has more than 10 tests
4. What property of discriminated unions makes them ideal for state machines?
   A) They run faster than enums
   B) They reduce bundle size
   C) They avoid using TypeScript
   D) Exhaustiveness: adding a new state makes the compiler flag every unhandled switch
5. What does module augmentation let you do for Playwright?
   A) Replace the Playwright runner
   B) Bypass the Playwright config
   C) Extend Page and Matchers with custom methods/assertions that feel first-class, no fork
   D) Disable type checking

Answers: 1:A 2:A 3:B 4:D 5:C

Docker and Containerised Testing

Q 265 LEVEL · Mid

How do you run Playwright tests in Docker?

Use the official mcr.microsoft.com/playwright image, no manual installs. ipc:host fixes Chromium's silent crash from a too-small /dev/shm. depends_on service_healthy gates Playwright on the app's health check. baseURL = http://app:3000 (the compose service name). The next question: ipc:host is for Chromium's shared-memory IPC. The common miss: installing browsers manually inside the Dockerfile.

```
# Dockerfile
FROM mcr.microsoft.com/playwright:v1.45.0-jammy
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npx", "playwright", "test"]

# docker-compose.yml
services:
  app:
    image: my-app:latest
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 5s
  playwright:
    image: mcr.microsoft.com/playwright:v1.45.0-jammy
    depends_on:
      app: { condition: service_healthy }
    environment:
...
```

Q 266 LEVEL · Mid

How do you optimise Docker image size for Playwright?

Full image ~2GB; Chromium-only ~800MB: ~60% reduction. Multi-stage build + rm -rf /ms-playwright/firefox*/webkit*. Pitch in time/cost: 90s saved x 50 CI runs = ~75 min/day. Pattern: Chromium in Docker for PRs, all browsers nightly on the host. Asked next: keep cross-browser coverage by running all three nightly. Avoid: over-optimising a low-frequency pipeline for theoretical gains.

```
# Multi-stage build: Chromium only
FROM mcr.microsoft.com/playwright:v1.45.0-jammy AS base
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM mcr.microsoft.com/playwright:v1.45.0-jammy AS test
WORKDIR /app
COPY --from=base /app/node_modules ./node_modules
COPY . .
RUN rm -rf /ms-playwright/firefox* /ms-playwright/webkit*  # ~2GB -> ~800MB
ENV PLAYWRIGHT_BROWSERS_PATH=/ms-playwright
CMD ["npx", "playwright", "test", "--project=chromium"]
```

Q 267 LEVEL · Junior to Mid

How do you handle browser display in headless Docker environments?

Headless is the default: no display needed in Docker for normal CI. Xvfb provides a virtual display for the rare headed-in-Docker case. Enforce headless in CI via config (headless: !!process.env.CI). --no-sandbox is a last resort; prefer running as non-root with sandbox on. Follow-up: --no-sandbox needed when container runs as root and you can't change that. The easy mistake: copy-pasting --no-sandbox everywhere from tutorials.

```
# Headless: default, no display configuration needed
npx playwright test

# Headed in Docker via Xvfb (debugging only)
RUN apt-get update && apt-get install -y xvfb
CMD ["sh", "-c", "Xvfb :99 -screen 0 1280x720x24 & DISPLAY=:99 npx playwright test --headed"]

// playwright.config.ts: enforce headless in CI
export default defineConfig({
  use: {
    headless: !!process.env.CI,
    launchOptions: { args: ['--no-sandbox'] }, // documented last resort
  },
});
```

Q 268 LEVEL · Mid to Senior

How do you implement parallel Docker execution for Playwright sharding?

Kubernetes Jobs: completions=parallelism=shard count, all pods start together. Shard index from batch.kubernetes.io/job-completion-index annotation. K8s handles scheduling, recovery, artifact collection across pods. Use when GitHub Actions matrix runner limit is hit (>20 shards / >1k tests). Expect next: GH matrix is enough until runner limit, K8s is the escalation. Trap: using K8s for a 200-test suite, over-engineering.

```
apiVersion: batch/v1
kind: Job
metadata: { name: playwright-tests }
spec:
  completions: 4
  parallelism: 4               # all 4 pods run simultaneously
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: playwright
          image: my-playwright:latest
          env:
            - name: SHARD_INDEX
              valueFrom:
                fieldRef:
                  fieldPath: metadata.annotations['batch.kubernetes.io/job-completion-index']
            - name: SHARD_TOTAL
              value: "4"
          command: ["npx", "playwright", "test", "--shard=$(SHARD_INDEX)/$(SHARD_TOTAL)"]
```

Q 269 LEVEL · Mid

How do you collect and merge test artifacts from Docker containers?

Mount test-results, blob-report, playwright-report, or artifacts vanish. Use playwright merge-reports to combine per-shard blob reports into one HTML. --exit-code-from playwright on compose up surfaces the container's exit code. Without it, docker-compose exits 0 on failure: CI marks green incorrectly. They usually ask next: CI misses failures because compose exits 0 by default. Where this goes wrong: missing mounts (lose artifacts) or missing flag (lose failure signal).

```
# docker-compose.yml: mount artifact dirs
services:
  playwright:
    image: my-playwright:latest
    volumes:
      - ./test-results:/app/test-results
      - ./playwright-report:/app/playwright-report
      - ./blob-report:/app/blob-report

# Run + propagate the Playwright container's exit code (else CI thinks 0)
docker-compose up --exit-code-from playwright playwright

# Merge per-shard blob reports into one HTML
docker run --rm -v $(pwd)/blob-reports:/blob-reports -v $(pwd)/merged-report:/merged-report \
  mcr.microsoft.com/playwright:v1.45.0-jammy \
  npx playwright merge-reports --reporter html /blob-reports
```

Q 270 LEVEL · Senior

How do you set up Docker networking for a Playwright suite that tests a multi-service app?

Compose default bridge network: services reach each other by service name. Custom networks: isolate groups (frontend net, backend net) when needed. Host network mode: fast, no isolation, only when tools require it. Service names as URLs in config: no hard-coded IPs, no container IDs. The next question: debug via docker-compose exec + curl/ping the service name. What loses the point: localhost in a container = the container itself, not the host.

```
# docker-compose.yml: multi-service test setup
version: '3.8'
services:
  app:
    build: .
    ports: ['3000:3000']
    environment:
      DATABASE_URL: postgres://postgres:postgres@db:5432/test
      QUEUE_URL:    redis://redis:6379
    depends_on: [db, redis]

  db:
    image: postgres:17
    environment: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: test }

  redis:
    image: redis:7-alpine

  playwright:
    image: mcr.microsoft.com/playwright:v1.45.0-jammy
...
```

Q 271 LEVEL · Senior

How do you use BuildKit and layer caching to keep Playwright Docker images fast to build in CI?

BuildKit: enable via DOCKER_BUILDKIT=1 or buildx, parallel layers + cache mounts. Layer ordering: COPY package*.json + install BEFORE COPY . . Cache mounts: RUN --mount=type=cache,target=/root/.npm, persists across rebuilds. Registry caching: cache-to/cache-from in CI, warm starts across runners. Rule: order from least to most frequently changing, cache invalidation cascades down. The common miss: COPY . . near the top: any code change rebuilds everything after.

```
# Dockerfile: optimised for layer caching
# syntax=docker/dockerfile:1.4               <- enables BuildKit features
FROM mcr.microsoft.com/playwright:v1.45.0-jammy

WORKDIR /work

# Step 1: copy only dependency manifests, stable, cached aggressively
COPY package.json package-lock.json ./

# Step 2: install deps with cache mount, cache survives layer rebuilds
RUN --mount=type=cache,target=/root/.npm \
    npm ci --prefer-offline --no-audit

# Step 3: now copy the source, changes here don't invalidate the install layer
COPY . .

CMD ["npx", "playwright", "test"]

# CI build with registry caching (GitHub Actions example)
#   docker buildx build \
...
```

QUICK QUIZ

1. What does ipc:host fix when running Playwright in Docker?
   A) Network routing between containers
   B) Browser binary downloads
   C) Chromium's silent crash from a too-small /dev/shm (default 64MB)
   D) Headed-mode display
2. What is the realistic Chromium-only image size after multi-stage build and browser pruning?
   A) ~800MB, about 60% reduction; ~90s saved per CI pull
   B) ~2GB, same as full image
   C) ~50MB, alpine-minimal
   D) ~5GB, with extra layers
3. How should --no-sandbox be treated in a Docker Playwright setup?
   A) Always on by default
   B) A documented last resort; prefer non-root container with sandbox enabled
   C) Never used under any circumstance
   D) Required for headless mode
4. When is a Kubernetes Job the right choice over a GitHub Actions matrix?
   A) Any suite over 50 tests
   B) When the suite exceeds ~1,000 tests and >20 shards hits GitHub Actions runner limits
   C) When using a single browser
   D) Always: GH Actions is unreliable
5. Why is --exit-code-from playwright critical on docker-compose up?
   A) It speeds up tests
   B) It mounts artifact directories
   C) It enables Chromium
   D) Without it, docker-compose exits 0 even when tests fail, CI marks the step green incorrectly

Answers: 1:C 2:A 3:B 4:B 5:D

Database and SQL Testing

Q 272 LEVEL · Mid

How do you integrate database validation into Playwright tests?

Fixture injects a shared DB client (pg.Pool): don't close per test. Pattern: UI action -> assert UI outcome -> query DB -> assert row + values. Highest-confidence pattern: catches silent-write-failure bugs. Also catches wrong-role saves and missing required fields. Asked next: stronger than UI alone because UI can lie about persistence. Avoid: closing the pool per test (and forgetting to clean up test rows).

```
// src/fixtures/db.fixture.ts
import { test as base } from '@playwright/test';
import { Pool } from 'pg';
const pool = new Pool({
  host: process.env.DB_HOST, port: Number(process.env.DB_PORT) || 5432,
  database: process.env.DB_NAME, user: process.env.DB_USER, password: process.env.DB_PASSWORD,
});
export const test = base.extend<{ db: Pool }>({
  db: async ({}, use) => { await use(pool); }, // shared, do not close per test
});

test('registration persists to the database', async ({ page, db }) => {
  const email = `test-${Date.now()}@example.com`;
  await page.goto('/register');
  await page.getByLabel('Email').fill(email);
  await page.getByLabel('Password').fill('SecurePass@123');
  await page.getByRole('button', { name: 'Create Account' }).click();
  await expect(page).toHaveURL('/dashboard');

  const result = await db.query('SELECT id, email, role FROM users WHERE email = $1', [email]);
...
```

Q 273 LEVEL · Mid

How do you use database seeding for test data setup?

globalSetup is the right home for data every test depends on. ON CONFLICT DO NOTHING makes the seed idempotent across re-runs. Scoped DELETE-before-INSERT for test-specific data, known clean state. Use %@test.com email pattern so cleanup queries are safe and specific. Follow-up: globalSetup runs twice, ON CONFLICT keeps it safe. The easy mistake: DELETE FROM users without a WHERE clause in a shared env.

```
// src/setup/global.setup.ts
import { Pool } from 'pg';
export default async function globalSetup() {
  const pool = new Pool({ connectionString: process.env.DATABASE_URL });
  try {
    // Scoped DELETE: only test-prefixed rows
    await pool.query("DELETE FROM users  WHERE email LIKE '%@test.com'");
    await pool.query("DELETE FROM orders WHERE created_by LIKE '%@test.com'");

    // Idempotent reference data
    await pool.query(`
      INSERT INTO product_categories (id, name, slug) VALUES
        (1, 'Electronics', 'electronics'),
        (2, 'Clothing',    'clothing'),
        (3, 'Books',       'books')
      ON CONFLICT (id) DO NOTHING
    `);

    // Idempotent test users
    await pool.query(`
...
```

Q 274 LEVEL · Mid to Senior

How do you test database transactions and rollback behaviour?

Verify multi-step operations are atomic: all-or-nothing. Pattern: mock the dependency to fail, assert UI error, assert DB unchanged. Critical for financial/inventory: partial writes are real data bugs. Compare row counts before and after to confirm rollback. Expect next: **most-needed for payments**, **orders**, **inventory decrements**. Trap: asserting only on the UI error and trusting the rollback.

```
test('failed payment rolls back order creation', async ({ page, db }) => {
  await page.route('**/api/payments', route => route.fulfill({
    status: 402, body: JSON.stringify({ error: 'Insufficient funds' }),
  }));

  const before = (await db.query('SELECT COUNT(*) FROM orders')).rows[0].count;

  await page.goto('/checkout');
  await page.getByRole('button', { name: 'Place Order' }).click();
  await expect(page.locator('.error-banner')).toContainText('Insufficient funds');

  const after = (await db.query('SELECT COUNT(*) FROM orders')).rows[0].count;
  expect(after).toBe(before); // rollback confirmed
});
```

Q 275 LEVEL · Mid to Senior

How do you test database migrations in a CI pipeline?

Run migration tests as a separate CI stage right after migration apply. Verify two things: schema correctness + existing-data preservation. Schema check: **information_schema for column existence**, **type**, **nullability**. Data check: query seeded rows, assert fields intact and nulls handled. They usually ask next: caught NOT NULL -> nullable before staging. Where this goes wrong: assuming migration-tool success means the migration is correct.

```
test('migration adds phone_number column to users', async ({ db }) => {
  const r = await db.query(`
    SELECT column_name, data_type, is_nullable
    FROM information_schema.columns
    WHERE table_name = 'users' AND column_name = 'phone_number'
  `);
  expect(r.rows).toHaveLength(1);
  expect(r.rows[0].data_type).toBe('character varying');
  expect(r.rows[0].is_nullable).toBe('YES'); // nullable for existing users
});

test('migration preserves existing data', async ({ db }) => {
  const r = await db.query(
    'SELECT id, email, role, phone_number FROM users WHERE email = $1',
    ['admin@test.com'],
  );
  expect(r.rows[0].role).toBe('admin');
  expect(r.rows[0].phone_number).toBeNull(); // correct null handling
});
```

Q 276 LEVEL · Mid to Senior

How do you handle test database isolation in parallel test execution?

Three strategies: **unique data**, **schema-per-worker**, **transaction rollback**. Unique data per test: **simple**, **reliable**, **default for most cases**. Schema-per-worker: complete isolation, for complex suites. Transaction rollback: read-only tests only; rollback undoes persistence. The next question: default is unique data per test. What loses the point: using rollback for write tests, false-negative assertions.

```
// 1) Unique data per test: default
test('user appears in the list', async ({ db, page }) => {
  const email = `test-${Date.now()}-${Math.random()}@example.com`;
  await db.query('INSERT INTO users (email, role) VALUES ($1, $2)', [email, 'user']);
  await page.goto('/admin/users');
  await expect(page.locator('tbody tr').filter({ hasText: email })).toBeVisible();
  await db.query('DELETE FROM users WHERE email = $1', [email]);
});

// 2) Schema-per-worker (in globalSetup): CREATE SCHEMA test_worker_$WORKER_INDEX

// 3) Transaction rollback: READ-ONLY tests only
test('read-only count', async ({ db }) => {
  await db.query('BEGIN');
  try {
    const r = await db.query('SELECT COUNT(*) FROM users');
    expect(Number(r.rows[0].count)).toBeGreaterThan(0);
  } finally { await db.query('ROLLBACK'); }
});
```

Q 277 LEVEL · Senior

How do you adapt SQL testing patterns when the database is NoSQL (MongoDB, DynamoDB, Firestore)?

No cross-document transactions: cleanup via registry, not BEGIN/ROLLBACK. Schema flexibility -> drift: validate shape with zod/ajv, not just values. Eventual consistency: writes don't appear in reads immediately, strong-read or poll. Understand the consistency model BEFORE writing tests. Asked next: biggest gotcha = eventual consistency, produces flake disguised as bugs. The common miss: **NoSQL-as-SQL-with-different-syntax**, **semantic differences**, **not just syntax**.

```
// MongoDB: cleanup registry (no easy ROLLBACK across documents)
export const test = base.extend<{ mongo: MongoClient; cleanup: { add: (col: string, id: ObjectId) => void...
  mongo: async ({}, use) => {
    const client = await MongoClient.connect(process.env.MONGO_URL!);
    await use(client);
    await client.close();
  },
  cleanup: async ({ mongo }, use) => {
    const tracked: { col: string; id: ObjectId }[] = [];
    await use({ add: (col, id) => tracked.push({ col, id }) });
    for (const { col, id } of tracked.reverse()) {
      await mongo.db('test').collection(col).deleteOne({ _id: id }).catch(() => {});
    }
  },
});

// DynamoDB: strong-read option to bypass eventual consistency
const result = await dynamo.send(new GetCommand({
  TableName: 'Users',
  Key: { userId: 'u_123' },
...
```

Q 278 LEVEL · Senior

How do you anonymise production data for use in test environments without losing test value?

Production data = realism; PII = legal/ethical risk. Anonymisation bridges. Deterministic hashing: same input -> same output. Relations preserved, values fake. Format-preserving substitution: faker for phone/name/address, validation still works. Field-level encryption/redaction: for non-user-visible sensitive fields. Rule: preserve distributions and relationships, NOT exact values. Avoid: partial anonymisation, free-text fields still full of PII.

```
// Anonymisation pipeline for production-cloned data
import { createHash } from 'crypto';
import { faker } from '@faker-js/faker';

const SALT = process.env.ANON_SALT!;          // never in code; pulled from secrets

function anonEmail(real: string): string {
  const h = createHash('sha256').update(real + SALT).digest('hex').slice(0, 12);
  return `${h}@example.com`;                  // deterministic + format-preserving
}

function anonName(real: string): string {
  faker.seed(hashToInt(real + SALT));         // deterministic faker output
  return faker.person.fullName();
}

function anonPhone(real: string): string {
  faker.seed(hashToInt(real + SALT));
  return faker.phone.number();                // valid format, fake number
}
...
```

QUICK QUIZ

1. Why is a DB-assertion test stronger than a UI-only assertion?
   A) It is faster
   B) The UI can show success while the write failed silently, only a DB query catches that
   C) It avoids using fixtures
   D) It doesn't need a database client
2. Which two patterns make database seeding safe and idempotent?
   A) ON CONFLICT DO NOTHING for reference data; scoped DELETE-before-INSERT for test-specific data
   B) DROP TABLE before insert; RAISE on duplicates
   C) Random data per run; no cleanup
   D) Only seed in tearDown
3. What does a transaction-rollback test verify?
   A) That the UI shows a loading spinner
   B) That the test is fast
   C) That a failed multi-step operation leaves the database unchanged (atomicity)
   D) That the API returns 200
4. What two things does a migration test verify?
   A) Speed and memory
   B) Code coverage and lint rules
   C) Only that the migration ran
   D) Schema correctness (via information_schema) AND existing-data preservation
5. Which DB isolation strategy fits ONLY read-only tests?
   A) Transaction rollback (BEGIN/ROLLBACK): the rollback undoes any persistence write-tests must verify
   B) Unique data per test
   C) Schema-per-worker
   D) Truncating tables before each test

Answers: 1:B 2:A 3:C 4:D 5:A

Git and Version Control

Q 279 LEVEL · Mid

How do you structure Git workflows for a Playwright framework?

Treat the framework as application code: same standards. Branch naming convention: **feature/**, **fix/**, **refactor/**, **chore/**. Protected main: passing CI + 1 reviewer, no direct pushes. husky + lint-staged: ESLint + Prettier on every commit. Follow-up: conventional commits power automatic changelog generation. The easy mistake: looser rules for the framework 'because it's only tests'.

```
# Branch naming convention
feature/add-checkout-tests
fix/flaky-login-test
refactor/extract-data-table-component
chore/update-playwright-1.46

// package.json: husky + lint-staged + commitlint
{
  "husky": {
    "hooks": {
      "pre-commit": "lint-staged",
      "commit-msg": "commitlint --edit $1"
    }
  },
  "lint-staged": { "*.ts": ["eslint --fix", "prettier --write"] }
}

// commitlint.config.js: enforce conventional commits
module.exports = {
  extends: ['@commitlint/config-conventional'],
...
```

Q 280 LEVEL · Mid

How do you handle Playwright baseline screenshot updates in Git?

Never --update-snapshots in CI: silently accepts every regression. Update locally; review diff image in the HTML report. Commit baselines with a clear intentional-change message. PR reviewer verifies the visual change is intentional before merge. Expect next: enforce via a gated env variable plus PR-template review. Trap: auto-updating baselines on CI failure 'to keep tests green'.

```
# Update baselines locally only
npx playwright test --update-snapshots

# .gitattributes: mark PNGs as binary
*.png binary

# .gitignore: ignore test output, KEEP baselines
test-results/
!tests/**/*.png

# Baseline update process
# 1. Visual test fails with diff image in HTML report
# 2. Reviewer opens diff image and verifies the change is intentional
# 3. If intentional: npx playwright test --update-snapshots
# 4. Commit:  git commit -m "test: update visual baselines for nav redesign"
# 5. PR reviewer confirms visual change matches the PR's intent
```

Q 281 LEVEL · Mid

How do you use Git hooks to enforce Playwright best practices?

- Pre-commit for fast checks; pre-push for slower (e.g. smoke).
- Block test.only/it.only/describe.only: prevents silent-skip incidents.
- Warn (not block) on waitForTimeout: educate without disrupting.
- Heavy blocks get bypassed with --no-verify: design hooks people accept.
- They usually ask next: caught 3 test.only incidents that would have hidden 849 skips.
- Where this goes wrong: making every check a hard block until people --no-verify everything.

```
#!/bin/sh
# .husky/pre-commit: fast checks on every commit
. "$(dirname "$0")/_/husky.sh"

npm run typecheck || exit 1
npm run lint || exit 1

# Block focused tests: the most valuable hook
if grep -r "test\.only\|it\.only\|describe\.only" tests/; then
  echo "ERROR: test.only found. Remove before committing."
  exit 1
fi

# Warn (not block) on anti-patterns
if grep -r "waitForTimeout" tests/ src/; then
  echo "WARNING: waitForTimeout is an anti-pattern. Use explicit waits."
fi

# .husky/pre-push: smoke before pushing to main
BRANCH=$(git rev-parse --abbrev-ref HEAD)
...
```

Q 282 LEVEL · Junior to Mid

How do you manage Playwright framework versioning and upgrades?

Playwright releases roughly monthly: upgrade monthly, not annually. Cycle: **chore branch**, **npm install**, **browsers**, **full suite**, **release notes**, **baselines**. Whole cycle ~30 minutes when done regularly. Dependabot weekly PRs: CI runs automatically, merge if green. The next question: fall behind, 6 months -> multi-day migration with conflicts. What loses the point: putting off upgrades because the current version 'works fine'.

```
# Monthly upgrade cycle (~30 min when done regularly)
git checkout -b chore/upgrade-playwright-1.46
npm install @playwright/test@latest
npx playwright install
npm run test                                # full suite
# Check release notes: https://playwright.dev/docs/release-notes
npx playwright test --update-snapshots --project=chromium  # if rendering shifted

# .github/dependabot.yml: automated weekly PR
version: 2
updates:
  - package-ecosystem: npm
    directory: /
    schedule: { interval: weekly }
    allow:
      - dependency-name: "@playwright/test"
```

Q 283 LEVEL · Mid to Senior

How do you use git bisect to find the commit that introduced a test failure?

Binary search through commit history: ~6-8 steps for 100 commits. git bisect run with a test script automates the whole search. Real example: 47 commits, ~10 min to find a 2-pixel CSS shift. Dramatic saving vs hours of manual diff inspection. Asked next: don't use bisect on intermittent tests, it lands wrong. The common miss: doing the binary search by hand when 'bisect run' exists.

```
# Manual bisect
git bisect start
git bisect bad                              # current commit fails
git bisect good v1.44.0                     # known-good tag
# Git checks out the midpoint: run the failing test
npx playwright test tests/checkout.spec.ts --project=chromium
git bisect good                             # or: git bisect bad
# repeat … typically 6-8 steps for 100 commits
git bisect reset

# Automated bisect: zero manual input
git bisect start HEAD v1.44.0
git bisect run npx playwright test tests/checkout.spec.ts --project=chromium
# "abc1234 is the first bad commit"
```

QUICK QUIZ

1. What's the right mindset for Git workflows on a Playwright framework?
   A) Looser rules: it's only tests
   B) Treat the framework like application code: protected main, hooks, conventional commits
   C) No PR reviews needed
   D) Commit only to main
2. What's the single non-negotiable rule for visual screenshot baselines?
   A) Update baselines on every commit
   B) Use JPEG instead of PNG
   C) Delete baselines weekly
   D) Never run --update-snapshots in CI: it silently accepts every regression
3. Which pre-commit hook has prevented real incidents?
   A) A hook that auto-formats code
   B) A hook that increments version numbers
   C) A grep that blocks test.only / it.only / describe.only, prevents silent-skip in CI
   D) A hook that runs all tests
4. What is the right Playwright upgrade cadence?
   A) Monthly, ~30 minutes when done regularly: Dependabot can automate it
   B) Yearly major migrations
   C) Only when forced by breaking changes
   D) Never: keep stable
5. What does 'git bisect run' do that manual bisect doesn't?
   A) It's slower
   B) It automates the binary search by running a test script at each step, zero manual input
   C) It bypasses the commit history
   D) It modifies the bad commit

Answers: 1:B 2:D 3:C 4:A 5:B

Playwright vs Other Frameworks

Q 284 LEVEL · Mid

Playwright vs Selenium: when do you choose each?

Architecture: Playwright = persistent WebSocket, classic Selenium = HTTP WebDriver. WebSocket enables auto-waiting, network interception, trace viewer, on every engine. Currency: Selenium 4 has CDP, and 4.18+ has WebDriver BiDi interception. Don't say it cannot. Playwright for new projects / TypeScript / modern SPAs. Selenium for large existing investment, real SafariDriver, Java mandate. Follow-up: stay on Selenium when migration cost exceeds benefit. Avoid: **'Playwright is always better'**, **reads as tribal**, **not senior**.

Q 285 LEVEL · Mid

Playwright vs Cypress: what are the key architectural differences?

- Cypress in-process (sync DOM, GUI); Playwright out-of-process (WebSocket).
- Cypress limits: **Chromium-mainly**, **no native multi-tab**, **cy**.origin for multi-origin.
- Playwright wins: **all three browsers**, **multi-tab/origin**, **mobile**, **multi-language**.
- Cypress wins: **developer experience**, **GUI**, **hot reload**, **time-travel debugging**.
- Expect next: choose Cypress for pure frontend JS team prioritising DX.
- The easy mistake: **dismissing Cypress's DX**, **reads as tribal**, **not informed**.

Q 286 LEVEL · Mid

Playwright vs WebdriverIO: when does WebdriverIO win?

WebdriverIO supports both WebDriver and CDP: bridge tool. Wins: first-class Appium for native mobile in one framework with web. Playwright wins for web-only fresh starts (simpler, better TS, richer built-ins). Decision driver: breadth of platforms actually needed, not modernity vibes. They usually ask next: don't migrate WDIO+Appium team to Playwright if they need native mobile. Trap: treating WebdriverIO as Selenium's smaller cousin.

Q 287 LEVEL · Junior to Mid

Playwright vs Puppeteer: what is the relationship?

Same team at Google: Playwright is the spiritual successor. Puppeteer: Chromium-only, no test runner; Playwright: 3 browsers + runner. Different purposes: Puppeteer for scripts, Playwright for tests. Use Puppeteer for scraping/PDFs/screenshots; Playwright for the suite. The next question: Puppeteer is not obsolete, it's the right tool for scripting. Where this goes wrong: implying Puppeteer is dead because Playwright is newer.

Q 288 LEVEL · Mid to Senior

How do you make the framework-selection decision for a new project?

Five steps: **team expertise**, **app type**, **existing investment**, **CI needs**, **POC**. Match language: TypeScript -> Playwright, Java -> Selenium/WDIO, JS -> Cypress/Playwright. Big existing investment -> gradual migration, not rewrite. POC = 10 representative tests; measure setup, runtime, flakiness, LOC. Asked next: POC data beats advocacy under stakeholder pressure. What loses the point: committing to a framework on hype rather than measured DX.

Q 289 LEVEL · Mid to Senior

How do you make the call between Playwright and a BDD framework (Cucumber, Behave, Robot Framework) for a new project?

BDD wins: **real cross-functional collab**, **regulated industries**, **heavy step reuse**. Playwright wins: **speed**, **debugging clarity**, **simpler tooling**. Combo: playwright-bdd or @cucumber/cucumber + Playwright for both worlds. Rule: adopt BDD only when the collaboration it enables is real. Follow-up: playwright-bdd works but is rarely the right default. Pitfall: **BDD as cargo cult**, **nobody reads the Gherkin**, **all maintenance cost**.

```
// PLAYWRIGHT direct (most teams): fast, clear, less to maintain
test('user can complete checkout', async ({ page }) => {
  await page.goto('/checkout');
  await page.getByLabel('Card number').fill('4242 4242 4242 4242');
  await page.getByRole('button', { name: 'Place order' }).click();
  await expect(page.getByText('Order confirmed')).toBeVisible();
});

// PLAYWRIGHT-BDD (when stakeholders truly co-author scenarios)
// features/checkout.feature
//   Feature: Checkout
//     Scenario: User completes checkout
//       Given the user is on the checkout page
//       When they submit valid payment details
//       Then they see the order-confirmation page

// steps/checkout.steps.ts
import { createBdd } from 'playwright-bdd';
const { Given, When, Then } = createBdd();
Given('the user is on the checkout page', async ({ page }) => {
...
```

TypeScript vs JavaScript vs Python/Java bindings for Playwright, what's the same across them and why do most teams pick TypeScript?

- Same engine, same WebSocket/CDP protocol across TS/JS/Python/Java/C#.
- Auto-waiting, web-first assertions, locators, tracing behave identically.
- JS/TS gets new features first and owns the richest runner (@playwright/test).
- JavaScript vs TypeScript = same binding; TS just adds static types.
- TS chosen for: **TS-first docs**, **compile-time safety**, **most mature runner**.
- Calibration: pick Python/Java when it fits the team, capability is identical.
- The common miss: calling non-JS bindings second-class, they share the same engine.

```
// Same API, different binding: the click reads the same everywhere

// TypeScript / JavaScript (@playwright/test, richest runner)
import { test, expect } from '@playwright/test';
test('login', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('qa@example.com');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByText('Welcome')).toBeVisible();
});

# Python (pytest-playwright): same engine, same protocol
# def test_login(page):
#     page.goto('/login')
#     page.get_by_label('Email').fill('qa@example.com')
#     page.get_by_role('button', name='Sign in').click()
#     expect(page.get_by_text('Welcome')).to_be_visible()

// Java (JUnit/TestNG): first-class, host-ecosystem runner
//   page.navigate("/login");
...
```

QUICK QUIZ

1. When is Selenium the better choice over Playwright?
   A) Never: Playwright wins everything
   B) Only for testing in Internet Explorer
   C) Significant existing investment, real Safari via SafariDriver, or Java mandate
   D) Whenever the team prefers it
2. Which architectural fact explains most Cypress vs Playwright differences?
   A) Cypress was created first
   B) Playwright doesn't support TypeScript
   C) Cypress uses Selenium internally
   D) Cypress runs in-process with the browser; Playwright runs out-of-process via WebSocket
3. When is WebdriverIO the right choice?
   A) When web + native iOS + native Android must live in one framework (Appium maturity)
   B) Web-only testing on TypeScript teams
   C) Only for legacy Selenium migration
   D) Never: always pick Playwright
4. How should Puppeteer be positioned vs Playwright?
   A) Puppeteer is obsolete
   B) Different purposes: Puppeteer for scripting (scrape/PDF/screenshots), Playwright for testing
   C) They are interchangeable
   D) Puppeteer supports all 3 browsers too
5. What is the single most reliable framework-selection step?
   A) Read benchmarks
   B) Ask Twitter
   C) Run a POC with ~10 representative tests; measure setup time, runtime, flakiness, LOC
   D) Pick the newest tool

Answers: 1:C 2:D 3:A 4:B 5:C

AI, MCP and Agentic Testing

Q 291 LEVEL · Mid to Senior

What is prompt engineering for TypeScript test generation, and how do you do it well?

- Prompt quality decides everything; TypeScript context dramatically lifts it.
- Include typed page-object signatures, fixtures path, strict-mode note.
- Constraints: test.extend fixtures, getByRole, no any, return only the file.
- tsc --noEmit on the generated file = first hallucination filter (JS can't).
- Expect next: typed signatures + compiler validation are the TS-only advantages.
- Avoid: vague prompts producing untyped, happy-path-only tests.

```
// Strong TypeScript test-generation prompt
const prompt = `
You are a senior SDET with 10 years of Playwright TypeScript experience.
Framework: Playwright + TypeScript, POM, fixtures in src/fixtures/index.ts. strict mode on.
Page Objects (typed):
  LoginPage: navigate(): Promise<void>; login(email: string, password: string): Promise<void>; getErrorMe...
  DashboardPage: welcomeHeading: Locator; getStatValue(label: string): Promise<string>
Imports: import { test, expect } from '@fixtures/index';

User Story: Registered user logs in with email and password.
Acceptance Criteria:
  - Valid credentials -> redirect /dashboard, welcome heading visible
  - Invalid password -> 'Invalid email or password' alert
  - Empty email -> 'Email is required'
  - 5 failed attempts -> 'Account locked'

Constraints: test.extend fixtures from @fixtures/index. getByRole locators.
Promise<void> return types. No any. test.describe + test.step.
Return only the typed test file. No explanation.`;

...
```

Q 292 LEVEL · Mid to Senior

What is LLM hallucination and how does TypeScript protect Playwright test quality?

- Hallucination = plausible-sounding but incorrect AI output.
- TypeScript is first-line defence: hallucinated methods fail the compiler.
- Three-step filter: compile (tsc --noEmit), ESLint, visual verify.
- no-explicit-any + prefer-web-first-assertions catch sneaked anys and isVisible.
- They usually ask next: TS catches at compile time; JS only at runtime.
- The easy mistake: trusting AI output because it looks right, the filter chain is mandatory.

```
// Three-step hallucination filter chain

// 1. Compile filter: catches hallucinated methods
tsc --noEmit src/tests/ai-generated.spec.ts
//   error TS2339: Property 'waitForNetworkIdle' does not exist on type 'Page'

// 2. ESLint filter: enforce AI hygiene
// .eslintrc.json
{
  "rules": {
    "@typescript-eslint/no-explicit-any": "error",          // no sneaked anys
    "playwright/prefer-web-first-assertions": "error",      // toBeVisible() not isVisible()
    "@typescript-eslint/no-floating-promises": "error"      // every action awaited
  }
}

// 3. Visual verify: run headed once before adding to the suite
npx playwright test src/tests/ai-generated.spec.ts --headed
```

Q 293 LEVEL · Mid to Senior

What is MCP (Model Context Protocol) and why does it matter for TypeScript SDETs?

- Open standard at modelcontextprotocol.io; broad industry adoption.
- Defines how AI connects to tools/data/systems: **files**, **code**, **DBs**, **browsers**, **APIs**.
- For SDETs: agents control browsers via Playwright, query CI, run suites.
- Analogy: MCP is to AI agents what WebDriver is to browser automation.
- The next question: transport is Streamable HTTP; SSE deprecated March 2025.
- Trap: calling MCP Anthropic-only, it is open and broadly governed.

Q 294 LEVEL · Mid to Senior

What is the Playwright MCP Server and how do you use it in a TypeScript workflow?

- Official MCP server exposing Playwright browser tools to AI agents.
- Configure in Claude Desktop OR call programmatically via the TS MCP SDK.
- browser_snapshot returns accessibility tree (semantic), not pixels.
- Snapshot for agent reasoning; screenshot for visual evidence only.
- Asked next: snapshot is faster/cheaper/more reliable, no vision model needed.
- Real win: AI found mobile checkout regression missed for 2 sprints.

```
// Configure in Claude Desktop (claude_desktop_config.json)
// {
//   "mcpServers": {
//     "playwright": {
//       "command": "npx",
//       "args": ["@playwright/mcp@latest", "--browser", "chromium"]
//     }
//   }
// }

// Programmatic TS usage
import { Client } from '@modelcontextprotocol/sdk/client/index.js';
import { StdioClientTransport } from '@modelcontextprotocol/sdk/client/stdio.js';

const transport = new StdioClientTransport({
  command: 'npx', args: ['@playwright/mcp@latest'],
});
const client = new Client({ name: 'ts-test-agent', version: '1.0.0' }, { capabilities: {} });
await client.connect(transport);

...
```

Q 295 LEVEL · Senior

How do you build a custom MCP server in TypeScript that exposes your Playwright framework?

- Custom server exposes test capabilities as AI-callable tools.
- MCP SDK Server class; ListTools + CallTool handlers.
- Typed arg interfaces (RunTestsArgs, CreateUserArgs) + typed results (TestSummary).
- Concrete win: 10-min CI check -> 30-sec Slack-to-Claude query.
- Follow-up: remote = StreamableHTTPServerTransport, not deprecated SSE.
- Where this goes wrong: describing MCP servers theoretically without ever building one.

```
import { Server } from '@modelcontextprotocol/sdk/server/index.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { CallToolRequestSchema, ListToolsRequestSchema, Tool } from '@modelcontextprotocol/sdk/types.js';
import { execSync } from 'child_process';

interface RunTestsArgs { grep?: string; project?: 'chromium'|'firefox'|'webkit'; }
interface TestFailure { title: string; error: string; file: string; }
interface TestSummary { passed: number; failed: number; durationMs: number; failures: TestFailure[]; }

const server = new Server({ name: 'playwright-ts', version: '1.0.0' }, { capabilities: { tools: {} } });

server.setRequestHandler(ListToolsRequestSchema, async (): Promise<{ tools: Tool[] }> => ({
  tools: [{
    name: 'run_playwright_tests',
    description: 'Run Playwright tests and return a typed summary',
    inputSchema: { type: 'object', properties: { grep: { type: 'string' }, project: { type: 'string', enu...
  }],
}));

server.setRequestHandler(CallToolRequestSchema, async (req) => {
...
```

Q 296 LEVEL · Senior

What are AI agents and how do you implement them in TypeScript for test automation?

- Four-stage loop: **Perceive**, **Decide**, **Act**, **Evaluate**.
- Perceive via browser_snapshot (semantic tree, no vision model).
- TS types make agents reliable: typed tool-call messages and results.
- Use for exploratory/bug-hunting; agents are non-deterministic by design.
- Expect next: NO agents in CI for regression, keep that deterministic.
- What loses the point: using an agent where deterministic regression is needed.

```
// Four-stage agent loop: typed
interface AgentStep {
  perception: { snapshot: unknown };          // 1. Perceive
  decision:   { tool: string; args: unknown };// 2. Decide
  action:     { result: unknown };            // 3. Act
  evaluation: { complete: boolean; finding?: string }; // 4. Evaluate
}

async function exploreCheckout(client: MCPClient): Promise<string[]> {
  const findings: string[] = [];
  while (true) {
    const snapshot = await client.callTool({ name: 'browser_snapshot' });  // Perceive
    const next = await claudeDecide({ goal: 'verify checkout works on mobile', snapshot });
    if (next.done) { findings.push(...next.findings); return findings; }
    await client.callTool({ name: next.tool, arguments: next.args });       // Act
    // Evaluate happens inside claudeDecide on the next loop iteration
  }
}
// Use for exploratory testing. NEVER for regression: non-deterministic by design.
```

Q 297 LEVEL · Mid to Senior

How do you test AI-powered features using Playwright TypeScript?

- Assert structure, not exact text: category + schema + relevance keywords.
- Use TS interfaces for AI response schemas: structural contract at type level.
- TS catches schema-change drift at compile time; JS teams catch it at runtime.
- Generous timeouts (~15s) for AI-powered visibility assertions.
- They usually ask next: assert category/schema/keywords, never exact text.
- The common miss: asserting exact generated wording, test flakes on every prompt nudge.

```
test('AI chatbot handles billing query', async ({ request }) => {
  const res = await request.post('/api/chat', {
    data: { message: 'How do I get a refund?', sessionId: `t-${Date.now()}` },
  });
  expect(res.status()).toBe(200);

  interface ChatResponse {
    reply: string; category: string; confidence: number; processingTimeMs: number;
  }
  const body = await res.json() as ChatResponse;

  // Assert structure, not exact text
  expect(body.category).toBe('billing');
  const relevant = ['refund', 'return', 'policy', 'contact', 'days'];
  expect(relevant.some(kw => body.reply.toLowerCase().includes(kw))).toBe(true);

  expect(body.reply.length).toBeGreaterThan(50);
  expect(body.reply.length).toBeLessThan(2000);
  expect(body.confidence).toBeGreaterThanOrEqual(0);
  expect(body.confidence).toBeLessThanOrEqual(1);
...
```

Q 298 LEVEL · Senior

What is model drift and how do you detect it with typed Playwright TypeScript tests?

- Drift = AI behaviour changing over time (category, wording, timing).
- DriftScenario interface: **input**, **expected category**, **must/must-not contain**, **timeout**.
- Dataset 'as const' = literal types + readonly: no accidental mutation.
- Run nightly on cron; any failure alerts the team before users notice.
- The next question: 'as const' gives literal types and immutability, schema-safe.
- Avoid: detecting drift only after users complain, trust is already lost.

```
import { test, expect } from '@playwright/test';

interface DriftScenario {
  readonly input: string;
  readonly expectedCategory: string;
  readonly mustContain: readonly string[];
  readonly mustNotContain: readonly string[];
  readonly maxResponseTimeMs: number;
}
interface ChatApiResponse { reply: string; category: string; processingTimeMs: number; }

const DATASET: readonly DriftScenario[] = [
  { input: 'How do I cancel my subscription?', expectedCategory: 'billing',
    mustContain: ['cancel','subscription'], mustNotContain: ['error'], maxResponseTimeMs: 5000 },
  { input: "My order hasn't arrived after 10 days", expectedCategory: 'shipping',
    mustContain: ['order','delivery','track'], mustNotContain: ['error'], maxResponseTimeMs: 5000 },
] as const;

for (const s of DATASET) {
  test(`Drift: "${s.input.slice(0, 40)}…"`, async ({ request }) => {
...
```

Q 299 LEVEL · Senior

How do you build a TypeScript self-healing agent for Playwright test maintenance?

- HealingSuggestion interface with locatorType union: getByRole/Label/TestId/locator.
- Include the interface in the prompt to constrain the LLM's valid outputs.
- Runtime type guard (isHealingSuggestion) rejects hallucinated shapes.
- Policy: high-confidence getByRole auto-applies; low-confidence routes to human.
- Asked next: what stops bad locators, union in prompt + type guard.
- The easy mistake: auto-applying every healing suggestion regardless of confidence.

```
import { test as base } from '@playwright/test';
import Anthropic from '@anthropic-ai/sdk';

interface HealingSuggestion {
  locatorType: 'getByRole' | 'getByLabel' | 'getByTestId' | 'locator';
  locatorValue: string;
  confidence: 'high' | 'medium' | 'low';
  explanation: string;
}
function isHealingSuggestion(o: unknown): o is HealingSuggestion {
  if (typeof o !== 'object' || o === null) return false;
  const s = o as Record<string, unknown>;
  return ['getByRole','getByLabel','getByTestId','locator'].includes(s.locatorType as string)
      && typeof s.locatorValue === 'string'
      && ['high','medium','low'].includes(s.confidence as string);
}

export const test = base.extend<{ healingPage: typeof base }>({
  healingPage: async ({ page }, use) => {
    const client = new Anthropic();
...
```

Q 300 LEVEL · Senior

How do you use AI in CI quality gates with typed structured output?

FailureCategory union: APPLICATION_BUG / TEST_BUG / ENVIRONMENT_ISSUE / NEEDS_INVESTIGATION. CIFailureAnalysis interface in the LLM prompt constrains valid output. Runtime type guard rejects malformed JSON before it reaches the pipeline. Deterministic action mapping: BLOCK / WARN / PASS, plus human on NEEDS_INVESTIGATION. Concrete ROI: ~80% reduction in manual PR triage, justifies investment. Trap: unconstrained LLM output making deployment decisions directly.

```
type FailureCategory = 'APPLICATION_BUG' | 'TEST_BUG' | 'ENVIRONMENT_ISSUE' | 'NEEDS_INVESTIGATION';
interface CIFailureAnalysis {
  category: FailureCategory;
  confidence: number;
  affectedTests: string[];
  deployRecommendation: 'BLOCK' | 'WARN' | 'PASS';
  reasoning: string;
}
function isCIFailureAnalysis(o: unknown): o is CIFailureAnalysis {
  if (typeof o !== 'object' || o === null) return false;
  const a = o as Record<string, unknown>;
  const cats: FailureCategory[] = ['APPLICATION_BUG','TEST_BUG','ENVIRONMENT_ISSUE','NEEDS_INVESTIGATION'];
  return cats.includes(a.category as FailureCategory)
      && typeof a.confidence === 'number'
      && ['BLOCK','WARN','PASS'].includes(a.deployRecommendation as string);
}

export async function analyseFailures(failures: unknown[]): Promise<CIFailureAnalysis> {
  const client = new Anthropic();
  const r = await client.messages.create({
...
```

QUICK QUIZ

1. What is the TypeScript-only advantage for AI-generated Playwright tests?
   A) TS is faster than JS at runtime
   B) TS removes the need for any testing
   C) tsc --noEmit catches hallucinated methods at compile time, free first filter
   D) Claude only works with TS
2. What is MCP, in one sentence a senior engineer would accept?
   A) An open protocol (modelcontextprotocol.io) connecting AI to tools/data/systems, widely adopted
   B) An Anthropic-only API for Claude
   C) A replacement for WebDriver
   D) A Playwright plugin
3. What transport does MCP use for remote servers in 2026?
   A) Server-Sent Events (SSE)
   B) gRPC
   C) WebSockets only
   D) Streamable HTTP: SSE was deprecated in March 2025
4. Why prefer browser_snapshot over browser_take_screenshot for an AI agent?
   A) Snapshot is prettier
   B) Snapshot is the only option available
   C) Screenshot is broken
   D) Snapshot returns a structured accessibility tree, semantic, no vision model needed; faster, cheaper
5. How do you keep AI safe in a CI deployment gate?
   A) Let the AI decide deployment freely
   B) Constrain output via a FailureCategory union type + deterministic action mapping (BLOCK/WARN/PASS) + human on NEEDS_INVESTIGATION
   C) Disable the gate entirely
   D) Trust the LLM confidence score

Answers: 1:C 2:A 3:D 4:D 5:B

Real-Time Interview Scenarios

Q 301 LEVEL · Senior

Design a Playwright TypeScript framework from scratch for a fintech application with 5 user roles.

- Four layers: **configuration**, **auth**, **framework**, **CI/organisation**.
- globalSetup -> one storageState per role: eliminates login overhead.
- BasePage + components + single fixtures file + faker data factories.
- CI: typecheck -> lint -> 4 shards x 4 workers, traces + JUnit XML.
- Follow-up: scaling 300+ tests is storageState + fixtures + sharding.
- Where this goes wrong: diving into one layer and missing the whole-system view.

Q 302 LEVEL · Mid to Senior

Your Playwright tests pass locally but fail in CI. Walk through your complete diagnosis process.

1. 8-step structured process: the structure itself is the experience marker.
2. Trace download is highest-leverage: answers most questions in 5 min.
3. Compare browser versions, headless mode, env vars, resource constraints.
4. Workers count exposes shared state; CI is slower so kill waitForTimeout.
5. Expect next: trace is fastest, shows DOM + network at exact failure point.
6. What loses the point: guessing the cause before downloading and reading the trace.

```
# Step 2: the trace is your fastest diagnostic
# 1. Download from CI artifacts
# 2. open https://trace.playwright.dev
# 3. Scrub timeline -> see DOM + network at every action

# Step 6: isolate shared-state bugs by varying workers
npx playwright test --workers=1   # passes? -> shared state with parallel
npx playwright test --workers=4   # reproduce CI condition locally
```

Q 303 LEVEL · Senior

How would you test a real-time chat application?

Two browser contexts: separate storageState per user. Generous timeout on receiver assertion (~10s for WS round-trip). page.on('websocket') for frame-level monitoring (framesent/received). Use route() to drop WS traffic and test the reconnecting UI. Pattern scales to notifications, collaborative editing, presence. The common miss: one context with two pages, same cookies, not two users.

```
test('message from user A appears for user B', async ({ browser }) => {
  const ctxA = await browser.newContext({ storageState: 'auth/user-a.json' });
  const ctxB = await browser.newContext({ storageState: 'auth/user-b.json' });
  const pageA = await ctxA.newPage();
  const pageB = await ctxB.newPage();
  await Promise.all([pageA.goto('/chat/room/123'), pageB.goto('/chat/room/123')]);

  await pageA.getByLabel('Message').fill('Hello from User A');
  await pageA.getByRole('button', { name: 'Send' }).click();

  await expect(pageB.locator('.message-bubble').last())
    .toHaveText('Hello from User A', { timeout: 10_000 });

  await Promise.all([ctxA.close(), ctxB.close()]);
});

// WebSocket frame monitoring
page.on('websocket', ws => {
  ws.on('framesent', f => console.log('sent:', f.payload));
  ws.on('framereceived', f => console.log('recv:', f.payload));
...
```

Q 304 LEVEL · Senior

How do you reduce a 2-hour Playwright suite to under 20 minutes?

Five phases: **audit**, **setup**, **parallelism**, **test design**, **infrastructure**. Audit first with --reporter=json: don't guess where time goes. Setup migration (UI -> API/storageState) = 30-40% alone. Sharding math: 120min ÷ 8 shards ÷ 4 workers ≈ 4min/shard. They usually ask next: highest-impact change is moving setup to API. Avoid: parallelising before fixing slow setup, just parallelises waste.

```
# Phase 1: audit
npx playwright test --reporter=json | tee results.json
node scripts/find-slowest.js results.json   # -> top 20 slowest

// Phase 2: UI setup -> API/storageState
// before: 30s of UI user creation per test
// after:  200ms API call in a fixture, storageState for the session

// Phase 3: parallelism + sharding (CI)
//   playwright.config.ts:  fullyParallel: true, workers: 4
//   CI matrix:             shard 1/8 … 8/8

// Phase 4: quarantine genuinely slow tests
test.slow();      // marks this test as slow -> nightly project
```

Q 305 LEVEL · Senior

A critical payment flow test is flaky: fails 1 in 10 runs. How do you debug it?

Collect 20 traces in a loop: compare passing vs failing side by side. Check iframe availability, network timing, element stability, retry pattern. Retry-1-only = environment warmup; fix with worker-scoped warmup fixture. Random = race; common one: iframe field visible but not yet editable. The next question: **wait on editable state**, **not just visible**, **for iframe fields**. The easy mistake: bumping timeout to mask the race rather than fixing it.

```
// Step 1: collect traces in a loop
test.use({ trace: 'on' });
// then: npx playwright test payment.spec.ts --repeat-each=20

// Step 4 fix: wait on editable, not just visible
const cardFrame = page.frameLocator('#stripe-card-iframe');
await cardFrame.getByLabel('Card Number').waitFor({ state: 'visible' });
await cardFrame.getByLabel('Card Number').waitFor({ state: 'editable' });
await cardFrame.getByLabel('Card Number').fill('4111111111111111');
```

Q 306 LEVEL · Senior

How do you build a Playwright framework that multiple teams can contribute to without breaking each other?

Three layers with ownership: core (platform team), domain + tests (feature teams). Platform team only gates core changes; feature teams ship independently. Core published as internal semver-versioned npm package. Feature teams pin and upgrade on their schedule; breaking = major bump. Asked next: prevent breaks with semver + migration guides + pinned versions. Trap: one shared codebase everyone edits, conflicts and broken contracts.

Q 307 LEVEL · Senior

How would you test an admin panel of a multi-tenant SaaS?

Generate storageState per role per tenant in globalSetup. Test functionality AND isolation: isolation is the critical half. Attempt cross-tenant access via direct URL AND direct API. Expect 403/404; never the other tenant's data. Follow-up: direct URL/API are what matter, attackers don't use the UI. Where this goes wrong: testing only functional happy paths and missing data leakage.

```
test('tenant A admin cannot access tenant B data', async ({ browser }) => {
  const ctx = await browser.newContext({ storageState: 'auth/admin-tenant-a.json' });
  const page = await ctx.newPage();

  // 1. Direct URL access attempt
  await page.goto('/admin/tenants/tenant-b/users');
  await expect(page).toHaveURL(/\/403|\/unauthorized/);

  // 2. Direct API access attempt: enforced separately from the UI router
  const res = await page.request.get('/api/tenants/tenant-b/users');
  expect(res.status()).toBe(403);
});
```

Q 308 LEVEL · Senior

Design a visual regression test strategy for a design system.

Three layers: component (per PR), page (nightly), cross-browser. Component layer uses Playwright CT: every variant snapshotted. Page layer masks dynamic content; designer review before baseline update. Per-browser baselines isolate real regressions from engine differences. Expect next: rendering differs by engine, shared baseline = constant noise. What loses the point: one global baseline that fails on every Chromium update.

```
// Layer 1: component (Playwright CT, every PR)
test('Button: primary variant', async ({ mount }) => {
  const c = await mount(<Button variant="primary">Save</Button>);
  await expect(c).toHaveScreenshot('button-primary.png');
});

// Layer 2: page (nightly, dynamic content masked)
await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.locator('[data-test="user-avatar"]'), page.locator('time')],
});

// Layer 3: per-browser baselines
// snapshots/dashboard-chromium.png
// snapshots/dashboard-firefox.png
// snapshots/dashboard-webkit.png
```

Q 309 LEVEL · Mid

How do you test Canvas or WebGL content?

Canvas/WebGL is pixels, not DOM: no getByRole possible. Visual: toHaveScreenshot on the canvas element. Interaction: page.mouse with coords; use boundingBox for layout-relative. Data: page.evaluate when the chart library exposes state (e.g. Chart.js). They usually ask next: assert chart values via page.evaluate, robust to styling. The common miss: trying DOM-based locators on canvas, nothing to find.

```
test('canvas drawing tool draws a line', async ({ page }) => {
  await page.goto('/drawing-tool');
  const canvas = page.locator('#drawing-canvas');
  const box = await canvas.boundingBox();
  if (!box) throw new Error('canvas not found');

  // Interaction: mouse coords relative to the canvas
  await page.mouse.move(box.x + 50,  box.y + 100);
  await page.mouse.down();
  await page.mouse.move(box.x + 200, box.y + 100);
  await page.mouse.up();

  // Visual: capture rendered pixels
  await expect(canvas).toHaveScreenshot('line-drawn.png');
});

// Data: read chart state when exposed by the library
const datasets = await page.evaluate(() => (window as any).myChart.data.datasets);
expect(datasets[0].data[0]).toBe(42);
```

Q 310 LEVEL · Senior

How would you migrate 500 Selenium tests to Playwright?

Phased over 3-4 sprints: NEVER a big-bang rewrite. Run both frameworks in CI in parallel during transition. Phase 2: migrate the 50 most-flaky tests first to prove value. Migrate by feature area; delete StaleElementReferenceException handlers. The next question: measure flakiness before/after first 50, sells the rest. Avoid: big-bang rewrite that loses coverage during transition.

Q 311 LEVEL · Mid

How do you test infinite scroll pagination?

Capture initial count; explicit scrollTo bottom; capture new count. Wait for loading indicator: visible THEN hidden, both matter. Visible-only races completion; hidden-only races the scroll. Pattern generalises: any transient async indicator (spinners, uploads). Asked next: both transitions because each alone has a race. The easy mistake: **waitForTimeout**, **too short flakes**, **too long is slow**.

```
test('infinite scroll loads more items', async ({ page }) => {
  await page.goto('/products');
  const initial = await page.locator('.product-item').count();

  await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight));

  // appear-then-disappear: deterministic
  await page.locator('#loading-more').waitFor({ state: 'visible' });
  await page.locator('#loading-more').waitFor({ state: 'hidden' });

  const after = await page.locator('.product-item').count();
  expect(after).toBeGreaterThan(initial);
});
```

Q 312 LEVEL · Mid

How do you test drag and drop interactions?

Try dragTo first: handles standard HTML5 drag-and-drop. Verify item IN target AND OUT of source after the drop. Custom drag libs (react-dnd): fall back to mouse.move/down/up + boundingBox. File drag-and-drop: DataTransfer via page.evaluateHandle + dispatch drop. Follow-up: dragTo fails -> manual mouse sequence with incremental moves. Trap: reaching for mouse events first when dragTo would work.

```
test('drag item to reorder list', async ({ page }) => {
  await page.goto('/kanban-board');
  const card    = page.locator('.card').filter({ hasText: 'Task A' });
  const target  = page.locator('.column').filter({ hasText: 'In Progress' });
  const source  = page.locator('.column').filter({ hasText: 'Backlog' });

  await card.dragTo(target);                                  // simple case

  await expect(target.locator('.card').filter({ hasText: 'Task A' })).toBeVisible();
  await expect(source.locator('.card').filter({ hasText: 'Task A' })).toBeHidden();
});

// Fallback for react-dnd / custom drag handlers
// const s = await source.boundingBox(); const t = await target.boundingBox();
// await page.mouse.move(s!.x + s!.width/2, s!.y + s!.height/2);
// await page.mouse.down();
// await page.mouse.move(t!.x + t!.width/2, t!.y + t!.height/2, { steps: 10 });
// await page.mouse.up();
```

Q 313 LEVEL · Mid

How do you test file upload and download flows?

Upload: setInputFiles for standard inputs; DataTransfer for drag-drop. Download: Promise.all([waitForEvent('download'), click]), listener first. Validate suggested filename, save to temp, assert size or content. Large downloads: size-only; content-critical: read and parse. Expect next: Promise.all registers listener BEFORE click fires the event. Where this goes wrong: click first, listen second, the download event is already gone.

```
// Upload: standard input
await page.getByLabel('Choose file').setInputFiles('tests/fixtures/report.pdf');
// Or by buffer:
// await input.setInputFiles({ name: 'r.pdf', mimeType: 'application/pdf', buffer: buf });

// Download: listener-before-click via Promise.all
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export CSV' }).click(),
]);
expect(download.suggestedFilename()).toBe('orders-export.csv');
const path = await download.path();
expect(fs.statSync(path!).size).toBeGreaterThan(0);

// Content-critical: parse and assert specific values
const csv = fs.readFileSync(path!, 'utf8');
expect(csv.split('\n')[0]).toBe('order_id,customer_email,total');
```

Q 314 LEVEL · Mid

How do you test internationalisation (i18n) across multiple locales?

test.use({ locale }) at describe-block level: self-documenting structure. Pair with test.use({ timezoneId }) for date-formatting tests. Assert translated strings from a locale fixture, not hardcoded literals. Test locale-dependent formatting (dates, numbers, currencies) separately. They usually ask next: timezone affects date rendering independently of locale. What loses the point: hardcoded translations in tests, one tweak breaks many tests.

```
// Locale block: self-documenting
test.describe('French locale: fr-FR', () => {
  test.use({ locale: 'fr-FR', timezoneId: 'Europe/Paris' });

  test('welcome page renders in French', async ({ page }) => {
    await page.goto('/');
    await expect(page.getByRole('heading'))
      .toHaveText(locales.frFR.welcomeMessage); // from fixture, not hardcoded
  });

  test('dates render in DD/MM/YYYY format', async ({ page }) => {
    await page.goto('/orders/123');
    await expect(page.locator('[data-test="order-date"]'))
      .toHaveText('31/10/2026'); // French format
  });
});
```

Q 315 LEVEL · Mid

How do you test dark mode and other CSS theme variants?

test.use({ colorScheme: 'dark' }) sets prefers-color-scheme media query. Separate dark-mode baselines (dashboard-dark.png, never shared). page.evaluate reads :root CSS custom properties for token assertions. Test the theme switcher: persistence in cookie/localStorage. The next question: verify tokens via CSS custom properties, exact, not pixel-based. The common miss: one baseline across themes, fails on every theme test.

```
test.describe('Dark mode', () => {
  test.use({ colorScheme: 'dark' });

  test('dashboard renders in dark mode', async ({ page }) => {
    await page.goto('/dashboard');
    await expect(page).toHaveScreenshot('dashboard-dark.png'); // separate baseline
  });

  test('CSS custom properties match dark tokens', async ({ page }) => {
    await page.goto('/');
    const tokens = await page.evaluate(() => {
      const s = getComputedStyle(document.documentElement);
      return {
        bg:   s.getPropertyValue('--color-bg').trim(),
        text: s.getPropertyValue('--color-text').trim(),
      };
    });
    expect(tokens.bg).toBe('#0f172a');
    expect(tokens.text).toBe('#f1f5f9');
  });
...
```

Q 316 LEVEL · Mid

How do you test flows that require browser permissions (geolocation, notifications, camera)?

context.grantPermissions per context (not per page). Set explicit coordinates with context.setGeolocation for determinism. Test BOTH the granted and denied paths. Use context.clearPermissions for the denied path. Asked next: denied-path tests catch real crashes when users block. Avoid: testing only the happy granted path.

```
// Granted path: deterministic location
test('shows nearby stores when geolocation granted', async ({ browser }) => {
  const ctx = await browser.newContext();
  await ctx.grantPermissions(['geolocation']);
  await ctx.setGeolocation({ latitude: 51.5074, longitude: -0.1278 }); // London
  const page = await ctx.newPage();
  await page.goto('/nearby');
  await expect(page.locator('.store-card').first()).toContainText('London');
});

// Denied path: the test most candidates skip
test('shows manual entry when geolocation denied', async ({ browser }) => {
  const ctx = await browser.newContext();
  await ctx.clearPermissions(); // ensure no permission
  const page = await ctx.newPage();
  await page.goto('/nearby');
  await expect(page.getByText('Enter your postcode')).toBeVisible();
});
```

Q 317 LEVEL · Mid to Senior

How do you test Progressive Web App (PWA) offline mode?

Three-phase lifecycle: online-cache -> offline-render -> online-sync. context.setOffline(true) to simulate network loss. Verify cached pages render AND pending actions queue with indicator. context.setOffline(false): verify queue syncs and indicator clears. Follow-up: **sync queue is hardest**, **lost actions**, **duplicates**, **ordering**. The easy mistake: testing only offline-render and missing the sync-on-reconnect.

```
test('PWA syncs queued actions on reconnect', async ({ browser }) => {
  const ctx = await browser.newContext();
  const page = await ctx.newPage();

  // 1. Online: service worker registers and caches
  await page.goto('/blog');
  await page.waitForFunction(() => navigator.serviceWorker.controller);

  // 2. Offline: cached page renders + action queues
  await ctx.setOffline(true);
  await page.reload();
  await expect(page.locator('h1')).toBeVisible(); // from cache
  await page.getByLabel('Comment').fill('Great post');
  await page.getByRole('button', { name: 'Post' }).click();
  await expect(page.locator('.sync-pending')).toBeVisible();

  // 3. Online: queue syncs, indicator clears
  await ctx.setOffline(false);
  await expect(page.locator('.sync-pending')).toBeHidden();
  // and verify the comment appears server-side on reload
...
```

Q 318 LEVEL · Mid to Senior

How do you test integrations with third-party APIs (Stripe, Twilio, SendGrid)?

NEVER call real third-party APIs in regular tests (cost, rate limits, PCI). Use page.route to mock both success and specific failure responses. Test failure responses the API actually returns: **402**, **429**, **503**, **malformed**. Add nightly contract tests against the third party's sandbox. Expect next: nightly contract tests catch API drift mocks would miss. Trap: only mocking (silent drift) OR calling real APIs in tests (cost/flake).

```
// Mocked test: deterministic failure response
test('checkout shows error when Stripe returns 402', async ({ page }) => {
  await page.route('**/api/payments', route => route.fulfill({
    status: 402,
    body: JSON.stringify({ error: 'insufficient_funds' }),
  }));
  await page.goto('/checkout');
  await page.getByRole('button', { name: 'Place Order' }).click();
  await expect(page.locator('.error')).toContainText('insufficient funds');
});

// Nightly contract test: against Stripe sandbox
// .github/workflows/contract-tests.yml schedules: "0 2 * * *"
// Hits api.stripe.com with test keys; catches API drift mocks miss.
```

Q 319 LEVEL · Mid to Senior

How do you implement automated accessibility testing in Playwright?

@axe-core/playwright for programmatic WCAG audits, inject + analyze. Fail on critical/serious violations; gate every PR. toMatchAriaSnapshot catches structural regressions axe might miss. page.keyboard.press for tab order, focus visibility, keyboard reachability. They usually ask next: automation catches ~50%, manual + assistive tech still essential. Where this goes wrong: claiming axe catches all a11y issues, it doesn't.

```
import AxeBuilder from '@axe-core/playwright';

test('checkout has no critical accessibility violations', async ({ page }) => {
  await page.goto('/checkout');
  const results = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa'])
    .analyze();
  const critical = results.violations.filter(v => ['critical','serious'].includes(v.impact ?? ''));
  expect(critical).toEqual([]);
});

test('navigation ARIA tree is stable', async ({ page }) => {
  await page.goto('/');
  await expect(page.locator('nav')).toMatchAriaSnapshot();
});

test('keyboard navigation cycles through interactive elements', async ({ page }) => {
  await page.goto('/checkout');
  await page.keyboard.press('Tab');
  await expect(page.getByLabel('Card number')).toBeFocused();
...
```

Q 320 LEVEL · Senior

How do you prove test coverage to stakeholders in a way they can act on?

Three artefacts: **traceability matrix**, **risk matrix**, **trend dashboard**. Tag every test with @REQ-ID; script parses results into requirement coverage. Risk matrix justifies asymmetric investment (payment 50 tests, about page 2). Report in business language; PMs understand 'REQ-103 has no test'. The next question: 'no test at all' is the actionable column. What loses the point: reporting code-coverage percentages to product, misleading.

```
// Requirement-ID tags drive traceability
test('user logs in', { tag: ['@REQ-101', '@smoke'] }, async ({ page }) => { /* ... */ });
test('invalid password error', { tag: ['@REQ-102', '@regression'] }, async ({ page }) => { /* ... */ });

// Script generates a stakeholder dashboard
//   npx playwright test --reporter=json | node scripts/coverage-report.js
//
// Output:
//   Requirement   Status            Tests        Owner
//   REQ-101       ✓ covered (pass)  3            Payments
//   REQ-102       ✗ covered (fail)  1            Auth
//   REQ-103       ⚠ NO TEST         0            Reports   <- actionable gap
```

Q 321 LEVEL · Mid to Senior

When and how do you combine API setup with UI verification?

Pattern: API setup -> UI verify -> API teardown, fastest reliable form. Speed math: 30s UI user creation -> 200ms API; 30min -> 5min for 50 tests. UI for what only UI tests: **rendering**, **interaction**, **validation messages**. API for setup/teardown/preconditions: it's already covered elsewhere. Asked next: UI tests the user-facing behaviour, API tests the data path. The common miss: doing all setup via UI 'for realism', slow with no coverage gain.

```
test('admin sees user order history', async ({ page, request }) => {
  // Setup via API: fast and deterministic
  const user = await new UserBuilder().asAdmin().create(request);
  const order = await new OrderBuilder().forUser(user.id).create(request);

  // Verify via UI: only the UI can test this
  await page.goto(`/admin/users/${user.id}/orders`);
  await expect(page.locator('.order-row').first()).toContainText(order.reference);
  await expect(page.locator('.order-total')).toHaveText(`£${order.total}`);

  // Teardown via API
  await request.delete(`/api/orders/${order.id}`);
  await request.delete(`/api/users/${user.id}`);
});
```

Q 322 LEVEL · Senior

How do you handle tests against an unstable or flaky test environment?

Distinguish environment flakiness from test flakiness, different owners. globalSetup health check fails fast on broken environment with clear message. Retries config for whole-test; withRetry utility for specific operations. Separate dashboards: platform team owns env, test author owns test. Follow-up: health check + isolation run distinguishes the two. Avoid: treating all flakiness as test bugs; fixes go to wrong people.

```
// globalSetup: fail fast on broken environment
export default async function globalSetup() {
  const r = await fetch(`${process.env.BASE_URL}/api/health`);
  if (!r.ok) throw new Error(`ENV UNHEALTHY: /api/health returned ${r.status}. Stopping suite.`);
  // Verify test users exist, DB reachable, etc.
}

// playwright.config.ts: retries for transient infra issues
// retries: process.env.CI ? 2 : 0

// Separate metrics dashboards
//   env-flakiness dashboard      -> platform-team owns it
//   test-flakiness dashboard     -> test author owns it
```

Q 323 LEVEL · Senior

How do you test advanced WebSocket scenarios (disconnection, reconnection, frame inspection)?

Three scenarios: **frame inspection**, **disconnection**, **reconnection**. page.on('websocket') + ws.on('framesent'/'framereceived') for frames. Disconnection via page.route abort: verify reconnecting UI + message queue. Reconnection via unroute: verify re-establish + queued send + state sync. Expect next: simulate disconnection by aborting the WebSocket URL via route. The easy mistake: testing only happy-path message delivery, missing recovery bugs.

```
// Frame inspection
page.on('websocket', ws => {
  ws.on('framesent', f => sentFrames.push(f.payload));
  ws.on('framereceived', f => receivedFrames.push(f.payload));
});
await page.goto('/chat');
expect(sentFrames.some(p => p.toString().includes('subscribe'))).toBe(true);

// Disconnection: abort the WebSocket URL after connection
await page.route('**/ws/**', route => route.abort());
await expect(page.locator('.connection-status')).toContainText('Reconnecting');

// Reconnection: restore the route
await page.unroute('**/ws/**');
await expect(page.locator('.connection-status')).toContainText('Connected');
```

Q 324 LEVEL · Mid

How do you onboard new team members to a Playwright framework so they ship quickly?

Progressive complexity: run -> read -> pair -> solo -> contribute. Day 1 run, Day 2 read 10 tests, Day 3 pair, Week 2 solo, Week 3 contribute. CONTRIBUTING.md (locator priority, fixtures, review checklist) for self-serve. Measure time-to-first-merged-test: a good program lands ~4 days. They usually ask next: the metric tells you when an onboarding change regressed. Trap: **ad-hoc onboarding**, **doesn't scale**, **can't be improved**.

How do you test a feature flag rollout (enabled, disabled, gradual percentage)?

Three scenarios: flag on, flag off, boundary (percentage rollout). Override deterministically (cookie/header/query/API), not via rollout %. Flag-off testing is critical: off-path break = regression for majority. Per-user override lets you test both sides of percentage boundaries. Centralise overrides in a withFeatureFlag fixture, one place to change. Where this goes wrong: testing only the new behaviour with the flag on.

```
// Centralised override fixture
export const test = base.extend<{ withFeatureFlag: (name: string, on: boolean) => Promise<void> }>({
  withFeatureFlag: async ({ context }, use) => {
    await use(async (name, on) => {
      await context.addCookies([{
        name: `ff_${name}`, value: String(on),
        domain: 'localhost', path: '/',
      }]);
    });
  },
});

test('new checkout flow works when flag is ON', async ({ page, withFeatureFlag }) => {
  await withFeatureFlag('new-checkout', true);
  await page.goto('/checkout');
  await expect(page.getByTestId('new-checkout-form')).toBeVisible();
});

test('old checkout flow works when flag is OFF', async ({ page, withFeatureFlag }) => {
  await withFeatureFlag('new-checkout', false);
...
```

QUICK QUIZ

1. What's the highest-leverage step when a Playwright test passes locally but fails in CI?
   A) Add a longer timeout
   B) Download the trace from CI and scrub the timeline at trace.playwright.dev
   C) Disable parallelism
   D) Roll back the last commit
2. Why use two browser contexts when testing a real-time chat application?
   A) It runs faster
   B) Chat apps require it by spec
   C) Separate storageState per user: one context with two pages shares cookies and is one user
   D) It avoids using WebSockets
3. What is typically the single highest-impact change when reducing a slow Playwright suite?
   A) Increasing parallelism first
   B) Removing all assertions
   C) Moving UI-based test data setup to API calls / storageState (often 30-40% alone)
   D) Switching browsers
4. Why must multi-tenant isolation tests hit the API directly, not just the UI?
   A) APIs are faster
   B) APIs always return 200
   C) It avoids using cookies
   D) UI and API enforce access separately; the UI may block tenant B's URLs while the API leaks tenant B's data
5. How should a 500-test Selenium-to-Playwright migration be approached?
   A) Phased over 3-4 sprints, both frameworks running in parallel, flakiest tests migrated first
   B) Big-bang rewrite over one sprint
   C) Migrate alphabetically
   D) Migrate only when forced

Answers: 1:B 2:C 3:C 4:D 5:A

Final Mock Quiz

Test yourself across the whole kit

1. Which tsconfig setting catches the most type bugs in test code without exhaustive per-rule configuration?
   A) noImplicitAny
   B) strict
   C) skipLibCheck
   D) esModuleInterop
2. In `function pick<T, K extends keyof T>(obj: T, key: K): T[K]`, what does `K extends keyof T` enforce?
   A) K can be any string
   B) K must be a key that actually exists on T
   C) K must be a string literal
   D) K must be readonly
3. Why prefer `interface` over `type` for API response shapes that the tests will extend frequently?
   A) Interfaces support declaration merging; `type` does not
   B) Interfaces are faster at runtime
   C) Interfaces auto-import; `type` does not
   D) Interfaces support union types; `type` does not
4. Which locator strategy should you reach for FIRST when the element is interactive?
   A) getByRole
   B) getByTestId
   C) getByText
   D) CSS selector
5. A test calls `page.getByRole('button').click()` and Playwright throws a strict mode violation. What's the right fix?
   A) Add `.first()`
   B) Set `strict: false` in the config
   C) Switch to `page.locator('button').click()`
   D) Add an accessible-name filter via `{ name: '...' }`
6. What's the difference between `locator.filter({ has })` and `locator.filter({ hasText })`?
   A) `has` filters by descendant locator; `hasText` filters by text content
   B) There is no difference
   C) `has` is for arrays; `hasText` is for strings
   D) `has` was deprecated in 2025
7. Before Playwright clicks an element, which actionability checks does it run?
   A) Only that the element exists
   B) Visible, stable, receives events and enabled
   C) All five checks, including editable, before every action
   D) Visible only; events are user-driven
8. A test uses `await page.waitForTimeout(2000)` to wait for a modal to appear. What's the right replacement?
   A) Increase to 5000ms for safety
   B) Use `await page.evaluate(() => new Promise(r => setTimeout(r, 2000)))`
   C) Use `await page.waitForLoadState('networkidle')`
   D) Replace with `expect(modal).toBeVisible()`
9. When is a Page Object NOT the right abstraction?
   A) A multi-step flow used by many tests
   B) Pages with dynamic loading
   C) Complex form with conditional fields
   D) A one-off verification of a specific button on a single test
10. Which assertion auto-retries until the condition holds or the timeout fires?
    A) expect(el).toBeVisible()
    B) expect(await el.isVisible()).toBe(true)
    C) expect(el.count()).toBeGreaterThan(0)
    D) assert(el.isVisible())
11. To assert a list is non-empty in a parallel-safe, race-free way:
    A) expect(await items.count()).toBeGreaterThan(0)
    B) expect(items).not.toHaveCount(0)
    C) expect(await items.allInnerTexts()).not.toEqual([])
    D) await items.first().waitFor()

12. Which Playwright unit provides full state isolation between tests by default?
    A) BrowserContext
    B) Browser
    C) Page
    D) Worker

13. You want to mock a specific endpoint for one test only. Which scope is right?
    A) browser.route() at config level
    B) page.route() inside the test
    C) context.route() in beforeAll
    D) Intercept via service worker

14. Why use APIRequestContext for test setup instead of driving the UI?
    A) It's 10x+ faster than UI flows and doesn't depend on form/field stability
    B) It's the only way to send authenticated requests
    C) It shares cookies with page.route mocks
    D) It bypasses Playwright's auto-waiting

15. An API SLA test fails intermittently because response times occasionally exceed the 500 ms threshold. The actual p95 is 300 ms. Best fix?
    A) Lower the threshold to 300 ms
    B) Retry the test 5 times
    C) Disable the assertion
    D) Set the threshold above the worst-case p99 (e.g. 800 ms) to absorb variance

16. On a 4-vCPU CI runner with 7 GB RAM, what's typically the right Playwright worker count?
    A) 4 (= cores)
    B) 6-8 (more than cores; browser tests are I/O-bound)
    C) 16 (max parallelism)
    D) 1 (avoid contention)

17. What's the relationship between Playwright workers and CI shards?
    A) They're the same thing
    B) Workers parallelise within one CI job; shards parallelise across CI jobs
    C) Shards are deprecated in 2025+
    D) Workers run sequentially; shards run in parallel

18. Which environment variable shows every Playwright API call with timing, useful for understanding why a CI run failed?
    A) PWDEBUG=1
    B) PLAYWRIGHT_TRACE=verbose
    C) DEBUG=pw:protocol
    D) DEBUG=pw:api

19. You need to seed a test database once per worker, not per test. Which fixture scope is right?
    A) Default test scope
    B) scope: 'browser'
    C) scope: 'worker'
    D) scope: 'global'

20. In the Meszaros test-doubles taxonomy, the distinction between a 'stub' and a 'mock' is:
    A) Stubs are real; mocks are fake
    B) Stubs control returned state; mocks verify behaviour (that calls happened as expected)
    C) Stubs are for HTTP; mocks are for databases
    D) There is no distinction; the terms are synonymous

21. Before trusting an LLM-as-judge assertion in CI, what's the minimum calibration step?
    A) Run it once and check the answer looks right
    B) Use a more expensive model
    C) Hand-label ~50 examples and confirm 80%+ agreement between the judge and humans
    D) Add 'be strict' to the prompt

22. What's the right use for `testInfo.annotations` vs `testInfo.attach`?
    A) They are the same API
    B) Annotations for HTML report; attach for JSON report
    C) Annotations for tags/metadata; attach for files and binary diagnostic data
    D) Annotations are deprecated; use attach

23. A feature has 5 browsers × 4 plans × 3 countries × 4 payment methods = 240 combinations. Pairwise testing reduces this to roughly:
    A) 5 cases

B) 60 cases
C) 20 cases
D) 120 cases

24. Which of these is the strongest single quality metric to report to leadership?
A) Defect escape rate (bugs reaching production)
B) Total test count
C) Code coverage percentage
D) Number of passing tests

25. Playwright's iPhone device emulation sets viewport, user-agent, and touch flags. Which gap matters most for catching iOS-specific bugs?
A) Playwright doesn't emulate the screen brightness
B) Touch events aren't supported in emulation
C) iPhone emulation lacks 5G network simulation
D) Playwright's WebKit ≠ iOS Safari: divergent rendering, memory limits, and CSS support

26. Contract tests (Pact-style) belong in which CI tier?
A) Only nightly/weekly: too slow for PRs
B) Replace e2e entirely
C) Tier 1 (consumer-side) on every PR; Tier 2 (provider verification) on provider PRs; Tier 3 (e2e) on integration
D) Only on release branches

27. What's the LCP target for Core Web Vitals to be considered 'Good' as of 2026?
A) Under 1 second
B) There is no fixed target
C) Under 4 seconds
D) Under 2.5 seconds

28. An authorisation-test matrix for 4 roles × 6 endpoints. To catch the bug where 'viewer' can delete users, which assertion type is essential?
A) Assert admin CAN delete (positive path)
B) Assert the endpoint exists
C) Assert viewer CANNOT delete (negative path expecting 403)
D) Assert all roles can read

29. What problem do branded types like `type UserId = string & { __brand: 'UserId' }` solve?
A) Performance: branded types are faster
B) Prevent the argument-swap bug where UserId and TenantId (both strings) get passed in wrong places
C) Convert TypeScript to runtime-typed
D) Replace zod validation

30. A NoSQL test on DynamoDB writes a record, then immediately reads it and gets stale data. Which fix is correct?
A) Use ConsistentRead: true on the read, OR poll with expect.poll until the read converges
B) Add waitForTimeout(500)
C) Switch to SQL
D) The test should be deleted; it's a flaky read

Answers: 1:B 2:B 3:A 4:A 5:D 6:A 7:B 8:D 9:D 10:A 11:B 12:A 13:B 14:A 15:D 16:B 17:B 18:D 19:C 20:B 21:C 22:C 23:C 24:A 25:D 26:C 27:D 28:C 29:B 30:A